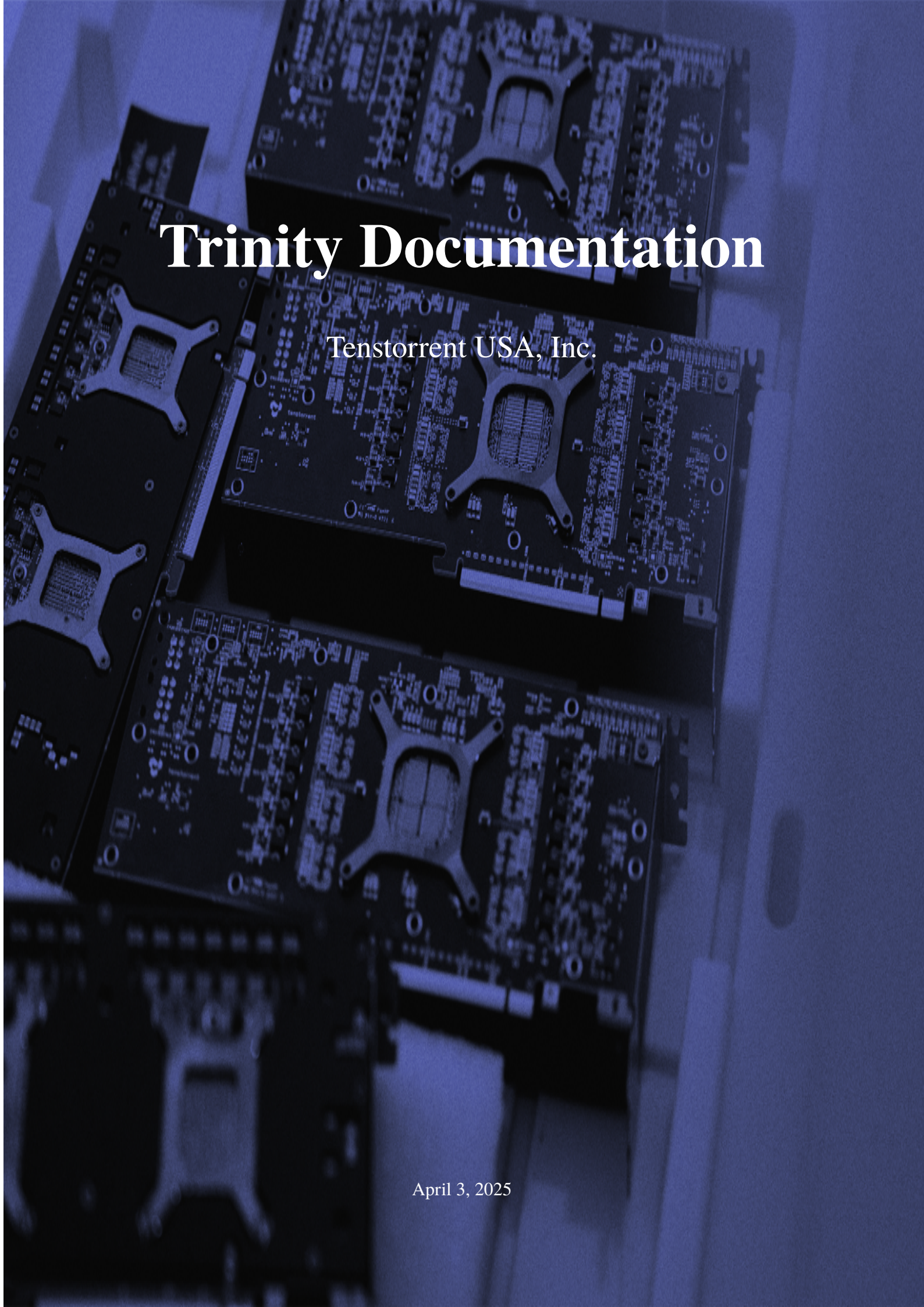




Trinity Documentation

Tenstorrent USA, Inc.

April 3, 2025

CONTENTS

1 Release Notes 3

1.1 Release Notes 3

2 Guides 9

2.1 Integration Guide 9

2.2 Synthesis and Floor-planning Guidelines 23

2.3 Tensix Neo Cluster User Manual 36

2.4 Verification Testbench 44

3 Register Maps 49

3.1 NOC2AXI Register Map Reference Guide 49

3.2 Tensix Debug Regs Memory map (per Tensix core) 70

3.3 Tensix NOC Register Map Reference Guide 102

3.4 Tensix Overlay Register Map Reference Guide 129

Welcome to the Trinity Documentation. Located below are release notes, guides, and detailed register maps.

RELEASE NOTES

1.1 Release Notes

1.1.1 Version 0.4.1

Additions on top of v0.4

- Additional Verification Tests:
 - Refer to the *Verification Documentation* for further details.
- Enhanced RTL:
 - Overlay reduced due to reduction in L2 and context-switch memory
 - Multicast , strided multicast support from NoC2AXI
- Revised Implementation Guidelines:
 - Updated SDC constraints.
 - Revised DEF files and partitioning.
 - For more information, consult the *Synthesis Documentation*.
- Memory Testbench Updates:
 - Some memory wrappers were modified , namely in the Overlay module . the memory testbench is updated to reflect this change.
- Updated Documentation:
 - update Tensix Register map
 - Update NoC2AXI register map
 - Added Overlay Register map
 - additional information is given regarding the IP in the integration manual

Limitations

- IO Ports are fixed
- The RTL is not fully verified
- Functional safety is not yet implemented
- NOC2AXI AXI interface does **not yet** support:
 - Fixed and Wrapping Burst
 - Sparse strobes (ie `wstrb = 0xaa`)

1.1.2 Version 0.4

Additions on top of v0.3

- Additional Verification Tests:
 - Refer to the [Verification Documentation](#) for further details.
- Enhanced RTL:
 - Inclusion of INT4 support.
 - Reduced FP16 capacity.
 - Preliminary support for ECC/Parity (not yet production-ready).
- Revised Implementation Guidelines:
 - Updated SDC constraints.
 - Revised DEF files and partitioning.
 - For more information, consult the [Synthesis Documentation](#).
- Memory Testbench Updates:
 - Removal of certain memories from the memory list.
- Updated Documentation:
 - Elimination of unused memories.
 - Addition of new primitives.
 - Description of new top-level signals.
 - Refinement of synthesis and verification documentation.

Limitations

- IO Ports are fixed
- The RTL is not fully verified
- Functional safety is not yet implemented
- NOC2AXI AXI interface does **not yet** support:
 - Fixed and Wrapping Burst
 - Sparse strobes (ie `wstrb = 0xaa`)

1.1.3 Version 0.3

Additions on top of v0.3

- Integration guide manual contains an updated memory list + primitive list
- Updated Memory testbench - see the [Memory Testbench Documentation](#) for more details
- Updated RTL - includes dispatch core, updated NoC2AXI, Updated Tensix clusters, Cluster count reduced to 12
- Updated implementation constraints

Contents

This release contains the following items:

- Documentation that describes the existing features in the IP.
- Unencrypted RTL - Not fully verified, but area accurate as per the POR minus extra features (INT4 , reduced FP16)
- A basic testbench that contains a single sanity test + documentation on how to run the test.
- A reference synthesis flow and some initial PD documentation
- Memory testbench to test proper memory integration.

Limitations

- Ports are ~95% fixed
- The RTL is not fully verified
- Functional safety is not yet implemented-3

3.

018269-

- INT4 support and reduced FP16 MACS is not yet implemented
- NOC2AXI AXI interface does **not yet** support:
 - Fixed and Wrapping Burst
 - Sparse strobes (ie `wstrb = 0xaa`)

EDA tool versions

- Synopsys VCS version V-2023.12-SP1-1
- Synopsys Verdi version V-2023.12-SP1-1
- Synopsys Synthesis version T-2022.03-SP5
- Synopsys Formality version T-2022.03-SP5
- Synopsys Spyglass version U-2023.03-SP1-1
- Python 3.9.18
- cocotb 1.9.0

file headers for open source packages :

As part of the Tenstorrent packaging process, we remove all comments from the source code. However, we make use of certain open-source libraries, including but not limited to APB, AXI interfaces, DMA package, Chipyard, and others. The list below provides the GitHub repositories from which these libraries are sourced. The user is hereby informed that the code included in the Tenstorrent release package should be regarded as originating from the specified GitHub repositories. Users are advised to consult the original files in these repositories for the applicable header information, licensing terms, and any restrictions.

all of the open source repositories that are used in the Trinity design come from : <https://github.com/pulp-platform> and <https://github.com/ucb-bar/chipyard>

Open Source repositories

- APB
- AXI
- AXI_STREAM
- COMMON_CELLS
- IDMA
- REGISTER_INTERFACE
- TECH_CELLS_GENERIC
- OBI
- CHIPYADS (Rocket cores)

1.1.4 Version 0.21

Additions on top of V0.2

- Integration guide manual contains memory list + primitive list
- Memory testbench - see the *Memory Testbench Documentation* for more details
- Encryption is now done separately for simulation and Synthesis. IEEE Std 1364-2005 encryption for simulation and using Synenc for encryption for synthesis.

Contents

Release contains the following items:

- Documentation that describes the existing features in the IP.
- Trinity RTL - Partially encrypted, where the ports of all design elements are left unencrypted. PD top levels, Trinity top level are not encrypted.
- Fully encrypted RTL in sv.e format for synthesis.
- A basic testbench that contains a single sanity test + documentation on how to run the test.
- Reference Synthesis flow and some initial PD documentation
- Memory testbench to test proper memory integration.

Limitations

- The top-level port list is subject to change
- Axi2NOC addressing is not yet implemented, Only Tensix Neo Cluster in position (0,0) can be accessed through the AXI ports.
- Binaries to load into the data movement cores are not available yet, hence outbound traffic is not supported.
- Dispatch cores + routers only are not available in the RTL
- Synthesis flows are provided to PD partitions only, not synthesis flow for op level in this release
- Tensix Neo cluster memory map is still subject to change

EDA Tool Version

This release was verified using the following tools :

- Synopsys VCS V-2023.12-SP1-1
- Synopsys Verdi V-2023.12-SP1-1
- Synopsys Design compiler T-2022.03-SP5
- Python 3.9.18
- cocotb 1.9.0

1.1.5 Version 0.2 (Integration Drop)

Contents

Release contains the following items:

- Documentation that describes the existing features in the IP.
- Trinity Initial RTL - Partially encrypted, where the ports of all design elements are left unencrypted. PD top levels, Trinity top level is not encrypted.
- A basic testbench that contains a single sanity test + documentation on how to run the test.
- Reference Synthesis flow and some initial PD documentation

Limitations

- The top-level port list is subject to change
- Axi2NOC addressing is not yet implemented, Only Tensix Neo Cluster in position (0,0) can be accessed through the AXI ports.
- Binaries to load into the data movement cores are not available yet, hence outbound traffic is not supported.
- Dispatch cores + routers only are not available in the RTL
- Synthesis flows are provided to PD partitions only, not synthesis flow for op level in this release
- Tensix Neo cluster memory map is still subject to change
- The release does not include a stand-alone memory testbench

EDA Tool Version

This release was verified using the following tools :

- Synopsys VCS V-2023.12-SP1-1
- Synopsys Design compiler V-2023.12-SP3
- Python 3.9.18
- cocotb 1.9.0

1.1.6 Version 0.1 (Preliminary)

Contents

Release contains the following items:

- Initial Documentation: Tensix User Manual, Initial integration manual
- Top-level Wrapper with IOs
- Initial floor planning Ideas (Based on Quasar project)

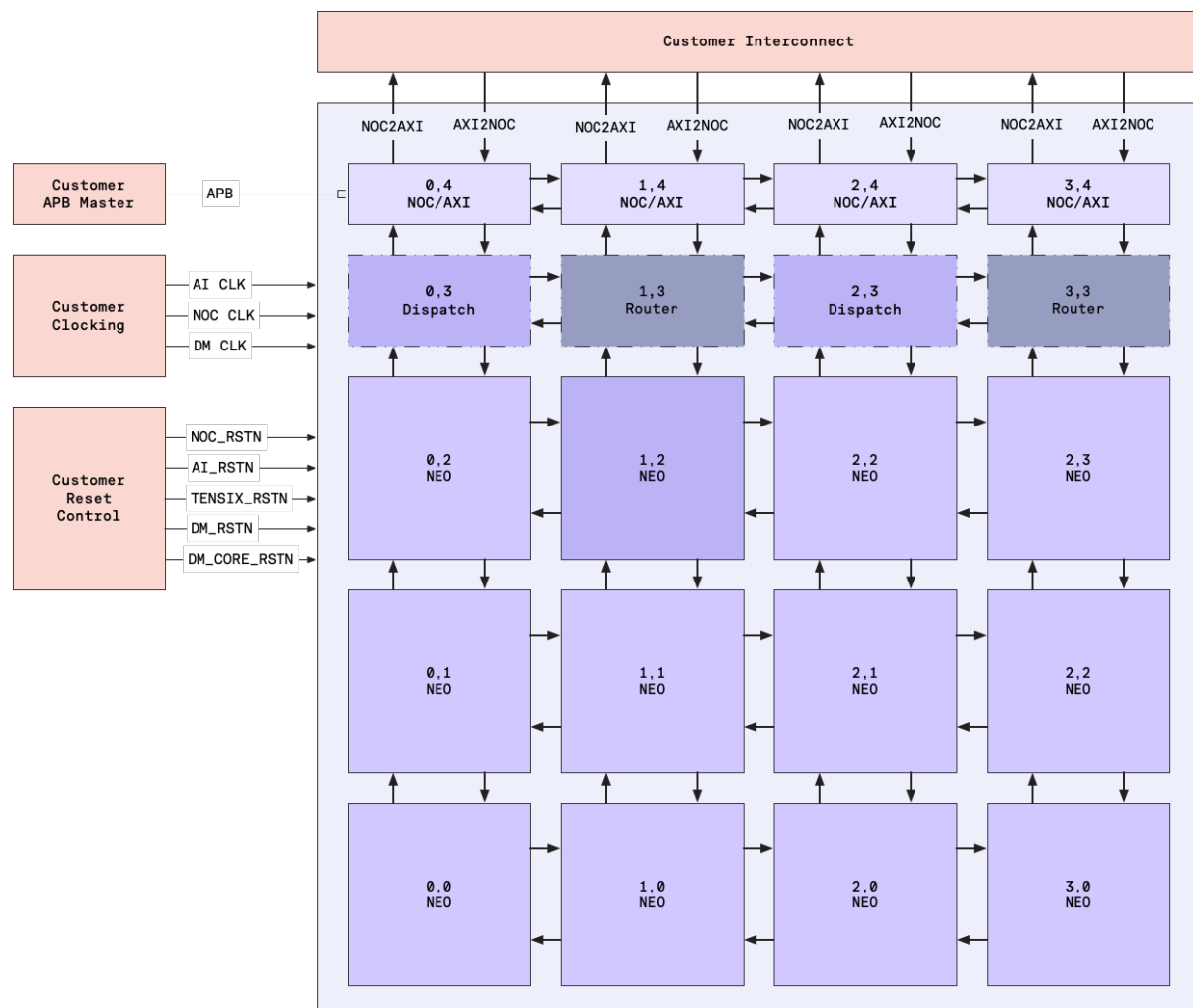
Limitations

- Everything that is delivered in this release package is not final and is subject to change.

2.1 Integration Guide

2.1.1 Logical IP Integration

The Trinity IP consists of a 4x5 grid of endpoints connected by a 256-byte bidirectional NoC mesh. The lower three rows (indices 0–2) contain Tensix NEO clusters. The fourth row (index 3) above the Tensix clusters includes two dispatch cores and two NoC routers, ensuring mesh connectivity. The topmost row hosts four NOC2AXI endpoints, each providing 64-byte AXI interfaces to the external interconnect.



Tensix Trinity Components

1. Tensix Trinity Clusters:

These are the compute blocks of the NPU. Each cluster contains:

- 4 Compute Engines (Tensix cores)
- A shared 3MB memory
- A NoC Interface Unit
- A Data Movement Element (Overlay)

For more details, refer to the [Tensix Manual](#).

2. Dispatch Engine:

These blocks handle fast kernel dispatch to the Tensix clusters. They pre-fetch and queue kernels to mitigate Host-to-IP latency, dispatching them to the clusters as soon as they are ready. A Dispatch Engine is similar to a Tensix cluster in structure but excludes the compute engines. It consists of:

- A Data Movement Element (Overlay)
- A NoC Interface Unit
- A shared 3MB memory

3. Router-Only Blocks:

These blocks ensure full mesh connectivity by routing NoC packets through their coordinates when no Tensix or Dispatch Engine is present. They consist solely of a NoC Interface Unit.

4. NOC2AXI Interfaces:

These blocks manage communication between the IP and the rest of the SoC or Host. All traffic between the Host, DRAM, and the NPU flows through the NOC2AXI interfaces. Each interface includes:

- 2 AXI Ports (Manager and Subordinate)
- An APB subordinate Port for NOC2AXI register configuration

For more details, refer to the [NOC2AXI Register Map](#).

2.1.2 Clocking & Reset

Clock Domains

Each major component within the Trinity IP operates on its own clock domain. All clock domains are asynchronous relative to each other. The main clocks are:

Clock	Frequency (MHz)	Purpose
ai_clk	1000–1500	Operates Tensix math units and SRAM. Impacts peak Tensix math Bandwidth
noc_cl	1000–1500	Drives the NoC; Impacts Tensix-to-Tensix and Tensix-to-NOC2AXI bandwidth.
axi_cl	750–1500	AXI domain for external interconnect traffic. Synchronization between axi_clk and noc_clk is handled inside NOC2AXI.
dm_clk	1500–2200	Data Movement CPU clock. Affects data movement instruction rate.

Resets

The Trinity IP and its AXI interface utilize synchronous, active-low resets throughout the design. These resets may be delivered to each tile asynchronously, as the reset inputs are synchronized to the correct clock domain internally. The reset signals include:

- `i_noc_reset_n`: Common NoC reset for all NEO clusters.
- `i_tensix_reset_n`: One per Tensix core (4 per NEO cluster).
- `i_dm_core_reset_n`: One per data movement core (8 per DM module).
- `i_dm_uncore_reset_n`: One reset for non-core DM logic per module.
- `i_ai_reset_n`: One reset for L1 memory and shared logic in NEO clusters.

Reset and Clock Sequencing

Because each tile is physically large, it is challenging to meet setup time requirements on reset-deassertion from the reset synchronizer to all resettable endpoints. Accordingly, it is recommended that the integrator add a multi-cycle-path from reset de-assertion. This multi-cycle path imposes a constraint on clock frequencies when de-asserting reset. To ensure proper operation of the Trinity IP, follow the reset and clock sequencing steps outlined below:

1. Assert all resets, and ensure that the clocks are running at a frequency sufficiently low to meet any multi-cycle path requirements.
2. De-assert `i_noc_reset_n`.
3. De-assert `i_ai_reset_n` (allows data transfer to/from Tensix SRAM).
4. De-assert `i_dm_uncore_reset_n` (enables DM subsystem configuration).
5. De-assert `i_tensix_reset_n` (starts Tensix cores).
6. De-assert `i_dm_core_reset_n` (DM cores start fetching instructions).

Once each reset has been de-asserted, clock frequencies may be ramped to their mission-mode frequencies

2.1.3 NOC2AXI Interfaces

AXI to NOC Interface (AXI Subordinate Interface)

The AXI to NOC interface supports standard AXI4 signals. The `region`, `qos`, `cache` and `prot`, and `lock` fields are not used by this interface.

AXI Signal Properties

Each AXI to NOC interface has the following signal widths

AXI Pa- rameter	Bit- Width	Notes
Address	56	
Data	512	
ID	12	
User	69	User bits map to inbound AXI transactions to NOC special functions, which are outlined in the table below. The mappings are subject to change in a future release

AXI to NOC User Bit Map

For detailed information on the various attributes of a NoC transaction that these user bits correspond to, please refer to the *Tensix Manual*.

User Field	Bit Index (MSB:L)	Description
sec_source	[3:0]	Security source ID
sec_group	[7:4]	Security group ID
cce_cmd	[8]	TODO
x_offset	[14:9]	The rectangle size on the X dimension of the broadcast range
y_offset	[20:15]	The rectangle size on the Y dimension of the broadcast range
cmd_resp	[21]	Marks transaction as non-posted (default is posted)
cmd_brcs	[22]	Marks the transaction as a broadcast transaction
cmd_link	[23]	Mark the transaction as part of a linked sequence transactions
cmd_forc	[24]	Forces the router to use standard dimension-order routing instead of Up-Down-Mesh routing (default)
cmd_forc	[25]	When set, the VC used by the resulting transaction would come from the following user bits. If not set, the VC is chosen based on the AXI ID.
cmd_stat	[29:26]	Selects the virtual channel (VC) for requests. $vc_req = \sim cmd_force_user_bits_vc ? (i_axi_id[3:0] \% 12) : cmd_static_vc$
cmd_resp	[33:30]	Selects the VC used for responses. $vc_resp = \sim cmd_force_user_bits_vc ? (12 + i_axi_id[1:0]) : cmd_resp_static_vc$
cmd_pkt	[37:34]	Sets the packet ID for diagnostics purposes
cmd_stric	[38]	Enforces strict ordering of transactions for AXI compliance purposes by stalling the next transaction until the previous one with the same AXI ID is completed. This ensures that the ordering on the AXI channel is maintained. If this is not set, the AXI protocol might not enforce ID ordering.
cmd_flush	[39]	Flushes the transactions currently in the buffer between the NoC and Tensix, ensuring all transactions are committed. Any subsequent request will be blocked until the flush is complete.
brcst_xy	[40]	Selects between X or Y direction for multicast. '1' - y first, '0' - X first
brcst_src	[41]	Includes source node in the broadcast. usually set to 0 when used in NoC2AXI
brcst_qua	[49:42]	Excludes specified X coordinates from the broadcast
brcst_qua	[57:50]	Excludes specified Y coordinates from the broadcast
brcst_qua	[58]	Excludes broadcasts in the X direction
brcst_qua	[59]	Excludes broadcasts in the Y direction
brcst_qua	[60]	Enables excluding certain coordinates/directions in a broadcast
brcst_stri	[62:61]	Defines how many nodes to keep in the X direction for a strided multicast
brcst_stri	[64:63]	Defines how many nodes to skip in the X direction for a strided multicast
brcst_stri	[66:65]	Defines how many nodes to keep in the Y direction for a strided multicast
brcst_stri	[68:67]	Defines how many nodes to skip in the Y direction for a strided multicast

Table Note 1.: The user is responsible to set the request VC between 0-7, broadcast between 8-11, and response between 12-15. You must set the response VC when forcing a VC on the request side. if VCs are not put between these values respectively, the IP behavior becomes unpredictable

Interface Properties

Property	Value
Maximum Clock Frequency	~1.5 GHz (technology-dependent)
Max Outstanding Transactions	32
Supported Burst Types	INCR
Unique ID Transactions	Yes

Restrictions and Behavioral Notes:

1. **No Sparse Write-Strobes for Multi-Beat Writes:**

For bursts longer than one beat ($axlen > 0$), $wstrb$ must indicate a contiguous sequence of bytes from the lowest to highest address. No “gaps” are allowed. For single-beat writes ($wstrb=0$), any $wstrb$ pattern is permissible.

2. **Transaction Data Availability:**

Write data transfer to the NoC does not start until all write beats for a given AXI transaction (up to 4kB) are received and buffered in NOC2AXI. Similarly, read data is only returned after the entire read burst has been fetched and buffered. Partial data availability does not initiate partial transfers.

3. **Read Ordering:**

All read transactions complete in the order in which they are received. Transactions that complete early on the NoC side must still wait for previously-issued transactions to be fully completed before their data or responses are returned.

NOC to AXI Interface (AXI Manager Interface)

AXI Signal Properties

AXI Sig-nal	Bit-Width	Notes
Ad-dress	56	Address Bus is meant to provide a way to access each Tensix core by applying the local address + uni/multicast x/y ranges on the extra 24 bits. This will change to a more concise address map by using internal address map configuration in a future release
Data	512	
ID	4	Read transactions use a consistent ID; Writes may use up to 16 unique IDs
User	9	User bits map to inbound NOC transactions to AXI special functions. These bits are currently reserved, and will eventually be used to encode security information.

User Signal Map

Field Name	Bit Range	Description
cce_cmd_snoop	[0:0]	CCE cmd snoop for this flit
sec_group_id	[4:1]	Security group ID for this flit
sec_source_id	[8:5]	Security source ID for this flit

Interface Properties

Property	Value
Maximum clock frequency	~1.5 GHz (technology dependent)
Max Number of Outstanding Transactions	64 (most likely to increase in later release)
Max number of out-of-order transactions	N/A
Issuing burst types (Incr/Fixed)	INCR
Average burst length	Max Burst Length is 255 (or 4kB)
Maximum number of unique IDs	Based on NOC Virtual Channel (increased to 32 in Neo)
Narrow Burst support	True, but aligned on 16B
Expected Transaction Patterns	In order, non-posted, bursty traffic (transfer sizes >1kB)

Restrictions and Behavioral Notes:

1. *Read Ordering

All read transactions issued on the AXI interface use the same address read ID (ARID) value of zero. Therefore all read data must be returned in the order in which the read requests are issued.

NOC2AXI Configuration Interface

Each of the four NOC2AXI links includes a 32-bit APB3 interface for CSR configuration. This interface is used to:

- Program the global address map, mapping global addresses to physical grid coordinates.
- Program NoC routes per VC + targetter endpoint
- Configure NOC2AXI settings, such as clock gating and other operational parameters.

This interface is clocked on `axi_clk`.

Addressing Scheme

Each NOC2AXI instance includes two Address Translation Tables (ATTs) and a TLB on its subordinate port. These components determine how incoming transactions are processed:

Address Translation Table (ATT)

The Address Translation Table (ATT) allows AXI transactions to be routed efficiently through the NoC. When enabled, AXI transactions flow through the ATT, which uses the transaction address (`awaddr` or `araddr`) to determine:

- **Target(s):** The physical coordinates and address that the transaction is routed to.
- **Virtual Channel (VC):** The NoC virtual channel to use for the transaction.
- **Route:** The path the transaction takes to reach its target(s).

Default Behavior (ATT Disabled)

If the ATT is disabled, inbound AXI traffic is routed to physical coordinates using the default scheme:

```
[6 bits y, 6 bits x, 40 bits local address]
```

For example, to send traffic to the Tensix core at (2, 3), the address would be:

```
const uint64_t DEFAULT_X_OFFSET = 40;
const uint64_t DEFAULT_Y_OFFSET = 46;

uint64_t get_tensix_address(uint8_t x, uint8_t y, uint64_t local_tensix_address) {
    return (3 << DEFAULT_Y_OFFSET) | (2 << DEFAULT_X_OFFSET) | local_tensix_address;
}
```

ATT Components

The ATT consists of the following key components:

1. Mask Table:

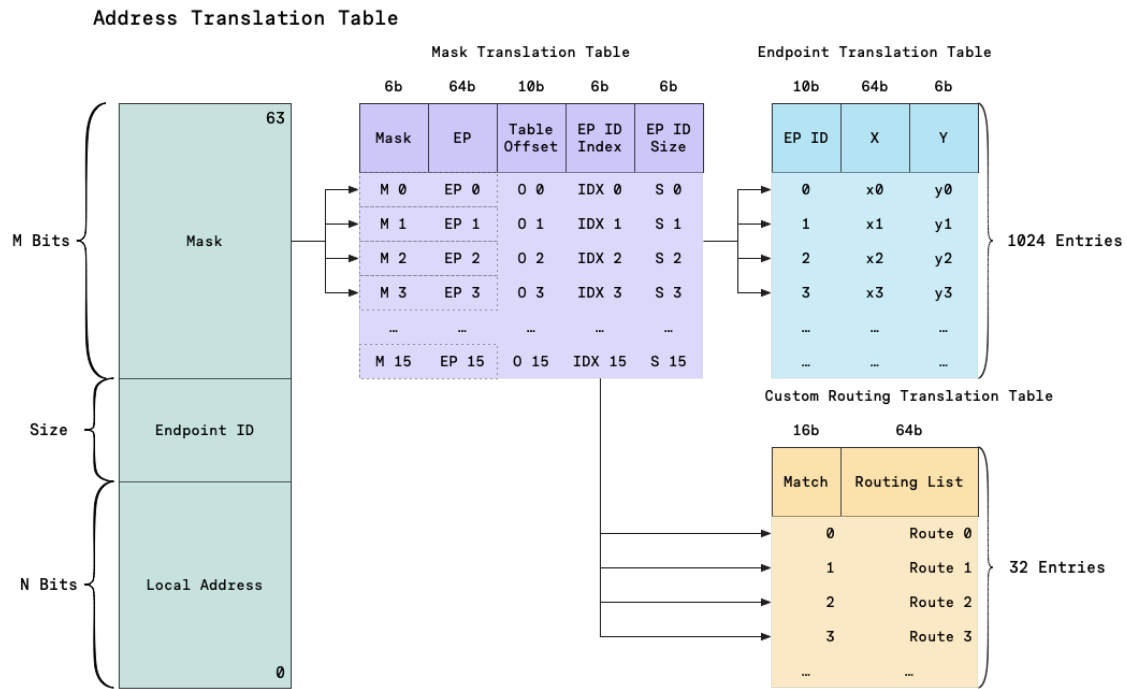
- Contains 16 entries.
- Matches incoming addresses to an Endpoint Table Index (IDX).

2. Endpoint Table:

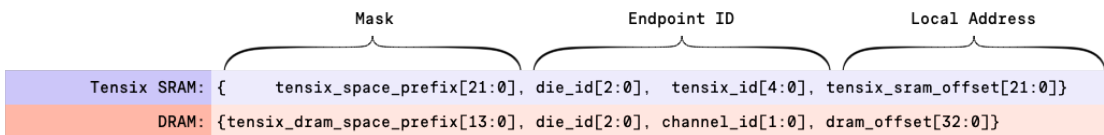
- Contains 1024 entries.
- Maps Endpoint Table IDX to physical NoC coordinates.

3. Routing Table:

- Contains 32 entries.
- Maps {Endpoint Table IDX, Virtual Channel} to a specific route.



Address Translation Example



Mask Table

Each Mask Table entry determines how an incoming address is matched and translated. The fields in each entry include: Masked Address = Address & $\sim((1 \ll (\text{MASK})) - 1)$

Example: If MASK = 54, the masked address will use {Address[55:54], 53{1'b0}}.

- **EP (Endpoint):**

The comparison value for matching: Match = Masked Address == EP

Note: Ensure that only one entry matches any given address. Overlapping entries (e.g., 0x400_0000_0000 with MASK = 22 and 0x700_0000_0000 with MASK = 24) must be avoided or use the more specific mask (e.g., MASK = 24).

- **EP ID IDX / EP ID SIZE:**

Specifies where to extract the Endpoint ID from the address: EP ID = Address[EP ID IDX + : EP ID SIZE]

- **Table Offset:**

Offset applied to the Endpoint Table IDX: Endpoint Table IDX = EP ID + Table Offset

- **Translate Addr:**

Specifies whether to rebase the address (0 = No, 1 = Yes).

- **New Base:**

If rebasing is enabled, specifies the new base address for translation. Rebased Address = Address[EP ID IDX:0] + New Base

APB Register interface

Each NoC2AXI instance has a local register interface via an APB bus. APB transactions are received through this port, and can target either a small set of NOC2AXI local registers or the NIU register space.

ATT Programming Guidelines

1. Determine Endpoint Addressing (Non-Sparse Address Map):

- Calculate the number of bits required to address all endpoints:

$$N = \text{clog2}(\text{Number of accessible NoC coordinates})$$

- Set EP_ID_IDX to the starting index of these N bits.
- Set EP_ID_SIZE to N.

For example, in Trinity:

- Total endpoints = 20 (12 Tensix clusters, 2 Dispatch, 2 Router-only, 4 NOC2AXI).
- $N = \text{clog2}(20) = 5$, so EP ID SIZE = 5.

2. Populate the Endpoint Table:

- For each match in the Mask Table, allocate corresponding entries in the Endpoint Table.

3. Optional Step: Enable Address Rebase:

- Set Address Translation to 1.
- Program the new_base field with the desired address offset.

4. Set Endpoint Coordinates:

- Populate the X and Y fields in ENDPOINT_TABLE_ENTRY[i] with NoC coordinates.
- Use the index generated by the Mask Table to fetch the relevant endpoint.
git stash- Coordinates do not need to be contiguous with the index.

Note: Each entry requires a single register write.

5. Configure Dynamic Routes:

- For non-standard routing (not Y-X), program dynamic routes:
 - Write the comparison value {Endpoint Table IDX, VC} to ROUTING_TABLE_MATCH[i]. -
Each Match register bit positions are : { 10 bits of endpoint index , 4 bit for VC }
 - Write the 128-byte route (32 x 32-bit entries) to ROUTING_TABLE_PART_ENTRY[i].
 - * Each 5-bit LSB of the 32 entries specifies the route direction for each node in the NoC.
 - * The following directions {West, South, East, North, NIU} are one-hot encoded (5 bits) for each node.
 - * LSB corresponds to node (0,0).
 - * All 32 registers of a route must be programmed if a route is to be used. Partial programming is not supported.

IP Ports & Port Descriptions

This section describes the ports of the three main IP components in a Tensix grid: the top-level wrapper, Tensix cores, and npu_out blocks.

Most signals are dimensioned for four parallel npu_out instances. This means each signal has four parallel instances, with each corresponding to a separate npu_out.

2.1.4 IP Ports & Port Descriptions

This section describes the ports on the three IPs that make up a Tensix grid, the top-level wrapper, the Tensix and the npu_out blocks. Most signals in the following table have their outermost dimension set to 4 because these signals are designed to be split or distributed to four different npu_out instances. This means that for each signal, there are four parallel instances, each corresponding to a separate npu_out.

Trinity Top-Level Port List

In the following table, SizeX and SizeY are the dimensions of the Tensix grid. For Trinity, SizeX is 4 and SizeY is 6.

Signal Name	Direction	Size (bits)	Description
i_axi_clk	Input	1	AXI clock signal, routed to all each NOC2AXI core
i_noc_clk	Input	1	NOC clock signal, routed to all NOC2AXI, NEO and
i_noc_reset_n	Input	1	NOC reset signal (active low), routed to all NOC2AXI
i_ai_clk	Input	1	AI clock signal, routed to all NEO and Dispatch cores
i_ai_reset_n	Input	1	AI reset signal (active low). routed to all NEO and Di
i_tensix_reset_n	Input	[SizeX-1:0] [SizeY-2:0] [3:0]	Tensix reset signals per cluster (active low).
i_dm_clk	Input	1	Data Movement (DM) clock signal.
i_dm_core_reset_n	Input	[SizeX-1:0] [SizeY-2:0] [7:0]	DM core reset signals per cluster (active low).
i_dm_uncore_reset_n	Input	[SizeX-1:0] [SizeY-2:0]	DM uncore reset signals per cluster (active low).
o_err	Output	[SizeX-1:0] [SizeY-1:0] [3:0]	Error signals per cluster.
i_security_fence_en	Input	1	Security fence enable signal
i_mem_config	Input	tt_mem_ctrl_pkg::mem_cfg_t	Customizable memory configuration structure for setti
i_reg_psel	Input	[SizeX]	APB interface for each NOC2AXI Interface.
i_reg_paddr	Input	[SizeX] [31:0]	APB interface for each NOC2AXI Interface.
i_reg_penable	Input	[SizeX]	APB interface for each NOC2AXI Interface.
i_reg_pwrite	Input	[SizeX]	APB interface for each NOC2AXI Interface.
i_reg_pwdata	Input	[SizeX] [31:0]	APB interface for each NOC2AXI Interface.
o_reg_pready	Output	[SizeX]	APB interface for each NOC2AXI Interface.
o_reg_prdata	Output	[SizeX] [31:0]	APB interface for each NOC2AXI Interface.
o_reg_pslverr	Output	[SizeX]	APB interface for each NOC2AXI Interface.
npu_out_awvalid	Output	[SizeX]	AXI interface from NPU to interconnect, one per NOC
npu_out_awid	Output	[SizeX] [3:0]	AXI interface from NPU to interconnect, one per NOC
npu_out_awaddr	Output	[SizeX] [55:0]	AXI interface from NPU to interconnect, one per NOC
npu_out_awlen	Output	[SizeX] [7:0]	AXI interface from NPU to interconnect, one per NOC
npu_out_awsz	Output	[SizeX] [2:0]	AXI interface from NPU to interconnect, one per NOC
npu_out_awburst	Output	[SizeX] [1:0]	AXI interface from NPU to interconnect, one per NOC
npu_out_awlock	Output	[SizeX]	AXI interface from NPU to interconnect, one per NOC
npu_out_awcache	Output	[SizeX] [3:0]	AXI interface from NPU to interconnect, one per NOC
npu_out_awprot	Output	[SizeX] [2:0]	AXI interface from NPU to interconnect, one per NOC
npu_out_awqos	Output	[SizeX] [3:0]	AXI interface from NPU to interconnect, one per NOC
npu_out_awregion	Output	[SizeX] [3:0]	AXI interface from NPU to interconnect, one per NOC
npu_out_awuser	Output	[SizeX] [8:0]	AXI interface from NPU to interconnect, one per NOC
npu_out_awready	Input	[SizeX]	AXI interface from NPU to interconnect, one per NOC
npu_out_wvalid	Output	[SizeX]	AXI interface from NPU to interconnect, one per NOC

Table 1 – continued from previous page

Signal Name	Direction	Size (bits)	Description
npu_out_wdata	Output	[SizeX] [511:0]	AXI interface from NPU to interconnect, one per NOC
npu_out_wstrb	Output	[SizeX] [63:0]	AXI interface from NPU to interconnect, one per NOC
npu_out_wlast	Output	[SizeX]	AXI interface from NPU to interconnect, one per NOC
npu_out_wready	Input	[SizeX]	AXI interface from NPU to interconnect, one per NOC
npu_out_bvalid	Input	[SizeX]	AXI interface from NPU to interconnect, one per NOC
npu_out_bid	Input	[SizeX] [3:0]	AXI interface from NPU to interconnect, one per NOC
npu_out_bresp	Input	[SizeX] [1:0]	AXI interface from NPU to interconnect, one per NOC
npu_out_bready	Output	[SizeX]	AXI interface from NPU to interconnect, one per NOC
npu_out_arvalid	Output	[SizeX]	AXI interface from NPU to interconnect, one per NOC
npu_out_arid	Output	[SizeX] [3:0]	AXI interface from NPU to interconnect, one per NOC
npu_out_araddr	Output	[SizeX] [55:0]	AXI interface from NPU to interconnect, one per NOC
npu_out_arlen	Output	[SizeX] [7:0]	AXI interface from NPU to interconnect, one per NOC
npu_out_arsize	Output	[SizeX] [2:0]	AXI interface from NPU to interconnect, one per NOC
npu_out_arburst	Output	[SizeX] [1:0]	AXI interface from NPU to interconnect, one per NOC
npu_out_arlock	Output	[SizeX]	AXI interface from NPU to interconnect, one per NOC
npu_out_aruser	Output	[SizeX] [8:0]	AXI interface from NPU to interconnect, one per NOC
npu_out_arcache	Output	[SizeX] [3:0]	AXI interface from NPU to interconnect, one per NOC
npu_out_arprot	Output	[SizeX] [2:0]	AXI interface from NPU to interconnect, one per NOC
npu_out_arqos	Output	[SizeX] [3:0]	AXI interface from NPU to interconnect, one per NOC
npu_out_arregion	Output	[SizeX] [3:0]	AXI interface from NPU to interconnect, one per NOC
npu_out_arready	Input	[SizeX]	AXI interface from NPU to interconnect, one per NOC
npu_out_rvalid	Input	[SizeX]	AXI interface from NPU to interconnect, one per NOC
npu_out_rid	Input	[SizeX] [3:0]	AXI interface from NPU to interconnect, one per NOC
npu_out_rdata	Input	[SizeX] [511:0]	AXI interface from NPU to interconnect, one per NOC
npu_out_rresp	Input	[SizeX] [1:0]	AXI interface from NPU to interconnect, one per NOC
npu_out_rlast	Input	[SizeX]	AXI interface from NPU to interconnect, one per NOC
npu_out_rready	Output	[SizeX]	AXI interface from NPU to interconnect, one per NOC
npu_in_awvalid	Input	[SizeX]	AXI interface from interconnect to NPU one per NOC
npu_in_awid	Input	[SizeX] [11:0]	AXI interface from interconnect to NPU one per NOC
npu_in_awaddr	Input	[SizeX] [55:0]	AXI interface from interconnect to NPU one per NOC
npu_in_awlen	Input	[SizeX] [7:0]	AXI interface from interconnect to NPU one per NOC
npu_in_awsiz	Input	[SizeX] [2:0]	AXI interface from interconnect to NPU one per NOC
npu_in_awburst	Input	[SizeX] [1:0]	AXI interface from interconnect to NPU one per NOC
npu_in_awlock	Input	[SizeX]	AXI interface from interconnect to NPU one per NOC
npu_in_awcache	Input	[SizeX] [3:0]	AXI interface from interconnect to NPU one per NOC
npu_in_awprot	Input	[SizeX] [2:0]	AXI interface from interconnect to NPU one per NOC
npu_in_awqos	Input	[SizeX] [3:0]	AXI interface from interconnect to NPU one per NOC
npu_in_awregion	Input	[SizeX] [3:0]	AXI interface from interconnect to NPU one per NOC
npu_in_awuser	Input	[SizeX] [68:0]	AXI interface from interconnect to NPU one per NOC
npu_in_awready	Output	[SizeX]	AXI interface from interconnect to NPU one per NOC
npu_in_wvalid	Input	[SizeX]	AXI interface from interconnect to NPU one per NOC
npu_in_wdata	Input	[SizeX] [511:0]	AXI interface from interconnect to NPU one per NOC
npu_in_wstrb	Input	[SizeX] [63:0]	AXI interface from interconnect to NPU one per NOC
npu_in_wlast	Input	[SizeX]	AXI interface from interconnect to NPU one per NOC
npu_in_wready	Output	[SizeX]	AXI interface from interconnect to NPU one per NOC
npu_in_bvalid	Output	[SizeX]	AXI interface from interconnect to NPU one per NOC
npu_in_bid	Output	[SizeX] [12:0]	AXI interface from interconnect to NPU one per NOC
npu_in_bresp	Output	[SizeX] [1:0]	AXI interface from interconnect to NPU one per NOC
npu_in_bready	Input	[SizeX]	AXI interface from interconnect to NPU one per NOC
npu_in_arvalid	Input	[SizeX]	AXI interface from interconnect to NPU one per NOC
npu_in_arid	Input	[SizeX] [11:0]	AXI interface from interconnect to NPU one per NOC
npu_in_araddr	Input	[SizeX] [55:0]	AXI interface from interconnect to NPU one per NOC
npu_in_arlen	Input	[SizeX] [7:0]	AXI interface from interconnect to NPU one per NOC
npu_in_arsize	Input	[SizeX] [2:0]	AXI interface from interconnect to NPU one per NOC
npu_in_arburst	Input	[SizeX] [1:0]	AXI interface from interconnect to NPU one per NOC

Table 1 – continued from previous page

Signal Name	Direction	Size (bits)	Description
npu_in_arlock	Input	[SizeX]	AXI interface from interconnect to NPU one per NOC
npu_in_arcache	Input	[SizeX] [3:0]	AXI interface from interconnect to NPU one per NOC
npu_in_arprot	Input	[SizeX] [2:0]	AXI interface from interconnect to NPU one per NOC
npu_in_arqos	Input	[SizeX] [3:0]	AXI interface from interconnect to NPU one per NOC
npu_in_arregion	Input	[SizeX] [3:0]	AXI interface from interconnect to NPU one per NOC
npu_in_aruser	Input	[SizeX] [68:0]	AXI interface from interconnect to NPU one per NOC
npu_in_arready	Output	[SizeX]	AXI interface from interconnect to NPU one per NOC
npu_in_rvalid	Output	[SizeX]	AXI interface from interconnect to NPU one per NOC
npu_in_rid	Output	[SizeX] [11:0]	AXI interface from interconnect to NPU one per NOC
npu_in_rdata	Output	[SizeX] [511:0]	AXI interface from interconnect to NPU one per NOC
npu_in_rresp	Output	[SizeX] [1:0]	AXI interface from interconnect to NPU one per NOC
npu_in_rlast	Output	[SizeX]	AXI interface from interconnect to NPU one per NOC
npu_in_rready	Input	[SizeX]	AXI interface from interconnect to NPU one per NOC

2.1.5 Technology-Specific Considerations

Memories

The table below outlines the required memories, the type of memory, and their sizes (in columns and rows). Behavioral models for each memory are located in the \$TENSIX_ROOT/rtl/primitives/wrappers/ pathh. These modules should not be synthesized. The customer is responsible for generating the appropriate memory configuration files for the Tensix and NOC2AXI blocks in the IP.

Memory Name	En-tries	Word Width	Write Enable (bit)	Single / Two Port	Usecase	Tensix Neo (Quasar) reference implemet-nation			
tt_mem_wrap_1024x12_2p_n	1024	12	False	Two Port	NoC End-point Trans-lation	single instance	bank	per	in-
tt_mem_wrap_32x1024_sp_n	32	1024	False	Single Port	Overlay Con-text Switch	8 instances of 32x128b			
tt_mem_wrap_32x152_sp_w	32	152	True	Single Port	L2 Directory	single instance	bank	per	in-
tt_mem_wrap_1024x52_sp_w	1024	52	True	Single Port	TRISC Local Memory	single instance	bank	per	in-
tt_mem_wrap_128x144_sp_w	128	144	True	Single Port	Data Arrays (Ext)	single instance	bank	per	in-
tt_mem_wrap_128x256_sp_n	128	256	False	Single Port	L2 Data	2 instances of 128x128b			
tt_mem_wrap_256x104_sp_w	256	104	True	Single Port	TRISC Local Memory	single instance	bank	per	in-
tt_mem_wrap_256x66_sp_w	256	66	True	Single Port	Data Arrays (0_0 Ext)	single instance	bank	per	in-
tt_mem_wrap_32x100_sp_w	32	100	True	Single Port	Tag Array (Ext)	single instance	bank	per	in-
tt_mem_wrap_32x1024_2p_n	32	1024	False	Two Port	NoC Routing Translation	8 instances of 32x128b			
tt_mem_wrap_32x86_sp_w	32	86	True	Single Port	Tag Array (0 Ext)	single instance	bank	per	in-
tt_mem_wrap_64x2048_2p_n	64	2048	False	Two Port	AXI to NoC Buffer	-			
tt_mem_wrap_64x2048_2p_n	64	2048	False	Two Port	Router Input Port	4 instances of 64x128b			
tt_mem_wrap_64x512_2p_n	64	512	False	Two Port	NoC RoCC Command Buffer	4 instances of 64x128b			
tt_mem_wrap_64x516_2p_n	64	516	False	Two Port	NoC to AXI Slave Read Data	-			
tt_mem_wrap_72x2048_2p_n	72	2048	False	Two Port	Router Input Port	-			
tt_mem_wrap_768x128_sp_n	768	128	False	Single Port	T6 L1 Cache	single instance	bank	per	in-
tt_mem_wrap_3072x128_sp_n	3072	128	False	Single Port	T6 L1 Cache	single instance	bank	per	in-
tt_mem_wrap_512x72_sp_w	512	72	True	Single Port	TRISC ICache	single instance	bank	per	in-

Memory Settings

To accommodate arbitrary memory macro configuration bits, we provide a customizable Verilog structure, `mem_cfg_t`, defined in `tt_mem_ctrl_pkg.sv`. This structure is broadcast to all memory wrappers, enabling a flexible mechanism for setting memory margins and other parameters, such as low-power entry.

Primitive Cells

The table below provides a list of the technology primitives used in the Tensix and NOC2AXI IP. The customer is responsible for replacing the behavioral code with the appropriate technology-specific primitives. Similar to the behavioral memory wrappers, the primitive behavioral models can be used as a reference to formally verify that the primitives have been replaced correctly.

these primitives could be found in `$TENSIX_ROOT/rtl/primitives/misc`

File Name	Description
<code>tt_libcell_aoi22_quad.sv</code>	Quad AOI22 gate
<code>tt_libcell_aoi22.sv</code>	AOI22 gate
<code>tt_libcell_clkand2.sv</code>	AND gate with clock input
<code>tt_libcell_clkbuf.sv</code>	Clock buffer
<code>tt_libcell_clkgate.sv</code>	Clock gate
<code>tt_libcell_clkinv.sv</code>	Clock inverter
<code>tt_libcell_clkmux2.sv</code>	2-to-1 clock multiplexer
<code>tt_libcell_clknand2.sv</code>	NAND gate with clock input
<code>tt_libcell_clkor2.sv</code>	OR gate with clock input
<code>tt_libcell_clkxor2.sv</code>	XOR gate with clock input
<code>tt_libcell_dffrxq.sv</code>	D flip-flop with active-low reset and Q output
<code>tt_libcell_dffsrqxq.sv</code>	D flip-flop with active-low set/reset and Q output
<code>tt_libcell_dff.sv</code>	D flip-flop
<code>tt_libcell_dffsxq.sv</code>	D flip-flop with active-low set and Q output
<code>tt_libcell_fulladder.sv</code>	Full adder
<code>tt_libcell_halfadder.sv</code>	Half adder
<code>tt_libcell_lhcnqd4.sv</code>	Latch with clear and Q output
<code>tt_libcell_metastab_hardened_dffr.sv</code>	Metastability-hardened D flip-flop with active-low reset
<code>tt_libcell_metastab_hardened_dff.sv</code>	Metastability-hardened D flip-flop
<code>tt_libcell_nd2.sv</code>	2-input NAND gate
<code>tt_libcell_nd3.sv</code>	3-input NAND gate
<code>tt_libcell_nd4.sv</code>	4-input NAND gate
<code>tt_libcell_or2.sv</code>	2-input OR gate
<code>tt_libcell_stdbuf.sv</code>	Standard buffer
<code>tt_libcell_stdinv.sv</code>	Standard inverter
<code>tt_libcell_stdmux2.sv</code>	2-to-1 multiplexer
<code>tt_libcell_stdmux3.sv</code>	3-to-1 multiplexer
<code>tt_libcell_stdand2.sv</code>	2-input AND gate
<code>tt_libcell_stdnand2.sv</code>	2-input NAND gate
<code>tt_libcell_stdand3.sv</code>	3-input AND gate
<code>tt_libcell_stdor2.sv</code>	2-input OR gate
<code>tt_libcell_sync2r_np.sv</code>	2-stage synchronizer with active-low reset (non-pipelined)
<code>tt_libcell_sync2r.sv</code>	2-stage synchronizer with active-low reset
<code>tt_libcell_sync2s.sv</code>	2-stage synchronizer with active-low set
<code>tt_libcell_sync2.sv</code>	2-stage synchronizer
<code>tt_libcell_sync3r.sv</code>	3-stage synchronizer with active-low reset
<code>tt_libcell_sync3s.sv</code>	3-stage synchronizer with active-low set
<code>tt_libcell_sync3.sv</code>	3-stage synchronizer
<code>tt_libcell_sync4r.sv</code>	4-stage synchronizer with active-low reset
<code>tt_libcell_sync4s.sv</code>	4-stage synchronizer with active-low set

continues on next page

Table 2 – continued from previous page

File Name	Description
tt_libcell_sync4.sv	4-stage synchronizer
tt_libcell_xor2.sv	2-input XOR gate

2.2 Synthesis and Floor-planning Guidelines

This Document is complimented by an Implementation Guidelines document. Please refer to said document for overall implementation guidelines. This section of the documentation is focused more on the provided synthesis flows and scripts.

2.2.1 Required `define Flags

The following defines are required for synthesis:

```
SYNTHESIS
NEO_PD_PROTO
TDMA_NEW_L1
RAM_MODELS
USE_VERILOG_MEM_ARRAY
TT_CUSTOMER
TRINITY
```

```
analyze -format sverilog -define "SYNTHESIS NEO_PD_PROTO TDMA_NEW_L1 RAM_MODELS USE_
↳VERILOG_MEM_ARRAY TT_CUSTOMER TRINITY" ....
```

2.2.2 Synthesis Reference Flow

The synthesis reference flow is based on Floorplan shown below. you can find a reference synthesis flow under \$TENSIX_ROOT/synth - here, TENSIX_ROOT is an environment variable that should map to the root of the uncompress delivery package. Within this path will find the following files:

1. `run_dc.sh` - a script to run the reference synthesis flow using Synopsys Design Compiler. please refer to the script itself for guidance on usage
2. `run_run_syn.tcl` - DC TCL script that manages the synthesis itself (e.g. calling Analyze, elaborate, link, etc...)
3. a directory for each Trinity Partition. each directory includes: corresponding SDC constraint file for its respective partition
4. Makefile that includes a target for every partition, all partitions and cleanup.

To run a reference synthesis flow, you could use the Makefile's different targets to run synthesis on different partitions, or use the `run_dc.sh` script directly.

the partition top levels are:

- `tt_fpu_gtile`
- `tt_overlay_wrapper`
- `tt_noc_niu_router`
- `tt_t6_l1_partition`
- `tt_instrn_engine_wrapper`
- `tt_tensix_with_l1` (Tensix top level)

- `tt_disp_eng_l1_wrap2`
- `tt_disp_eng_noc_niu_router`
- `tt_disp_eng_overlay_wrapper`
- `tt_dispatch_engine` (dispatch engine top level)
- `trinity_noc2axi`

run_dc.sh Example

```
./run_dc.sh tensix.f tt_fpu_gtile
```

Where `tensix.f` is the unravelled source file list from the `$TENSIX_ROOT/tb` directory and `tt_fpu_gtile` is the target partition to be synthesized.

Makefile Examples

```
make syn_fpu
```

2.2.3 Primitives and Memory Wrappers

you will find all primitives and memory wrappers under `$TENSIX_ROOT/rtl/primitives` please take care to replace the simulation model that is defined in each memory and primitive cell with your own technology library cell implementation. you can condition the usage of a library cell vs. a behavioral model using an ``ifdef`, condition on the `SYNTHESIS` define that is already being used in the design

2.2.4 Floorplan

This floorplan is our current plan of record. It is based on a Neo cluster with a 4MB L1 memory , and enhanced MXFP4 math engine. All sizes are estimated based on our Samsung SF4 4nm process, that is used for our next generation AI chiplet

Please refer to the accompanying Physical design implementation guidelines document for further details.

Overall Tensix Floorplan with port placement

Partition Hierarchy Scheme - Tensix

The table below outlines the design hierarchies for the Tensix floorplan. The hierarchies highlighted in light grey represent the physical design (PD) partitions. These partitions are critical for the synthesis and floor-planning process, ensuring that each section of the design is properly organized and optimized for implementation.

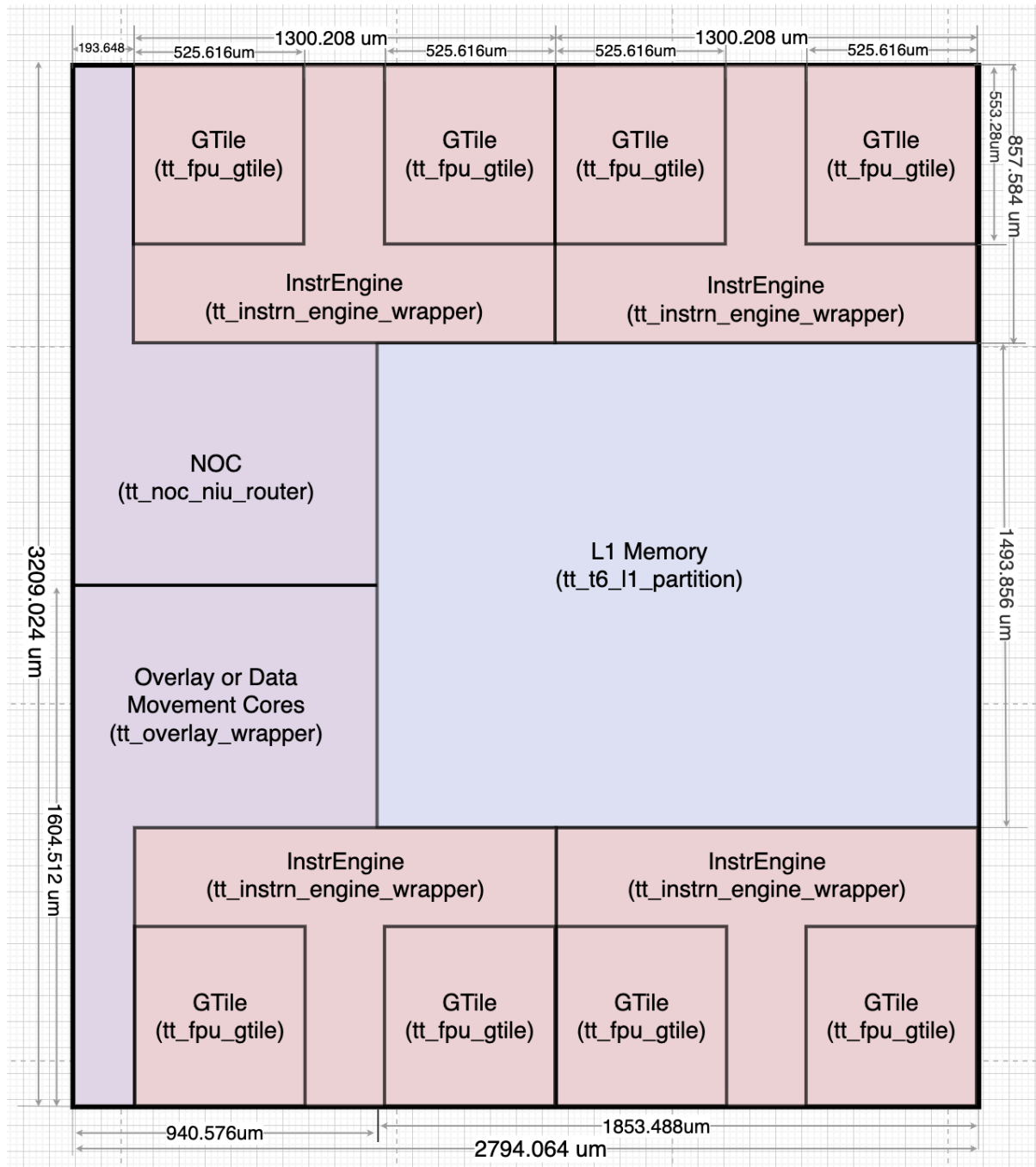
hierarchy level 0	Hierarchy level 1	Hierarchy level 2	Hierarchy level 3
tt_tensix_with_l1			
	tt_tensix[3:0]		
		tt_instrn_engine_wrapper	
		tt_fpu_gtile[1:0]	
	tt_t6_l1_partition		
	tt_overlay_noc_wrap		
		tt_overlay_noc_niu_router	
			tt_overlay_wrapper
			tt_noc_niu_router

Partition Hierarchy Scheme - Dispatch Engine

The table below outlines the design hierarchies for the Dispatch engine.

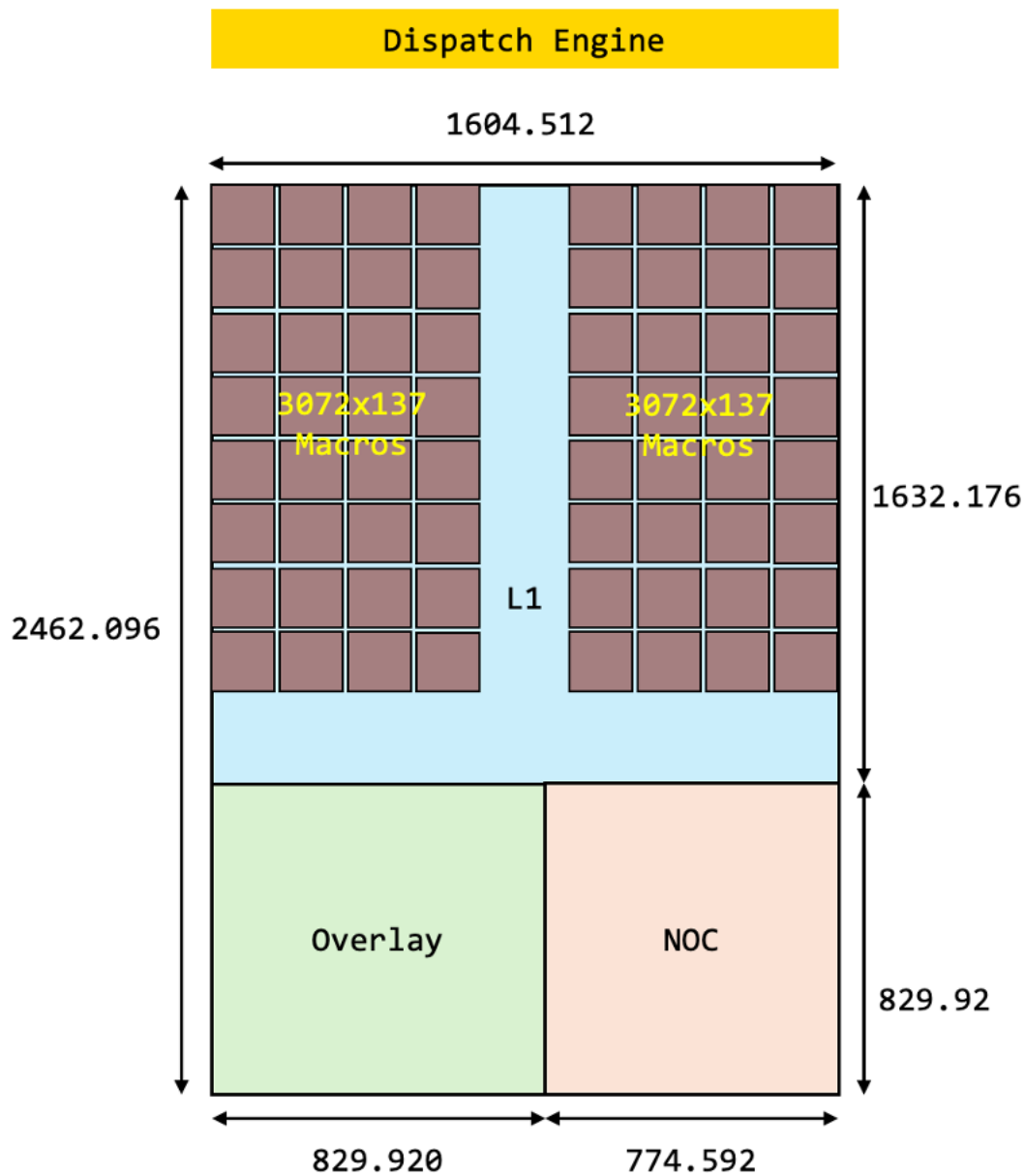
Hierarchy Level 0	Hierarchy Level 1	Hierarchy Level 2	Hierarchy Level 3	Hierarchy Level 4
tt_dispatch_top				
	tt_dispatch_eng			
		tt_disp_eng_l1_partition		
			tt_clkmux2	
			tt_disp_eng_l1_wrap2	
				tt_disp_eng_l1_wrap
				tt_t6_l1_mem_wrap
		tt_disp_eng_overlay_no		
			tt_disp_eng_overlay_noc_ni	
				tt_disp_eng_overlay_wrapper
				tt_disp_eng_noc_niu_router

Floor Plan - Tensix

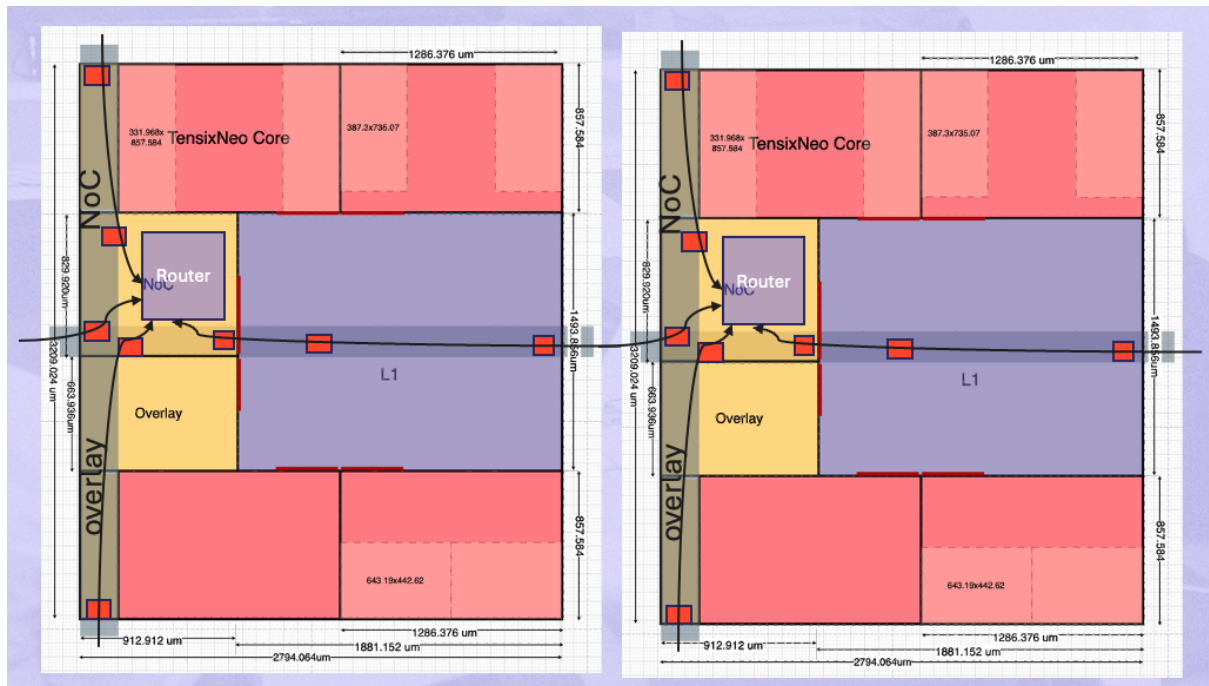


Floorplan - Dispatch core

Note This floor plan is based on our implementation of the dispatch engine in our own silicon products. please use your own judgement and create your own floorplan that would take in into account your own unique SoC constraints.

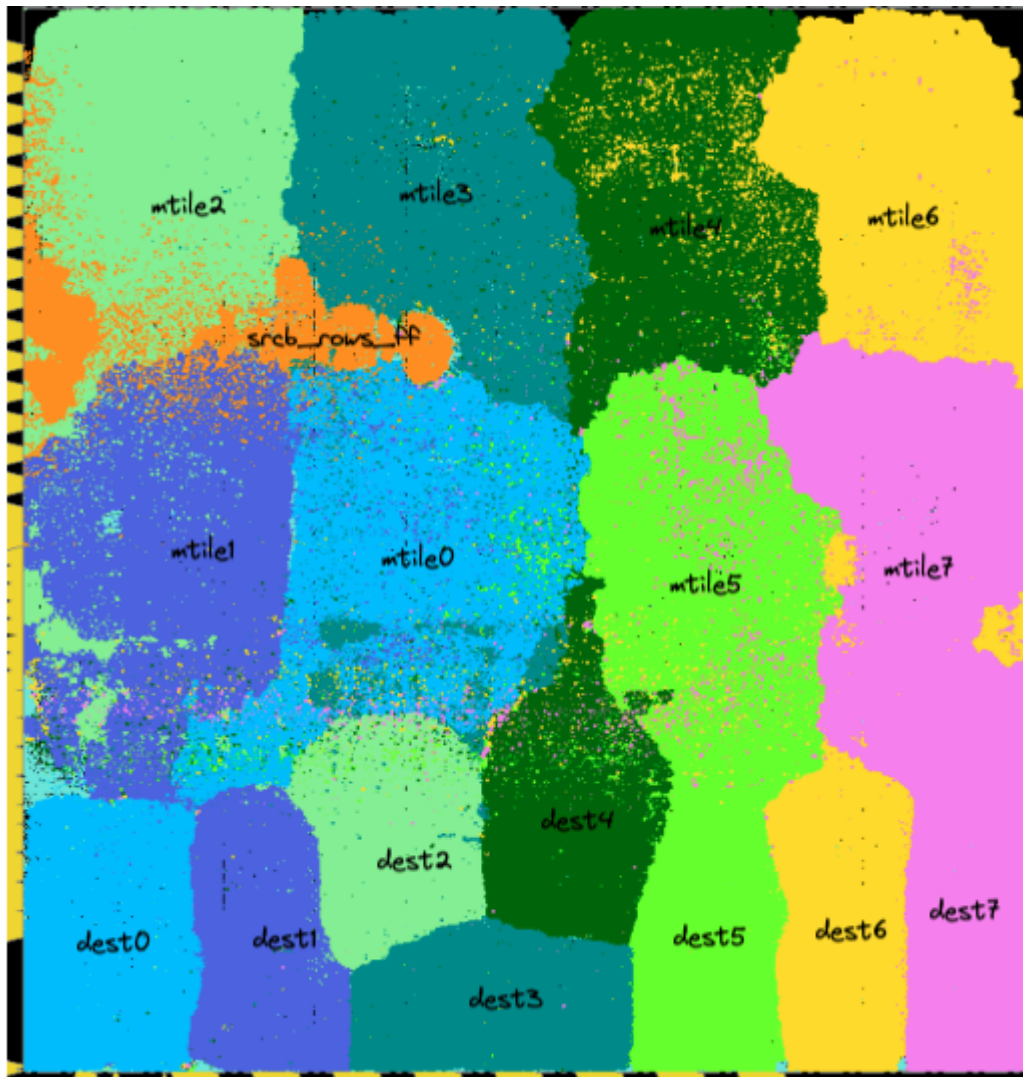


Overall Tensix Floorplan with NOC repeaters placement

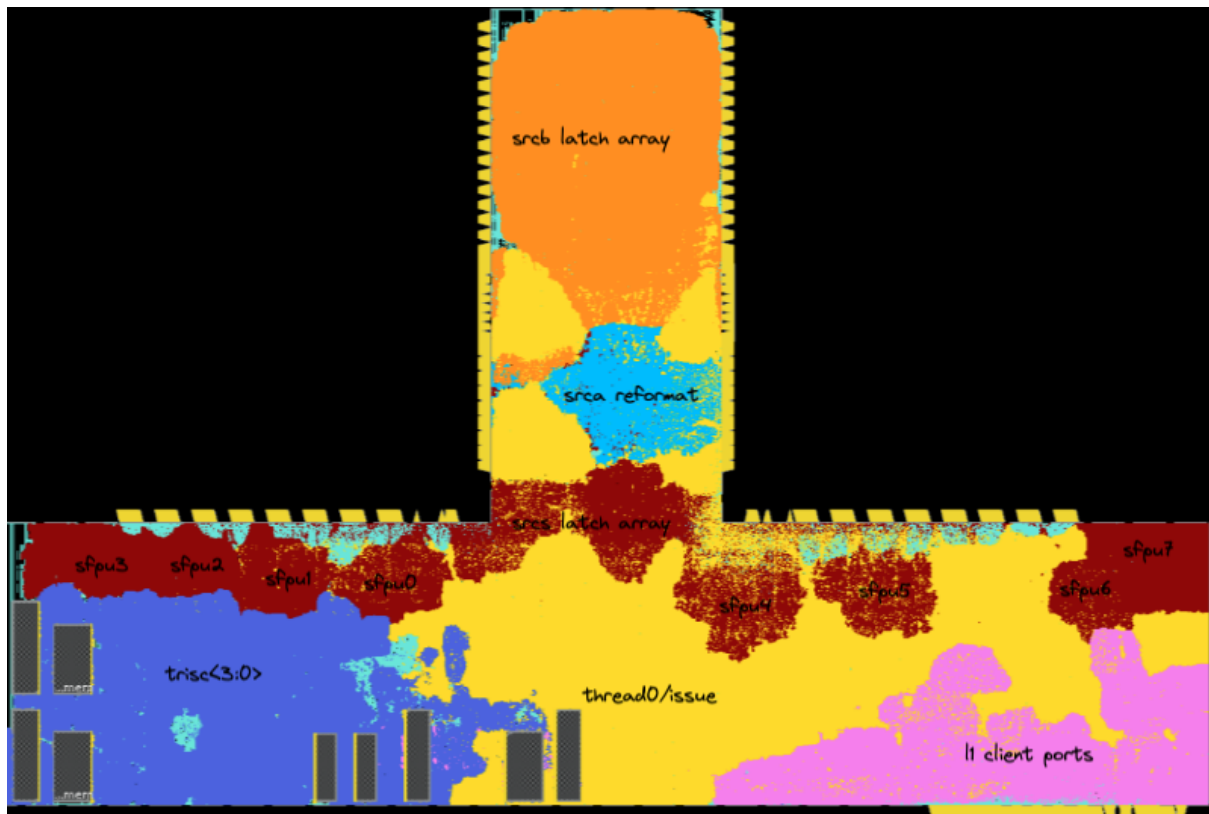


2.2.5 Submodule placement - Tensix

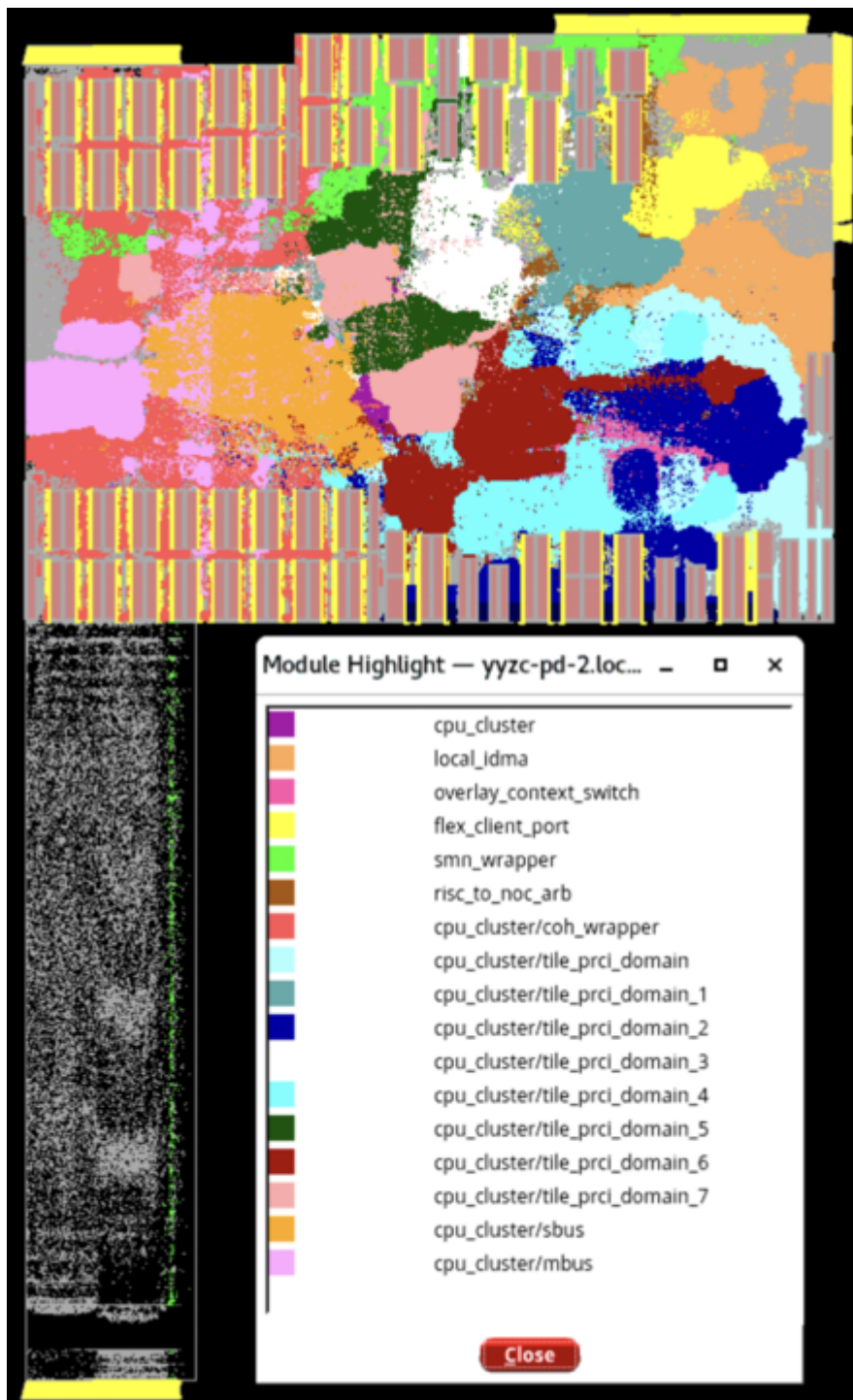
Gtile



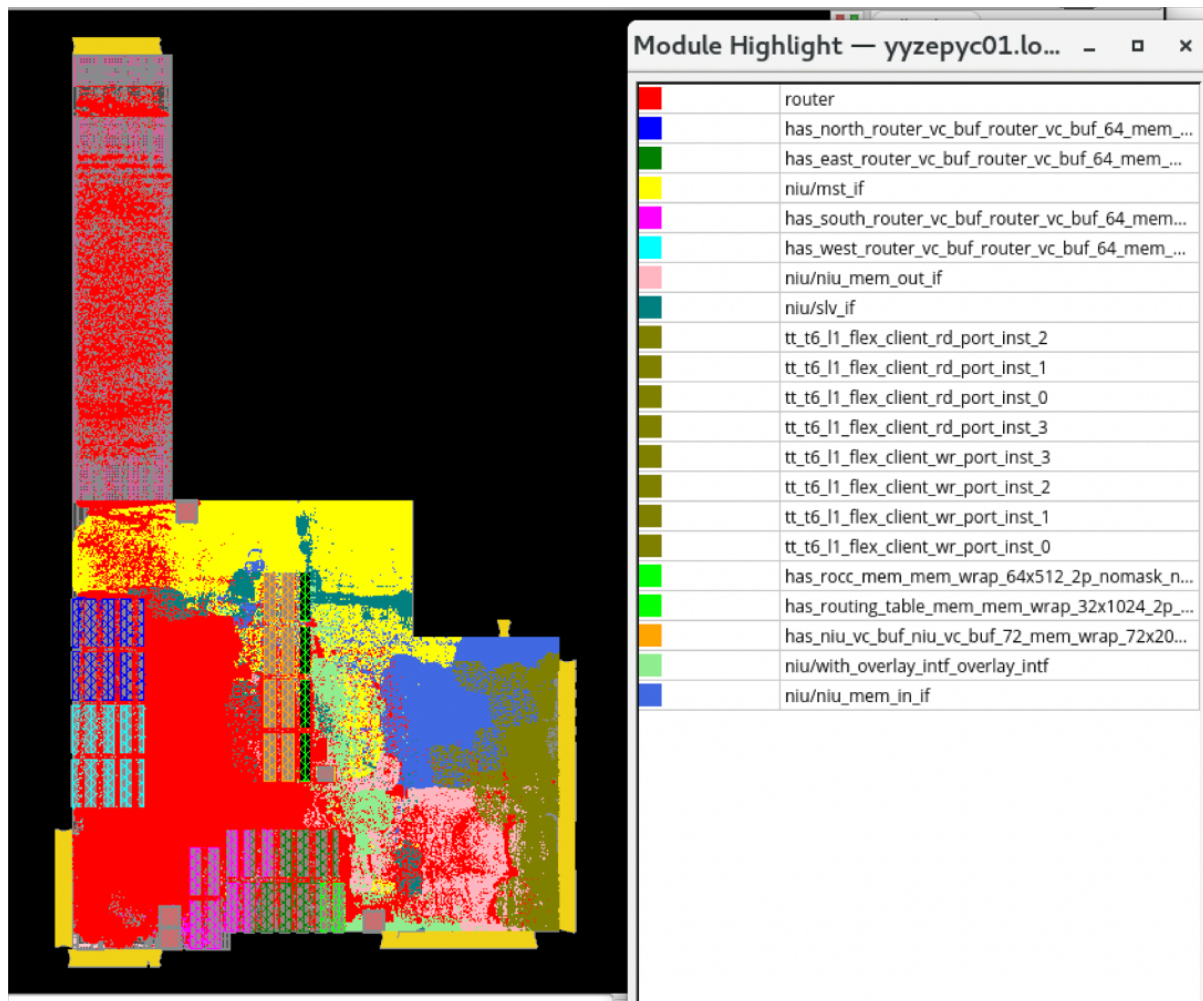
Instruction Engine



Overlay

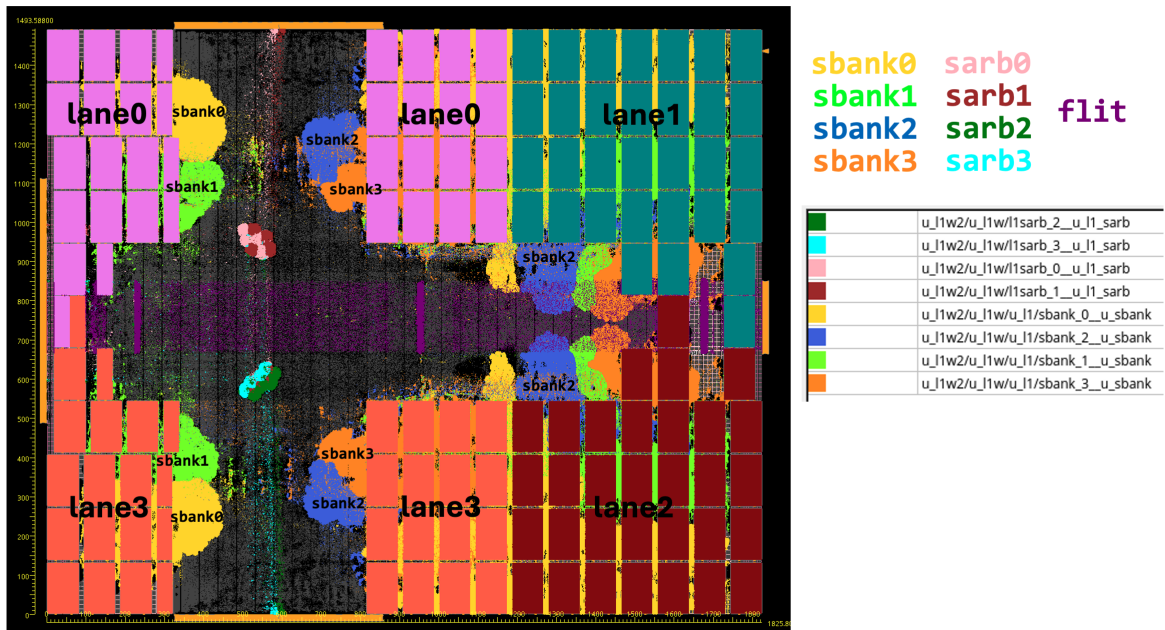


NoC



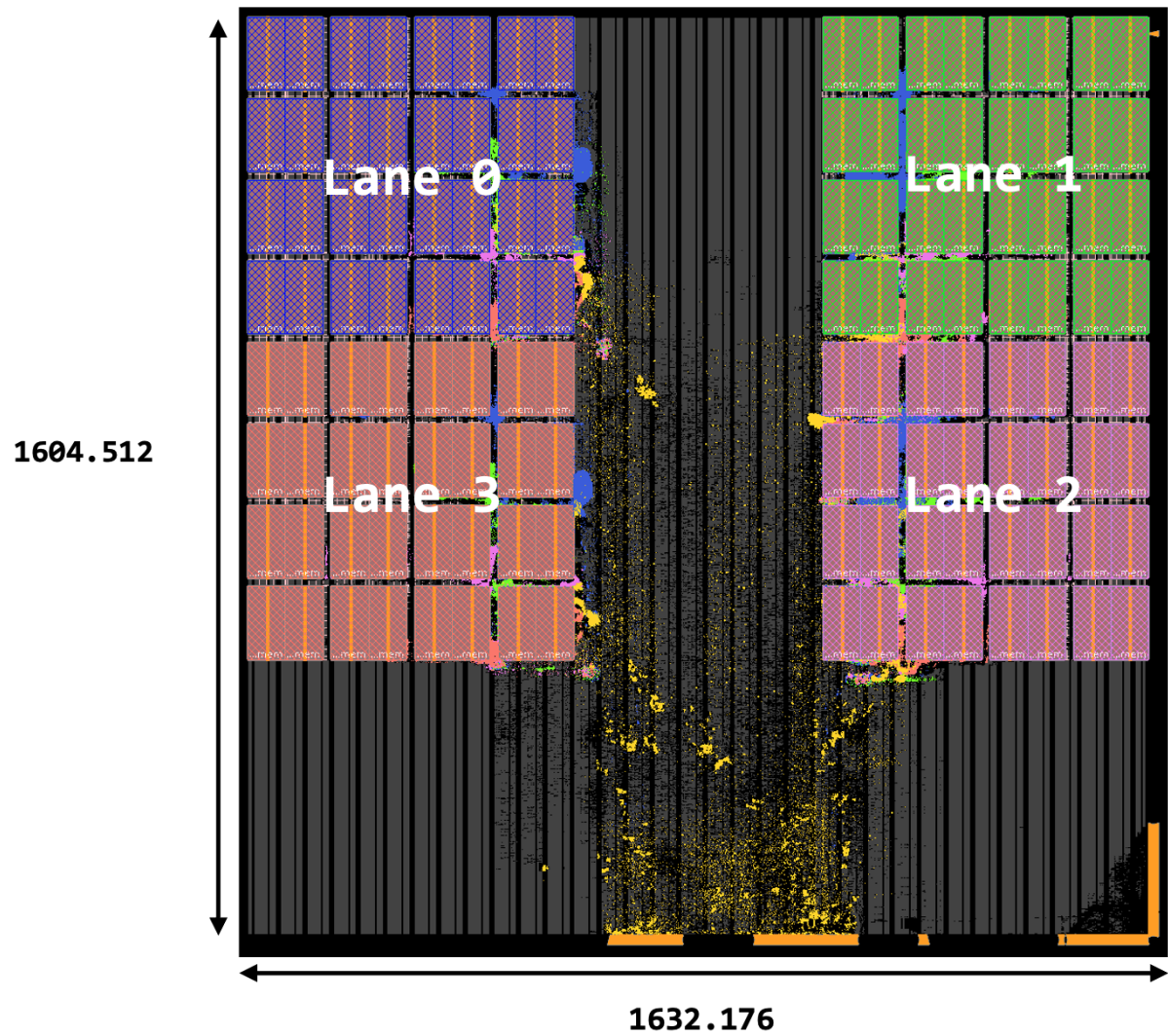
L1

T6 L1 Block Placement

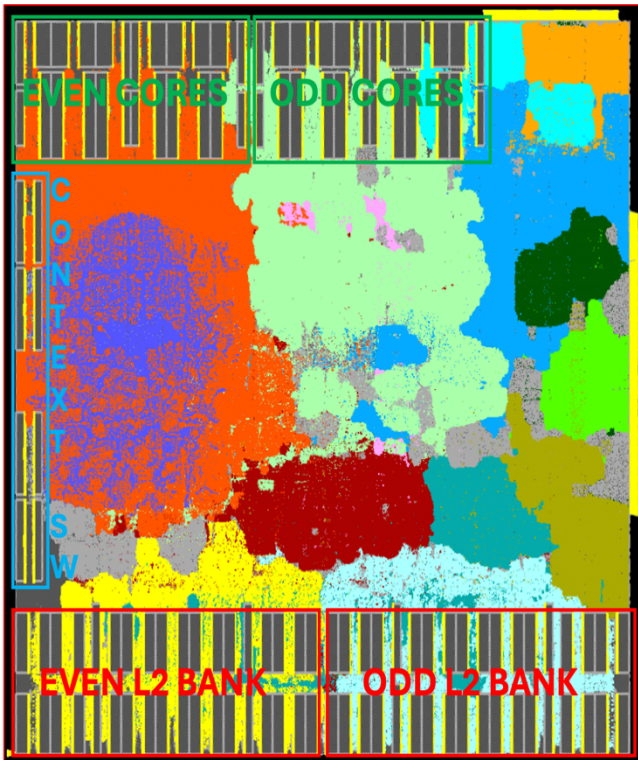


2.2.6 Submodule placement - Dispatch Engine

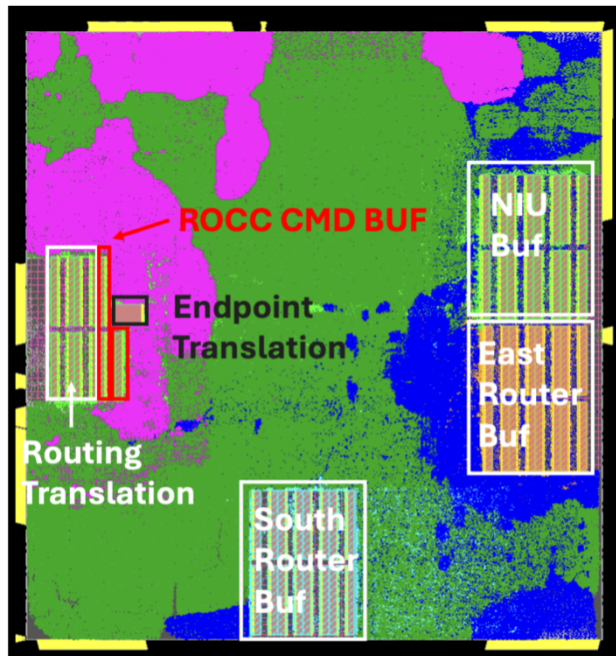
Dispatch engine L1



Dispatch engine Overlay

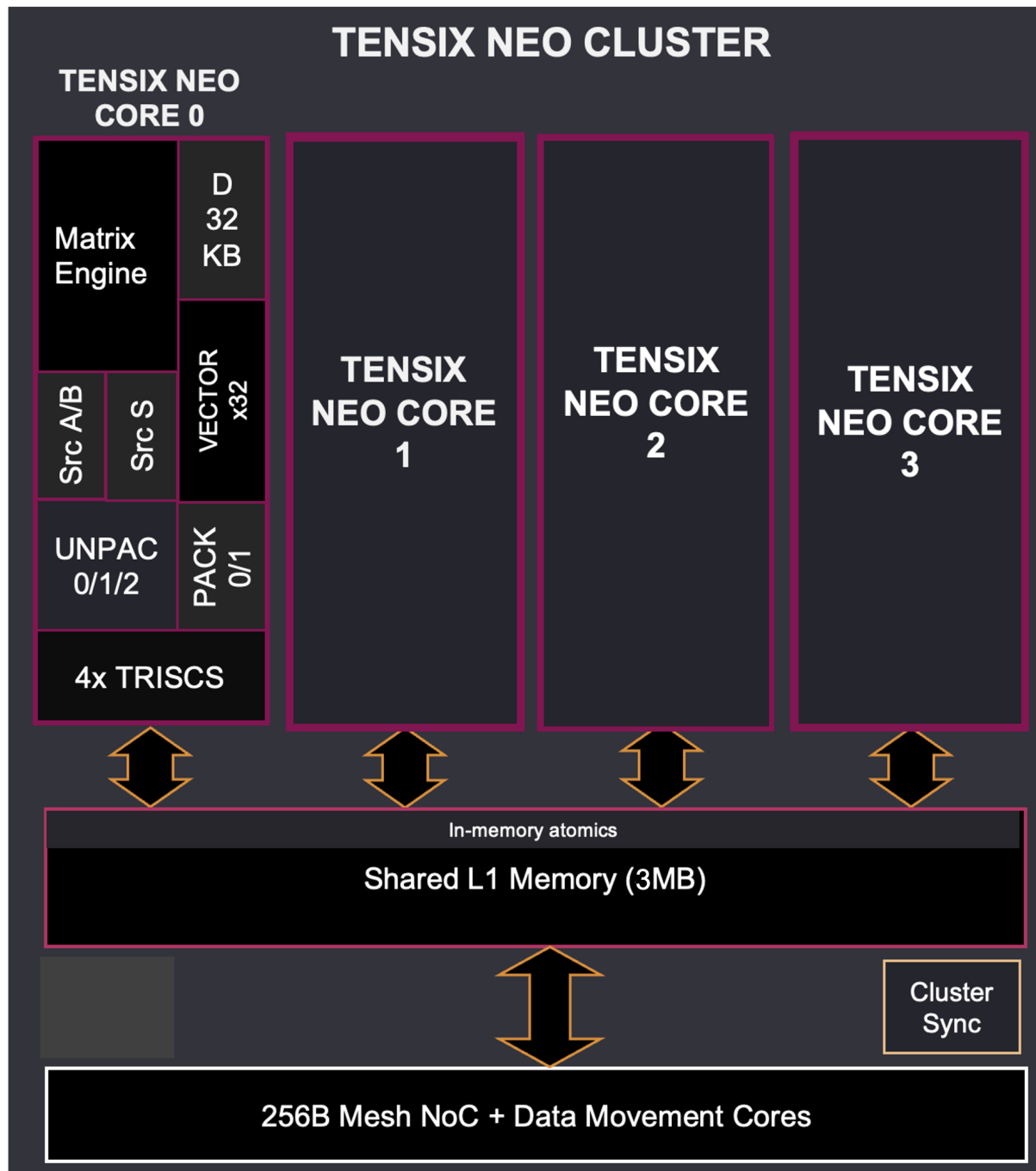


Dispatch engine NoC



2.3 Tensix Neo Cluster User Manual

2.3.1 Tensix Neo Cluster Top-Level Block Diagram



2.3.2 Tensix Neo Core Components

The goal of this section is to get you up to speed on what components exist in each Tensix core, and what they are used for. This is also a good chance to provide canonical names and terminology. Each Tensix Neo Cluster is composed of 4 Neo cores. These Core provide the fundamental matrix and vector compute capability. Each core is capable of performing 8K INT operations per clock in the Matrix Engine. It can also handle 2K floating-point operands per cycle in BF16 format.

Supported Data Formats

Formats supported by the NPU

FLOAT16, FLOAT16_B, FP8R, FP8P, MXFP8R, MXFP8P, MXFP6R, MXFP6P, MXFP4, MXINT8, MXINT4, MXINT2, INT8, UINT8, INT32 (accumulation)

MACs Throughput

The following table depicts the expected throughput of MACs (Multiply + Add) per Tensix Neo cluster per supported data formats

Data Format	Tensix Neo Trinity lo-fi	Tensix Neo Trinity Hi-Fi
FP32, INT32 (accumulation only)	-	-
TF32	2048	512
Float16	2048	512
Float16_B	2048	2048
FP8R	-	2048
FP8P	-	2048
MXFP8R	-	2048
MXFP8P	-	2048
MXFP6R	-	2048
MXFP6P	-	2048
MXFP4	-	2048
MXINT8	-	2048
MXINT4	-	2048
MXINT2	-	2048
MXFP4	-	2048
INT8	-	8192
UINT8	8192	8192

TDMA - Unpackers

There are 3 “unpackers” in a Tensix core. They provide high-bandwidth path from L1 and the four register files (SrcA, SrcB, SrcS, Dest). They also perform format conversions. Unpacker 0 always unpacks into SrcA, unpacker 1 always unpacks into SrcB and unpacker 2 always unpacks into SrcS. Unpack to SrcA, SrcB and Dest will provide support for inline transpose.

Unpackers support the following operations when moving data from L1 to Src registers (based on TDMA config registers):

1. Format conversion - converts the format from the incoming format to the respective Src register format (configurable via Src TDMA config registers)
2. Transpose (SrcA/B only) - Rows are transposed into columns
3. Bank Initialization - Every datum in a bank can be initialized to a particular value

TDMA - Packers

There are 2 “packers” in a Tensix core, like the unpackers, they are a DMA engine. However, they read from Dest/SrcS and writes to L1. They can also perform format conversions, allowing math to run at a higher precision and the result to be written into a lower precision number, reducing L1 footprint. It also has dedicated logic to perform, relu, relumax, crelu and flushing small number to zero.

TRISC 0, 1, 2 and 3

The TRISC are used to execute low-level Tensix kernels. These kernels are written in C with intrinsic instructions (Tensix instructions) that are mapped to Tensix execution units e.g. Packer, Unpacker, FPU, SFPU, CfgExu, and SyncExu.

Although all three TRISC are (more-or-less) identical, by convention we use each one for a different set of tasks.

- TRISC 0 is typically used for unpacking, i.e. copying data from L1 into the srcA/srcB registers.
- TRISC 1 is typically used for math
- TRISC 2 is typically used for packing, i.e. copying data from the dest register into L1
- TRISC 3 Is typically used to run SFPU work in parallel with the regular FPU work running

Trisc kernel C-code does not necessarily execute in lockstep with Tensix instructions that it uses. Once a Tensix instruction is issued by Trisc, it continues to execute next instructions (RiscV or intrinsic Tensix instructions). The issued Tensix instructions can take multiple cycles to execute. If a Tensix instruction is expected to use the results of another Tensix instruction, the hardware is expected to stall to prevent RAW hazards, unless the two instructions were executed by different execution units. In the latter case, the programmer must add synchronization instructions or wait for the producer execution unit to go idle before issuing the consumer instruction.

MOP Decoder

This stands for “Macro-Operation Decoder”. All Tensix instructions leaving the TRISC will pass through a MOP Decoder before going any further. Normally, instructions enter in one end and leave out the other unchanged. In this mode of operation, the TRISC is completely responsible for sending out Tensix instructions.

However, forcing the TRISC to issue every single Tensix instruction can limit performance of some kernels. For such kernels, the MOP decoder acts like a loop accelerator. The MOP decoder can be programmed with loop dimensions and Tensix instructions to execute in the loop. When this programmed MOP is triggered, the MOP Decoder will generate instructions by itself. Not only does this push Tensix instructions as fast as the core can accept them, but it also frees up the TRISC to do other work. The MOP registers are double buffered

Replay Unit

This module is like the MOP Decoder in that normally instructions pass through unchanged. However, you can ask it to “replay” a set of instructions that were issued in the past. This, in combination with the MOP Decoder, makes it possible to generate a lot of Tensix instructions with minimal TRISC involvement.

SrcA/SrcB/SrcS Registers

These registers store the inputs for math instructions. SrcA and SrcB are used for FPU operations while SrcS is used for SFPU operations.

The srcA registers store floating point numbers to be used in subsequent math operations. The registers are divided into 16 columns with each column containing 2 banks. Each bank is divided into 4 sets of 16 19-bit registers for a total of 2048 registers.

The srcB registers store floating point numbers to be used in subsequent math operations. The registers are divided into 2 banks of 64 rows each, 16 columns per row, and 19 bits per column. This gives a total of 2048 registers.

Source register SrcS is used to perform operations using the SFPU independent of the FPU. This allows for threads to be running on both FPU and SFPU at the same time, thus increase utilization. SrcS will use Unpacker 2 as well as Packer 1 to fetch and store data from/to L1. The registers are comprised of 2 banks with each bank containing 3 slices of registers. Each slice consists of 2 instances of tt_reg_bank (8x16x16). Each instance is composed of 8 rows x 16 columns x 16bits of data. Unpacker 2 stores data in slices 0-1. SFPU loads and executes data from these sets and writes to slice 3. The packer then reads data from set 3 to store in L1. For the next set of operations, the slices from the next bank are used.

For better performance, we double-bank these registers. That means including an identical second copy of srcA and srcB, which allows us to let math instructions read from one bank of the src registers while the unpacker writes into the other.

These register banks could be implemented as LATCH arrays.

Dest Registers

The dest register is where FPU and SFPU math instructions will write their outputs. It has space for 512 rows x 16 columns x 16-bit datums, and like the src registers, is double-banked. Unlike the source registers. The Dest registers have two modes of operation: 16-bit and 32-bit mode. In 16-bit mode, it has 1024 rows. In 32-bit mode, we combine pairs of rows together, making it effectively have 512 rows. The dest register supports any 16-bit or 32-bit data type.

This register banks could be implemented as LATCH arrays.

FPU

The floating-point unit (FPU) is the matrix multiplication engine. It supports a variety of math operations. Typically, these operations read operands from SrcA and SrcB and write results to Dest.

SFPU

SFPU is a vector engine that is used to support activation functions and general vector code including conditional execution. SFPU reads data from dest or SrcS and has 8 internal registers that the SFPU code can use for temporary working set. results are written either to Dest or SrcS.

NOC and Data Movement

The Tensix Neo cluster can control data movement to/from DRAM, other Tensix cores and locally between L1 <-> L1, by using its data movement section. the Data movement section contains 3 main components:

Dimension-Order Routing

The Network-on-Chip (NoC) uses dimension-order routing, where each packet header is tagged for either X->Y or Y->X routing.

X->Y Routing (Default):

- If $X2 \neq X1$, route along the X-coordinate until $(X2, Y1)$ is reached.
- If $Y2 == Y1$, the destination is reached. Otherwise, route along the Y-coordinate until $(X2, Y2)$ is reached.

Y->X Routing:

- If $Y2 \neq Y1$, route along the Y-coordinate until $(X1, Y2)$ is reached.
- If $X2 == X1$, the destination is reached. Otherwise, route along the X-coordinate until $(X2, Y2)$ is reached.

Example:

- For a packet routed from $(X1, Y1)$ to $(X2, Y2)$:

– Using X->Y routing:

* If $X1 = 1$, $Y1 = 1$ and $X2 = 3$, $Y2 = 4$, the packet routes from (1,1) to (3,1) and then from (3,1) to (3,4).

– Using Y->X routing:

* If $X1 = 1$, $Y1 = 1$ and $X2 = 3$, $Y2 = 4$, the packet routes from (1,1) to (1,4) and then from (1,4) to (3,4).

In a mesh network, dimension-order routing guarantees deadlock safety for unicast traffic, provided each virtual channel is always routed with the same dimension order (i.e., never using both X->Y and Y->X on the same VC).

Data Movement Cores

The Tensix Neo architecture incorporates a cluster of **eight 64-bit RISC-V cores** specifically designed to manage efficient data movement within Tensix clusters, between clusters, and with external interfaces. These cores employ a **5-stage in-order pipeline** and support a comprehensive set of RISC-V extensions to enhance functionality and performance, including:

- **rv64imac**: Integer, multiplication/division, atomic operations, and compressed instructions.
- **zicsr**: Control and Status Register instructions.
- **zifencei**: Instruction-fetch fence for synchronization.
- **zbs, zba, zbb**: Bit manipulation extensions for data-processing efficiency.

The cores are further equipped with prefetchers for L1 instruction and data caches, alongside robust branch predictors to optimize execution flow. They use a **write-back cache hierarchy** consisting of a **4 KB L1 Data Cache**, a **4 KB L1 Instruction Cache**, and a shared **128 KB L2 Cache**. This hierarchical setup ensures that data flow is both rapid and coherent across the eight cores.

Hardware Data Movement Acceleration

The data movement cores are enhanced with specialized accelerators that significantly improve their ability to handle complex data transfer tasks. These accelerators, accessible via custom RISC-V instructions, allow for highly efficient orchestration of data processing and movement:

1. NoC Copy Command Accelerator

To maximize utilization of the Network-on-Chip (NoC), each core is equipped with a dedicated hardware block that enhances the rate at which commands are issued. Command buffers unique to each core facilitate ultra-low-latency 64-bit stores, enabling rapid construction of NoC commands. With multiple buffers available per core, the need for frequent reprogramming is minimized, allowing uninterrupted and efficient operation.

2. Patterned Address Generation Accelerator

Address generation, a critical factor in achieving NoC saturation, is streamlined through dedicated accelerators for each core. Two address generators per core use a custom instruction interface, allowing CPUs to program and request subsequent addresses dynamically. These addresses are generated based on parameters such as inner and outer dimensions, sizes, and strides. By directly populating NoC commands, this feature eliminates bottlenecks in data transfer, particularly for smaller payloads.

3. Fast Context Switching

Standard RISC-V context switching is inherently slow, presenting a challenge for workloads requiring rapid transitions between execution streams. The data movement cores address this with a shared context-switching accelerator that can save and restore the entire RISC-V register file in significantly fewer cycles. This capability enables seamless and efficient switching, improving overall responsiveness for dynamic workloads.

4. Local Data Shuffling

Data reorganization within the Tensix SRAM is facilitated by eight 16-byte DMA engines integrated into the data movement cluster. These engines allow for efficient data reshaping and shuffling, either through direct register-based control or custom instructions issued by the RISC-V cores. This feature is particularly advantageous for tensor-processing tasks, where data reformatting is often required for computational efficiency.

NOC Transaction types and attributes

Broadcast

a transaction that goes into a rectangle that is defines by it's edges (start-x ,start-y , end-x, end-y). BRoadcast has a wide veriaty of other attributes:

X/Y Direction

A broadcast transasction can either be routed X-first or Y-first This becomes useful in MatmUI operations where activations are broadcasted over the X direction , and weights and biases are broadcasted on the Y axis

Strided

Introduces the ability to create braodcast patterns based on parameters like keep x/y , skip x/y , where you can determin how many clusters to braodcast to , and how many are skipped in a given pattern For example, a strided broadcast transactions with attributes : keep_x = 1 , keep_y = 1, skip_x=1 , skip_y=1 , is sent to Tensix (0,0)

Y / X	0	1	2	3
2	1	0	1	0
1	0	0	0	0
0	1	0	1	0

Another example would be a strided broadcast transactions with attributes : keep_x = 0 , keep_y = 2, skip_x=1 , skip_y=2 , sent to Tensix (0,0)

Y / X	0	1	2	3
2	0	0	0	0
1	1	1	1	1
0	1	1	1	1

Exclusion :

Broadcast supports a way to exclude some portions of the grid from being broadcasted to. This is supported through a feature called 'exclusion' This exclusion would initiate from the excluded coordinate and on . As an exmaple if the excluded coordiantes are x=2, y=3 , and we send a broadcast to the entire grid starting from 0,0:

if the broadcast goes X first :

Y / X	0	1	2	3
3	1	1	0	0
2	1	1	0	0
1	1	1	1	1
0	1	1	1	1

if the broadcast would go Y first, then X

Y / X	0	1	2	3
3	0	0	0	1
2	0	0	0	1
1	1	1	1	1
0	1	1	1	1

Linked

Linked transactions in the NoC have a 'linked' indicator that locks the path from source to target for the entire traffic stream. Each transaction in the sequence must have the linked field set. The last transaction in the sequence should have the linked field set to 0 to de-allocate the route, ensuring the path is freed for other traffic.

A series of transactions that have the 'linked' indicator will lock the path they take from source to target specifically for this traffic stream. You must have the linked field set for all transactions in the linked sequence, and set to 0 for the last transaction in the linked sequence to de-allocate the route for the linked sequence.

Posted / Non-posted

A posted transaction does not require a response from the endpoint target to be cleared. This means that once the transaction is sent, the sender does not wait for an acknowledgment or completion signal from the receiver.

In contrast, a non-posted transaction requires a response from the endpoint target. The sender must wait for a confirmation that the transaction has been received and processed before considering it complete.

Virtual channel assignment

The Trinity NOC contains 16 available virtual channels for traffic to be routed on. Virtual channels 0-11 are allocated for unicast reads/writes virtual channels 12-15 are allocated for broadcast writes

Tags

Tags are unique identifiers assigned to NoC transactions for diagnostics and debugging. They do not affect ordering, virtual channel assignments, or any other functional aspects.

L1 Memory

The Tensix Neo architecture features **3 MB of L1 memory** per cluster, accessible to all Tensix cores, the Network-on-Chip (NoC), data movement units, and the packer/unpacker. Designed for high efficiency, the L1 memory supports atomic operations that can be invoked via RISC-V atomic instructions, the NoC, or the packer for L1 accumulation tasks.

Tensix SRAM Memory Structure

The Tensix SRAM is structured to optimize both performance and accessibility:

- **Total Capacity:** 3 MB
- **Organization:**
 - The memory is divided into **64 banks**, grouped into **4 super banks**.
 - Each super bank contains **16 banks**, and each bank is composed of:
 - * **64B (512 bits) width**
 - * **768-depth SRAM**
 - * Each bank is further subdivided into **4 sub-banks**, with each sub-bank configured as **128b × 768 SRAMs**.
 - Requests are processed at a **512b granularity**, with responses aggregating data from **4 sub-banks**.
- **Error Correction:**
 - **ECC (Error Correcting Code)** is implemented using **SEDED** (Single Error Correction, Double Error Detection) at a **128b granularity**.
 - This ECC configuration results in a final memory structure of **137b × 768 SRAMs**.

Memory Mapping

The L1 memory is mapped into the RISC-V address space for streamlined access:

- **Address Range:** 0x0000_0000 to 0x002F_FFFF for all RISC-V cores within the Tensix Neo cluster.
- Additionally, L1 memory is accessible via the NoC and packer/unpacker units to facilitate efficient data movement and computation.

Atomic Operations Support

The L1 memory is designed to support a variety of atomic operations, ensuring flexibility and performance for data processing. These operations can be invoked through:

- **RISC-V Atomic Instructions:** Enabling direct, hardware-level atomic operations.
- **NoC Commands:** Facilitating atomic interactions over the cluster's data communication fabric.
- **Packer Integration:** Supporting accumulation tasks directly within the memory subsystem.

Cluster Sync

Each Tensix core in a cluster will have a CSR that corresponds to its ID within the Tensix Neo cluster. A global sync unit will be implemented to support semaphore synchronization between Tensix cores in a Neo cluster as well as between NOC, data movement cores and Tensix cores. 16 semaphore registers will be added at cluster-level. TRISC in Tensix cores can access the semaphores and mutexes using Tensix instructions.

2.4 Verification Testbench

This delivery includes a standalone testbench intended validate that each AXI subordinate can receive inbound traffic. The testbench also serves as an example of how to bring up simple Tensix Test vectors. The testbench environment requires Python 3.9 or above and Synopsys VCS installed. Please refer to the release notes for detailed versioning information.

2.4.1 Testbench Setup & Test Execution

The standalone testbench is built using Synopsys' VCS and cocotb, an open-source library that provides a simple and intuitive way to interact with digital designs using Python. The testbench leverages the cocotbext-axi extension, which provides an ergonomic interface to initiate and receive valid AXI transactions.

2.4.2 Testbench Setup and Execution

Note: The steps below will be automatically run if you execute the `setup.sourceme` script within `/tb/`. (and under `/rtl/primitives/wrappers/tb/`)

Environment Setup

Follow the steps below to set up the testbench environment:

1. Create and Activate a Python Virtual Environment:

- Navigate to the `/tb/` directory.
- Run the following command to create the environment:

```
python3 -m venv tensix_tb_venv
```

- Activate the environment:

```
source tensix_tb_venv/bin/activate
```

2. Install Required Python Packages:

- Install cocotb:

```
pip3 install cocotb
```

- Install the AXI extension:

```
pip3 install cocotbext-axi
```

- Install YAML support:

```
pip3 install pyyaml
```

3. Set the \$TENSIX_ROOT Environment Variable:

- Point `$TENSIX_ROOT` to the top-level directory of the delivery package tree. For example:


```
export TENSIX_ROOT=/path/to/tt_rtl_enc/
```

4. Activate the virtual environment created in the previous step:

- Activate the Python environment that was created in the previous steps

```
source tensix_tb_venv/bin/activate
```

Test Execution

Assuming that the steps in the previous section have been completed, follow the steps below to execute the testbench:

1. Run the testbench:

```
make
```

This will initiate a VCS compile, followed by the tests contained in `test_trinity.py`. This file has an additional dependency `coco_tensix_api.py` and `test_utils.py`.

NOTE : you might need to set the cocotb X handling environment variable :

```
export COCOTB_RESOLVE_X=ZEROS
```

2. View the test results in the console.

The Makefile supports a few targets and command-line options:

Command Line	Description
<code>make</code>	Compile and simulate
<code>make sim</code>	Compile and simulate
<code>make WAVES=0 sim</code>	Compile and simulate, disable waveform generation
<code>make TESTCASE=<test_name></code>	Run a specific test case
<code>make USE_GUI=1 sim</code>	Compile and simulate using interactive Verdi GUI
<code>make verdi</code>	Launch the Verdi waveform viewer and load waveform
<code>make clean</code>	Remove all simulation output files

2.4.3 Test Lists

Currently, there is only one test that is available to run in this delivery

Test Name	Test Type	Description
san-ity_noc2axi_uni	Inbound AXI Test	Sends traffic from each NoC2AXI to each Tensix cluster and Dispatch Engine memory.
san-ity_noc2axi_mul	Inbound AXI Test	Sends multicast traffic from each NoC2AXI to all Tensix clusters.
san-ity_axi_random	Inbound AXI Test	Sends random AXI transactions from each NoC2AXI to the Tensix clusters and Dispatch Engine memories.
strided_multicast	Inbound AXI Test	Exercises the strided multicast feature through NoC2AXI[0]. Refer to the <i>Tensix Manual</i> for details on strided multicast requests.
att_translation_c	Inbound AXI Test	Preloads the address translation table on NoC2AXI[0] with endpoint mappings, then verifies that traffic is routed to different endpoints as per the translation table.
att_routing_conf	Inbound AXI Test	Configures custom routes on NoC2AXI[0] for a specific source, target, and VC combination, then verifies that transactions with the same parameters are routed correctly.
addr_pinger_test	Embedded Test	Initiates traffic from different Tensix cores using their overlay RISC-V cores.

2.4.4 SRAM Integration Testbench

Each tile contains a set of memories that the customer is responsible for implementing using technology-specific memory macros. For a comprehensive list of memories, refer to the [Integration Guide](#). A SRAM integration testbench is provided to provide confidence that the customer-implemented memories match the expected behavior of the memory.

The expectation is to incorporate technology-specific memories into the Tenstorrent memory wrappers found in `$TENSIX_ROOT/rtl/primitives/wrappers/`. The wrappers use the naming convention `tt_mem_wrap_<mem_config>.sv`.

For verification of correct memory macro replacement, utilize tests in the `tb` directory under `$TENSIX_ROOT/rtl/primitives/wrappers/`. This directory houses a cocotb-python file and a simple wrapper that instantiates each memory. The cocotb concurrently tests all memories.

Running the Test

To execute the test:

1. Activate your virtual environment refer to [Cocotb Setup](### Cocotb Setup). a similar flow could be ran from `$TENSIX_ROOT/rtl/primitives/wrappers/tb`
2. Inside `$TENSIX_ROOT/rtl/primitives/wrappers/tb`, run `make`. This compiles the RTL and executes a sequence of memory tests
3. Verify that each test passes.

It's advisable to perform this test before and after memory swaps. Both runs should be successful.

The tests performed are:

Test Name	Description
fill_entire_memc	Writes one to each bit of every memory index
zero_entire_memc	Writes zero to each bit of every memory index
incremental_acce	Writes the index of each word to all indices in the memory
back_to_back_wri	Executes back-to-back write-to-read access
bit_enables	Applies only to memories with bit-masks. Tests the bit mask for each index by writing every bit to memory with mask 0xa, followed by mask 0x5
simultaneous_acc	Applies only to dual-port memories. Confirms that a memory can be accessed from both ports simultaneously

REGISTER MAPS

3.1 NOC2AXI Register Map Reference Guide

This document details the registers within the NOC2AXI block, broken down by sub-module. The overall base address for this block is **0x20000000**.

3.1.1 Instantiated Modules Summary

Module Name	Base Address	Description
<code>noc_niu_risc</code>	0x20000000	NIU, Config, Status, Security Registers
<code>noc2axi_local</code>	0x2001000	Local NOC2AXI Control Registers
<code>noc_address_translation_table</code>	0x2010000	Address Translation & Routing Tables

(Note: The original document grouped many conceptually separate blocks like NIU, Config, Status, and Security Fence under a single 'noc_niu_risc' heading and base address. This formatting retains that structure but separates the detailed definitions logically where possible.)

3.1.2 Address Map: `noc_niu_risc`

Description: Combined NIU, Configuration, Status, and Security Registers.

Base Offset: **0x20000000**

Register Instance Summary

*(Note: Registers listed as `SENT_` are read-only aliases of the corresponding registers without the `SENT_` prefix)**
(Note: Registers listed with `_0..N` indicate multiple instances)

Instance Name	Relative Offset
<code>TARGET_ADDR_LO</code>	0x0
<code>TARGET_ADDR_MID</code>	0x4
<code>TARGET_ADDR_HI</code>	0x8
<code>RET_ADDR_LO</code>	0xC
<code>RET_ADDR_MID</code>	0x10
<code>RET_ADDR_HI</code>	0x14
<code>PACKET_TAG</code>	0x18
<code>CMD_BRCST</code>	0x1C
<code>AT_LEN</code>	0x20

continues on next page

Table 1 – continued from previous page

Instance Name	Relative Offset
AT_LEN_1	0x24
AT_DATA	0x28
BRCST_EXCLUDE	0x2C
L1_ACC_AT_INSTRN	0x30
SECURITY_CTRL	0x34
CMD_CTRL	0x40
NODE_ID	0x44
ENDPOINT_ID	0x48
NUM_MEM_PARITY_ERR	0x50
NUM_HEADER_1B_ERR	0x54
NUM_HEADER_2B_ERR	0x58
ECC_CTRL	0x5C
NOC_CLEAR_OUTSTANDING_REQ_CNT	0x60
CMD_BUF_AVAIL	0x64
CMD_BUF_OVFL	0x68
SENT_TARGET_ADDR_LO	0x80
SENT_TARGET_ADDR_MID	0x84
SENT_TARGET_ADDR_HI	0x88
SENT_RET_ADDR_LO	0x8C
SENT_RET_ADDR_MID	0x90
SENT_RET_ADDR_HI	0x94
SENT_PACKET_TAG	0x98
SENT_CMD_BRCST	0x9C
SENT_AT_LEN	0xA0
SENT_AT_LEN_1	0xA4
SENT_AT_DATA	0xA8
SENT_BRCST_EXCLUDE	0xAC
SENT_L1_ACC_AT_INSTRN	0xB0
SENT_SECURITY_CTRL	0xB4
NIU_CONFIG	0x100
ROUTER_CONFIG	0x104
BROADCAST_CONFIG_0	0x108
BROADCAST_CONFIG_1	0x10C
BROADCAST_CONFIG_2	0x110
BROADCAST_CONFIG_3	0x114
MEM_SHUTDOWN_CONTROL	0x118
DEBUG_COUNTER_RESET	0x11C
MST_IF_INTP_TRANS_COUNT_RTZ_CFG	0x120
MST_IF_INTP_TRANS_COUNT_RTZ_CLR	0x124
VC_DIM_ORDER	0x128
THROTTLER_CYCLES_PER_WINDOW	0x12C
THROTTLER_HANDSHAKES_PER_WINDOW_NIU	0x130
THROTTLER_HANDSHAKES_PER_WINDOW_NORTH	0x134
THROTTLER_HANDSHAKES_PER_WINDOW_EAST	0x138
THROTTLER_HANDSHAKES_PER_WINDOW_SOUTH	0x13C
THROTTLER_HANDSHAKES_PER_WINDOW_WEST	0x140
NIU_TIMEOUT_DETECTED	0x144
NIU_TIMEOUT_VALUE_0	0x148
NIU_TIMEOUT_VALUE_1	0x14C
INVALID_FENCE_START_ADDR_LO_0	0x150
INVALID_FENCE_START_ADDR_HI_0	0x154
INVALID_FENCE_END_ADDR_LO_0	0x158
INVALID_FENCE_END_ADDR_HI_0	0x15C
INVALID_FENCE_START_ADDR_LO_1	0x160
INVALID_FENCE_START_ADDR_HI_1	0x164

continues on next page

Table 1 – continued from previous page

Instance Name	Relative Offset
INVALID_FENCE_END_ADDR_LO_1	0x168
INVALID_FENCE_END_ADDR_HI_1	0x16C
... (Instances 2-5)	...
INVALID_FENCE_START_ADDR_LO_5	0x1A0
INVALID_FENCE_START_ADDR_HI_5	0x1A4
INVALID_FENCE_END_ADDR_LO_5	0x1A8
INVALID_FENCE_END_ADDR_HI_5	0x1AC
VC_THROTTLER_HANDSHAKES_PER_WINDOW_0	0x1B0
VC_THROTTLER_HANDSHAKES_PER_WINDOW_1	0x1B4
VC_THROTTLER_HANDSHAKES_PER_WINDOW_2	0x1B8
VC_THROTTLER_HANDSHAKES_PER_WINDOW_3	0x1BC
VC_THROTTLER_CYCLES_PER_WINDOW	0x1C0
NIU_MST_ATOMIC_RESP_RECEIVED	0x200
NIU_MST_WR_ACK_RECEIVED	0x204
NIU_MST_RD_RESP_RECEIVED	0x208
NIU_MST_RD_DATA_WORD_RECEIVED	0x20C
NIU_MST_CMD_ACCEPTED	0x210
NIU_MST_RD_REQ_SENT	0x214
NIU_MST_NONPOSTED_ATOMIC_SENT	0x218
NIU_MST_POSTED_ATOMIC_SENT	0x21C
NIU_MST_NONPOSTED_WR_DATA_WORD_SENT	0x220
NIU_MST_POSTED_WR_DATA_WORD_SENT	0x224
NIU_MST_NONPOSTED_WR_REQ_SENT	0x228
NIU_MST_POSTED_WR_REQ_SENT	0x22C
NIU_MST_NONPOSTED_WR_REQ_STARTED	0x230
NIU_MST_POSTED_WR_REQ_STARTED	0x234
NIU_MST_RD_REQ_STARTED	0x238
NIU_MST_NONPOSTED_ATOMIC_STARTED	0x23C
NIU_MST_REQS_OUTSTANDING_ID_0	0x240
... (Instances 1-15)	...
NIU_MST_REQS_OUTSTANDING_ID_15	0x27C
NIU_MST_WRITE_REQS_OUTGOING_ID_0	0x280
... (Instances 1-15)	...
NIU_MST_WRITE_REQS_OUTGOING_ID_15	0x2BC
NIU_SLV_ATOMIC_RESP_SENT	0x2C0
NIU_SLV_WR_ACK_SENT	0x2C4
NIU_SLV_RD_RESP_SENT	0x2C8
NIU_SLV_RD_DATA_WORD_SENT	0x2CC
NIU_SLV_REQ_ACCEPTED	0x2D0
NIU_SLV_RD_REQ_RECEIVED	0x2D4
NIU_SLV_NONPOSTED_ATOMIC_RECEIVED	0x2D8
NIU_SLV_POSTED_ATOMIC_RECEIVED	0x2DC
NIU_SLV_NONPOSTED_WR_DATA_WORD_RECEIVED	0x2E0
NIU_SLV_POSTED_WR_DATA_WORD_RECEIVED	0x2E4
NIU_SLV_NONPOSTED_WR_REQ_RECEIVED	0x2E8
NIU_SLV_POSTED_WR_REQ_RECEIVED	0x2EC
NIU_SLV_NONPOSTED_WR_REQ_STARTED	0x2F0
NIU_SLV_POSTED_WR_REQ_STARTED	0x2F4
MST_IF_INTP_TRANS_COUNT_RTZ_NUM	0x2F8
MST_IF_INTP_TRANS_COUNT_RTZ_SOURCE	0x2FC
SEC_FENCE_RANGE_0_START_LO	0x400
SEC_FENCE_RANGE_0_START_HI	0x404
SEC_FENCE_RANGE_0_END_LO	0x408
SEC_FENCE_RANGE_0_END_HI	0x40C
... (Instances 1-7)	...

continues on next page

Table 1 – continued from previous page

Instance Name	Relative Offset
SEC_FENCE_RANGE_7_START_LO	0x470
SEC_FENCE_RANGE_7_START_HI	0x474
SEC_FENCE_RANGE_7_END_LO	0x478
SEC_FENCE_RANGE_7_END_HI	0x47C
SEC_FENCE_ATTRIBUTE_0	0x480
... (Instances 1-7)	...
SEC_FENCE_ATTRIBUTE_7	0x49C
SEC_FENCE_MASTER_LEVEL	0x4A0
SEC_FENCE_VIOLATION_FIFO_STATUS	0x4A4
SEC_FENCE_VIOLATION_FIFO_RDDATA	0x4A8
SEC_FENCE_DEFAULT_RANGE_ATTRIBUTE	0x4EC
SEC_FENCE_DEFAULT_RANGE_START_LO	0x4F0
SEC_FENCE_DEFAULT_RANGE_START_HI	0x4F4
SEC_FENCE_DEFAULT_RANGE_END_LO	0x4F8
SEC_FENCE_DEFAULT_RANGE_END_HI	0x4FC

Register Definitions

(Note: Many definitions are based on standard NOC blocks and may share descriptions. SENT_ registers are read-only aliases.)*

Register: TARGET_ADDR_LO

Field Name	Bit Range	SW Access	Reset	Description
TARGET_ADDR_LO	[31:0]	rw	0x0	Lower 32 bits of target address.

Register: TARGET_ADDR_MID

Field Name	Bit Range	SW Access	Reset	Description
TARGET_ADDR_MID	[27:0]	rw	0x0	Middle bits (e.g., [59:32]) of target address.

Register: TARGET_ADDR_HI

Field Name	Bit Range	SW Access	Reset	Description
TARGET_ADDR_HI	[31:0]	rw	0x0	Upper bits (e.g., [91:60]) of target address including coordinates.

Register: RET_ADDR_LO

Field Name	Bit Range	SW Access	Reset	Description
RET_ADDR_LO	[31:0]	rw	0x0	Lower 32 bits of return address.

Register: RET_ADDR_MID

Field Name	Bit Range	SW Access	Reset	Description
RET_ADDR_MID	[27:0]	rw	0x0	Middle bits (e.g., [59:32]) of return address.

Register: RET_ADDR_HI

Field Name	Bit Range	SW Access	Access	Reset	Description
RET_ADDR_HI	[31:0]	rw		0x0	Upper bits (e.g., [91:60]) of return address including coordinates.

Register: PACKET_TAG

Field Name	Bit Range	SW Access	Reset	Description
PACKET_TAG	[15:0]	rw	0x0	Tag associated with the packet/transaction.

Register: CMD_BRCST

Field Name	Bit Range	SW Access	Reset	Description
CMD_BRCST	[31:0]	rw	0x0	Command and Broadcast control bits for header.

Register: AT_LEN

Field Name	Bit Range	SW Access	Reset	Description
AT_LEN	[31:0]	rw	0x0	Atomic operation length/control 0.

Register: AT_LEN_1

Field Name	Bit Range	SW Access	Reset	Description
AT_LEN_1	[31:0]	rw	0x0	Atomic operation length/control 1.

Register: AT_DATA

Field Name	Bit Range	SW Access	Reset	Description
AT_DATA	[31:0]	rw	0x0	Atomic operation data.

Register: BRCST_EXCLUDE

Field Name	Bit Range	SW Access	Reset	Description
BRCST_EXCLUDE	[23:0]	rw	0x0	Broadcast exclude coordinates.

Register: L1_ACC_AT_INSTRN

Field Name	Bit Range	SW Access	Reset	Description
L1_ACC_AT_INSTRN	[15:0]	rw	0x0	L1 Accumulator Atomic Instruction.

Register: SECURITY_CTRL

Field Name	Bit Range	SW Access	Reset	Description
RSVD	[31:4]	r	0x0	Reserved
SEC_FENCE_MASTER_	[3:0]	r	0x0	Security Fence master level from programming. Read-only in header.

Register: CMD_CTRL

Field Name	Bit Range	SW Access	Reset	Description
CMD_PENDING	[31:0]	rw	0x0	Command pending.

Register: NODE_ID

Field Name	Bit Range	SW Access	Reset	Description
NODE_ID_X	[5:0]	r	0x0	Local Node X ID
NODE_ID_Y	[11:6]	r	0x0	Local Node Y ID
NOC_X_SIZE	[18:12]	r	0x0	NOC X Dimension Size
NOC_Y_SIZE	[25:19]	r	0x0	NOC Y Dimension Size
DATELINE_NODE_X	[26:26]	r	0x0	X Dateline Wrap Node ID
DATELINE_NODE_Y	[27:27]	r	0x0	Y Dateline Wrap Node ID
ROUTING_DIM_ORDER_XY	[28:28]	r	0x0	Routing Dimension Order (XY)

Register: ENDPOINT_ID

Field Name	Bit Range	SW Access	Reset	Description
ENDPOINT_ID	[31:0]	r	-	Endpoint ID

Register: NUM_MEM_PARITY_ERR

Field Name	Bit Range	SW Access	Reset	Description
NUM_MEM_PARITY_ERR	[15:0]	r	0x0	Memory Parity Error Count

Register: NUM_HEADER_1B_ERR

Field Name	Bit Range	SW Access	Reset	Description
NUM_HEADER_1B_ERR	[15:0]	r	0x0	Header 1-Bit Error Count

Register: NUM_HEADER_2B_ERR

Field Name	Bit Range	SW Access	Reset	Description
NUM_HEADER_2B_ERR	[15:0]	r	0x0	Header 2-Bit Error Count

Register: ECC_CTRL

Field Name	Bit Range	SW Access	Reset	Description
ECC_ERR_CLEAR	[2:0]	w	0x0	Clear ECC Error Flags
ECC_ERR_FORCE	[5:3]	w	0x0	Force ECC Error Injection

Register: NOC_CLEAR_OUTSTANDING_REQ_CNT

Field Name	Bit Range	SW Access	Ac-cess	Re-set	Description
NOC_CLEAR_OUTSTANDING_REQ	[15:0]	rw		0x0	Clear Outstanding Request Counter Bits

Register: CMD_BUF_AVAIL

Field Name	Bit Range	SW Access	Reset	Description
RSVD3	[31:29]	r	0x0	Reserved
BUF3_AVAIL	[28:24]	r	0x0	Available entries in Buffer 3
RSVD2	[23:21]	r	0x0	Reserved
BUF2_AVAIL	[20:16]	r	0x0	Available entries in Buffer 2
RSVD1	[15:13]	r	0x0	Reserved
BUF1_AVAIL	[12:8]	r	0x0	Available entries in Buffer 1
RSVD0	[7:5]	r	0x0	Reserved
BUF0_AVAIL	[4:0]	r	0x0	Available entries in Buffer 0

Register: CMD_BUF_OVFL

Field Name	Bit Range	SW Access	Reset	Description
RSVD	[31:4]	r	0x0	Reserved
BUF3_OVFL	[3:3]	r	0x0	Buffer 3 Overflow
BUF2_OVFL	[2:2]	r	0x0	Buffer 2 Overflow
BUF1_OVFL	[1:1]	r	0x0	Buffer 1 Overflow
BUF0_OVFL	[0:0]	r	0x0	Buffer 0 Overflow

Register: SENT_TARGET_ADDR_LO

Description: Read-only alias of TARGET_ADDR_LO.

Field Name	Bit Range	SW Access	Reset	Description
TARGET_ADDR_LO	[31:0]	r	0x0	Lower 32 bits of target address.

Register: SENT_TARGET_ADDR_MID

Description: Read-only alias of TARGET_ADDR_MID.

Field Name	Bit Range	SW Access	Reset	Description
TARGET_ADDR_MID	[27:0]	r	0x0	Middle bits (e.g., [59:32]) of target address.

Register: SENT_TARGET_ADDR_HI

Description: Read-only alias of TARGET_ADDR_HI.

Field Name	Bit Range	SW Access	Access	Reset	Description
TARGET_ADDR_HI	[31:0]	r		0x0	Upper bits (e.g., [91:60]) of target address including coordinates.

Register: SENT_RET_ADDR_LO

Description: Read-only alias of RET_ADDR_LO.

Field Name	Bit Range	SW Access	Reset	Description
RET_ADDR_LO	[31:0]	r	0x0	Lower 32 bits of return address.

Register: SENT_RET_ADDR_MID

Description: Read-only alias of RET_ADDR_MID.

Field Name	Bit Range	SW Access	Reset	Description
RET_ADDR_MID	[27:0]	r	0x0	Middle bits (e.g., [59:32]) of return address.

Register: SENT_RET_ADDR_HI

Description: Read-only alias of RET_ADDR_HI.

Field Name	Bit Range	SW Access	Access	Reset	Description
RET_ADDR_HI	[31:0]	r		0x0	Upper bits (e.g., [91:60]) of return address including coordinates.

Register: SENT_PACKET_TAG

Description: Read-only alias of PACKET_TAG.

Field Name	Bit Range	SW Access	Reset	Description
PACKET_TAG	[15:0]	r	0x0	Tag associated with the packet/transaction.

Register: SENT_CMD_BRCST

Description: Read-only alias of CMD_BRCST.

Field Name	Bit Range	SW Access	Reset	Description
CMD_BRCST	[31:0]	r	0x0	Command and Broadcast control bits for header.

Register: SENT_AT_LEN

Description: Read-only alias of AT_LEN.

Field Name	Bit Range	SW Access	Reset	Description
AT_LEN	[31:0]	r	0x0	Atomic operation length/control 0.

Register: SENT_AT_LEN_1

Description: Read-only alias of AT_LEN_1.

Field Name	Bit Range	SW Access	Reset	Description
AT_LEN_1	[31:0]	r	0x0	Atomic operation length/control 1.

Register: SENT_AT_DATA

Description: Read-only alias of AT_DATA.

Field Name	Bit Range	SW Access	Reset	Description
AT_DATA	[31:0]	r	0x0	Atomic operation data.

Register: SENT_BRCST_EXCLUDE

Description: Read-only alias of BRCST_EXCLUDE.

Field Name	Bit Range	SW Access	Reset	Description
BRCST_EXCLUDE	[23:0]	r	0x0	Broadcast exclude coordinates.

Register: SENT_L1_ACC_AT_INSTRN

Description: Read-only alias of L1_ACC_AT_INSTRN.

Field Name	Bit Range	SW Access	Reset	Description
L1_ACC_AT_INSTRN	[15:0]	r	0x0	L1 Accumulator Atomic Instruction.

Register: SENT_SECURITY_CTRL**Description:** Read-only alias of SECURITY_CTRL.

Field Name	Bit Range	SW Access	Reset	Description
RSVD	[31:4]	r	0x0	Reserved
SEC_FENCE_MASTER_	[3:0]	r	0x0	Security Fence master level from programming. Read-only in header.

Register: NIU_CONFIG

Field Name	Bit Range	SW Access	Reset	Description
CLK_GATING_DISABLED	[0:0]	rw	0x0	Disable clock gating
CLK_GATER_HYSTERESIS	[7:1]	rw	0x2	Clock gater hysteresis value
ECC_NIU_MEM_PARITY_ENABLE	[8:8]	rw	0x0	Enable NIU Memory Parity/ECC
ECC_INTERRUPT_ENABLE	[11:9]	rw	0x0	ECC Interrupt Enables
TILE_CLOCK_DISABLE	[12:12]	rw	0x0	Disable Tile Clock
NOC2AXI_TILE_HEADER_DOUBL	[13:13]	rw	0x0	Disable NOC2AXI Header Double Write
NIU_TIMEOUT_IRQ_EN	[14:14]	rw	0x0	Enable NIU Timeout Interrupt
AXI_ENABLE	[15:15]	rw	0x0	Enable AXI ports (disables L1 ports). Delayed effect (~12 cycles).
CMD_BUFFER_FIFO_EN	[16:16]	rw	0x0	Enable usage of NIU Command Buffer FIFO (otherwise passthrough).

Register: ROUTER_CONFIG

Field Name	Bit Range	SW Access	Reset	Description
CLK_GATING_DISABLED	[0:0]	rw	0x0	Disable router clock gating
CLK_GATER_HYSTERESIS	[7:1]	rw	0x2	Router clock gater hysteresis
MAX_BACKOFF_EXP	[11:8]	rw	0x8	Maximum backoff exponent
BASIC_TIMEOUT_DELAY	[15:12]	rw	0x5	Log2 of basic timeout delay
ECC_ROUTER_MEM_PARITY_ENABE	[16:16]	rw	0x0	Enable Router Memory Parity/ECC
ECC_HEADER_CHK_BITS_ENABLE	[17:17]	rw	0x0	Enable Header Check Bits (ECC)
ECC_HEADER_SECDDED_CHK_CORREC	[18:18]	rw	0x0	Enable Header SECDDED Correct (ECC)

Register: BROADCAST_CONFIG

Field Name	Bit Range	SW Access	Reset	Description
BCAST_ROW_DISABLE BCAST_COL_DISABLE	/ [31:0]	rw	0x0	Bitmask to disable broadcast rows/cols.

Register: MEM_SHUTDOWN_CONTROL

Field Name	Bit Range	SW Access	Reset	Description
RSVD	[31:3]	rw	0x0	Reserved
DSLPLV_BIT	[2:2]	rw	0x0	Deep Sleep Retention Diode Mode
DSLPLV_BIT	[1:1]	rw	0x0	Deep Sleep Mode
SD_BIT	[0:0]	rw	0x0	Shut Down Mode

Register: DEBUG_COUNTER_RESET

Description: Programmable reset to various debug counters.

Field Name	Bit Range	SW Access	Reset	Description
RSVD1	[31:5]	r	0x0	Reserved
CMD_BUFFER_FIFO_OVFL_CL	[4:4]	rw	0x0	Write 1 to clear the Command Buffer FIFO Overflow flag. Self-clearing.
RSVD0	[3:1]	r	0x0	Reserved
ROUTER_OUTGOING_FLIT_CC	[0:0]	rw	0x0	Write 1 to reset the Router Outgoing Flit Counters. Self-clearing.

Register: MST_IF_INTP_TRANS_COUNT_RTZ_CFG

Description: Config register to control the MST_IF TAG_ID Transaction Counter Return-to-Zero Interrupt.

Field Name	Bit Range	SW Access	Reset	Description
RSVD1	[31:2]	r	0x0	Reserved
CLEAR_ON_F	[28:2]	r	0x0	If set, Clear-On-Read is disabled for MST_IF_INTP_TRANS_COUNT_RTZ_SOURCE.BIT_MAP. If not set (default), reading RTZ_NUM clears the corresponding bit.
RSVD0	[27:1]	r	0x0	Reserved
INTER- RUPT_ENABL	[15:0]	rw	0x0	Per-TagID interrupt enable for return-to-zero event.

Register: MST_IF_INTP_TRANS_COUNT_RTZ_CLR

Description: Config register to clear the detection of MST_IF TAG_ID Transaction Counter Return-to-Zero event. Writing a 1 clears the corresponding bit in RTZ_SOURCE. Self-clearing write pulse. Reads 0.

Field Name	Bit Range	SW Access	Reset	Description
RSVD0	[31:16]	r	0x0	Reserved
CLEAR	[15:0]	rw	0x0	Bitmask to clear detected events

Register: VC_DIM_ORDER

Description: A bit map that specifies whether a particular VC will route x then y, or y then x. Each bit corresponds to a particular VC.

Field Name	Bit Range	SW Access	Reset	Description
data	[31:0]	rw	0xAAAAAAAA	Bit per VC: 0=XY routing, 1=YX routing.

Register: THROTTLER_CYCLES_PER_WINDOW

Description: The number of cycles within a throttler window.

Field Name	Bit Range	SW Access	Reset	Description
data	[31:0]	rw	0x0	Defines the window size for port/VC throttlers. Set to 0 to disable. See examples.

Register: THROTTLER_HANDSHAKES_PER_WINDOW

Description: The number of handshakes allowed within a throttler window for a specific port (NIU, NORTH, EAST, SOUTH, WEST).

Field Name	Bit Range	SW Access	Reset	Description
data	[31:0]	rw	0x0	Defines allowed handshakes per window for port throttling. See examples in CYCLES reg.

Register: NIU_TIMEOUT_DETECTED

Description: Whether a timeout has been detected on a non-posted transaction.

Field Name	Bit Range	SW Access	Reset	Description
data	[0:0]	rw	0x0	Timeout status. Write 1 to clear.

Register: NIU_TIMEOUT_VALUE

Description: The amount of NoC clock cycles that must pass without response for a non-posted transaction to trigger a timeout. Instantiated twice.

Field Name	Bit Range	SW Access	Reset	Description
data	[31:0]	rw	0x0	Timeout value in cycles. 0 disables the feature.

Register: INVALID_FENCE_START_ADDR_LO

Description: Lower 32 bits of the start address for an invalid access fence.

Field Name	Bit Range	SW Access	Reset	Description
start_addr	[31:0]	rw	0xFFFFFFFF	Any address within [start_addr, end_addr] (inclusive) is rejected and logged.

Register: INVALID_FENCE_START_ADDR_HI

Description: Upper 32 bits of the start address for an invalid access fence.

Field Name	Bit Range	SW Access	Reset	Description
start_addr	[31:0]	rw	0xFFFFFFFF	Any address within [start_addr, end_addr] (inclusive) is rejected and logged.

Register: INVALID_FENCE_END_ADDR_LO

Description: Lower 32 bits of the end address for an invalid access fence.

Field Name	Bit Range	SW Access	Reset	Description
end_addr	[31:0]	rw	0x0	Any address within [start_addr, end_addr] (inclusive) is rejected and logged.

Register: INVALID_FENCE_END_ADDR_HI

Description: Upper 32 bits of the end address for an invalid access fence.

Field Name	Bit Range	SW Access	Reset	Description
end_addr	[31:0]	rw	0x0	Any address within [start_addr, end_addr] (inclusive) is rejected and logged.

Register: VC_THROTTLER_HANDSHAKES_PER_WINDOW

Description: The number of handshakes within a throttler window per VC for NIU output flits. Instantiated four times.

Field Name	Bit Range	SW Access	Reset	Description
vc0	[7:0]	rw	0x0	Handshakes allowed for VC0
vc1	[15:8]	rw	0x0	Handshakes allowed for VC1
vc2	[23:16]	rw	0x0	Handshakes allowed for VC2
vc3	[31:24]	rw	0x0	Handshakes allowed for VC3

Register: VC_THROTTLER_CYCLES_PER_WINDOW

Description: The number of cycles within a throttler window for all VCs for NIU output flits.

Field Name	Bit Range	SW Access	Reset	Description
data	[7:0]	rw	0x0	Window size for per-VC throttling. Set to 0 to disable. See examples in HANDSHAKES.

(*Status Registers* NIU_MST_, NIU_SLV_, MST_IF_ Definitions: These generally report counts or status flags related to NIU operations. Most are 32-bit read-only registers for SW.)* (*Security Fence Registers* SEC_FENCE_ Definitions: These define address ranges and attributes for access control. Details depend on specific RDL but involve start/end addresses and attribute settings.)*

3.1.3 Address Map: noc2axi_local

Description: Local NOC2AXI Control Registers

Base Offset: 0x2001000

Register Instance Summary

Instance Name	Relative Offset
MST_WR_MEM_FULL_THRESH	0x0
MST_RD_MEM_FULL_THRESH	0x4
MST_WR_DISABLE_BYTE_ENABLE	0x8
SCRATCH_0	0x10
SCRATCH_1	0x14
SCRATCH_2	0x18
SCRATCH_3	0x1C
AXI2NOC_HEADER_TLB_START_ADDR_LO_0	0x20
AXI2NOC_HEADER_TLB_START_ADDR_HI_0	0x24
AXI2NOC_HEADER_TLB_END_ADDR_LO_0	0x28
AXI2NOC_HEADER_TLB_END_ADDR_HI_0	0x2C
... (Instances 1-15)	...
AXI2NOC_HEADER_TLB_START_ADDR_LO_15	0x110
AXI2NOC_HEADER_TLB_START_ADDR_HI_15	0x114
AXI2NOC_HEADER_TLB_END_ADDR_LO_15	0x118
AXI2NOC_HEADER_TLB_END_ADDR_HI_15	0x11C
CHICKEN_REG	0x120
DISABLE_AUTOINLINE	0x124
DISABLE_SLV_RD_MEM_CHECK	0x128
AXI2NOC_MST_LOW_BOUNDARY_ADDR_LO	0x12C
AXI2NOC_MST_LOW_BOUNDARY_ADDR_HI	0x130
AXI2NOC_MST_HIGH_BOUNDARY_ADDR_LO	0x134
AXI2NOC_MST_HIGH_BOUNDARY_ADDR_HI	0x138
ENABLE_AXI_ERR	0x13C

Register Definitions

Register: MST_WR_MEM_FULL_THRESH

Field Name	Bit Range	SW Access	Re-set	Description
mst_wr_mem_	[11:0]	rw	16	Threshold (in words) for AXI AWREADY/WREADY deassertion. Deasserts when empty buffer words < threshold.

Register: MST_RD_MEM_FULL_THRESH

Field Name	Bit Range	SW Access	Re-set	Description
mst_rd_mem_t	[11:0]	rw	16	Threshold (in words) for AXI ARREADY deassertion. Deasserts when empty buffer words < threshold.

Register: MST_WR_DISABLE_BYTE_ENABLE

Field Name	Bit Range	SW Access	Reset	Description
mst_wr_disable_byt	[0:0]	rw	0	0: Use AXI byte enables (WSTRB). 1: Disable byte enables (assume full word writes).

Register: SCRATCH

Description: Scratch register for SW use.

Field Name	Bit Range	SW Access	Reset	Description
scratch	[31:0]	rw	0	General purpose storage.

(AXI2NOC_HEADER_TLB Registers: These define 16 Translation Lookaside Buffer entries. Each entry uses 4 registers (_START_ADDR_LO/HI, _END_ADDR_LO/HI) to define an address range for header lookup.)

Register: AXI2NOC_HEADER_TLB_START_ADDR_LO

Field Name	Bit Range	SW Access	Reset	Description
start_addr	[31:0]	rw	var	Lower 32 bits of TLB entry start address.

Register: AXI2NOC_HEADER_TLB_START_ADDR_HI

Field Name	Bit Range	SW Access	Reset	Description
start_addr	[31:0]	rw	var	Upper 32 bits of TLB entry start address.

Register: AXI2NOC_HEADER_TLB_END_ADDR_LO

Field Name	Bit Range	SW Access	Reset	Description
end_addr	[31:0]	rw	var	Lower 32 bits of TLB entry end address.

Register: AXI2NOC_HEADER_TLB_END_ADDR_HI

Field Name	Bit Range	SW Access	Reset	Description
end_addr	[31:0]	rw	var	Upper 32 bits of TLB entry end address.

Register: CHICKEN_REG

Field Name	Bit Range	SW Access	Re-set	Description
disable_axi_resp	[0:0]	rw	0	0: Pass BRESP/RRESP from NOC slave. 1: Force OKAY response (legacy).
detected_rdata_i	[1:1]	r	-	Read data was SLVERR/DECERR (extremely rare case where this indicates false error if disable_axi_resps=0).

Register: DISABLE_AUTOINLINE

Field Name	Bit Range	SW Access	Re-set	Description
disable_autoinline_wr_re	[0:0]	rw	0	1: Disable generating autoinline write requests.
disable_autoinline_rd_re	[1:1]	rw	0	1: Disable generating autoinline read responses.

Register: DISABLE_SLV_RD_MEM_CHECK

Field Name	Bit Range	SW Access	Re-set	Description
disable_slv_rd_m	[0:0]	rw	0	1: Send read request without checking internal memory space (potential overflow if not careful).

Register: AXI2NOC_MST_LOW_BOUNDARY_ADDR_LO

Field Name	Bit Range	SW Access	Re-set	Description
low_addr_bou	[31:0]	rw	0x0	Lower 32 bits of the low address boundary for AXI master access. Access below causes decode error.

Register: AXI2NOC_MST_LOW_BOUNDARY_ADDR_HI

Field Name	Bit Range	SW Access	Re-set	Description
low_addr_bou	[31:0]	rw	0x0	Upper 32 bits of the low address boundary for AXI master access. Access below causes decode error.

Register: AXI2NOC_MST_HIGH_BOUNDARY_ADDR_LO

Field Name	Bit Range	SW Access	Reset	Description
high_addr_bo	[31:0]	rw	0xFFFFF	Lower 32 bits of the high address boundary for AXI master access. Access above causes decode error.

Register: AXI2NOC_MST_HIGH_BOUNDARY_ADDR_HI

Field Name	Bit Range	SW Access	Reset	Description
high_addr_bo	[31:0]	rw	0xFFFFF	Upper 32 bits of the high address boundary for AXI master access. Access above causes decode error.

Register: ENABLE_AXI_ERR

Field Name	Bit Range	SW Access	Reset	Description
enable_axi	[0:0]	rw	0	1: Enable AXI master transaction error checking (boundary, TLB). 0: Disable checking (bypass axi_err module).

3.1.4 Address Map: noc_address_translation_table**Description:** Address Translation & Dynamic Routing Tables**Base Offset:** 0x2010000**Register Instance Summary**

Instance Name	Relative Offset
ENABLE_TABLES	0x0
CLK_GATING	0x4
DEBUG_LAST_SRC_ADDR_LO	0x8
DEBUG_LAST_SRC_ADDR_HI	0xC
DEBUG_LAST_DEST_ADDR_LO	0x10
DEBUG_LAST_DEST_ADDR_HI	0x14
DEBUG_LAST_TRANS_SRC_ADDR_LO	0x18
DEBUG_LAST_TRANS_SRC_ADDR_HI	0x1C
DEBUG_LAST_TRANS_DEST_ADDR_LO	0x20
DEBUG_LAST_TRANS_DEST_ADDR_HI	0x24
DEBUG_LAST_TRANSLATION	0x28

continues on next page

Table 2 – continued from previous page

Instance Name	Relative Offset
DEBUG_MISC	0x2C
MASK_TABLE_ENTRY_0	0x30
MASK_TABLE_EP_LO_0	0x38
MASK_TABLE_EP_HI_0	0x3C
MASK_TABLE_BAR_LO_0	0x40
MASK_TABLE_BAR_HI_0	0x44
... (Instances 1-15)	...
MASK_TABLE_ENTRY_15	0x198
MASK_TABLE_EP_LO_15	0x1A0
MASK_TABLE_EP_HI_15	0x1A4
MASK_TABLE_BAR_LO_15	0x1A8
MASK_TABLE_BAR_HI_15	0x1AC
ROUTING_TABLE_MATCH_0	0x200
... (Instances 1-31)	...
ROUTING_TABLE_MATCH_31	0x27C
ROUTING_TABLE_PART_ENTRY_0	0x300
... (Instances 1-31)	...
ROUTING_TABLE_PART_ENTRY_31	0x37C
ENDPOINT_TABLE_ENTRY_0	0x2000
... (Instances 1-1023)	...
ENDPOINT_TABLE_ENTRY_1023	0x2FFC

Register Definitions

Register: ENABLE_TABLES

Field Name	Bit Range	SW Access	Reset	Description
en_address_translation	[0:0]	rw	0	Enable address translation
en_dynamic_routing	[1:1]	rw	0	Enable dynamic routing

Register: CLK_GATING

Description: Clock gating control.

Field Name	Bit Range	SW Access	Reset	Description
clk_gating_enable	[0:0]	rw	0	Enable clock gating
clk_gating_hysteresis	[7:1]	rw	4	Clock gating hysteresis value

(Debug Registers *DEBUG_LAST_*: Contain values related to the last translation performed. Details omitted.)*

Register: MASK_TABLE_ENTRY

Field Name	Bit Range	SW Access	Reset	Description
mask	[5:0]	w	0	Bit position for masking address: Masked Addr = Addr & $\sim((1 \ll \text{mask}) - 1)$
ep_id_idx	[11:6]	w	0	Bit offset into address where Endpoint ID index starts.
ep_id_size	[17:12]	w	0	Size (in bits) of the Endpoint ID index.
table_offset	[27:18]	w	0	Offset added to Endpoint ID index to form lookup index into Endpoint Table.
translate_addr	[28:28]	w	0	1: Translate address using BAR. 0: Pass address unchanged.

Register: MASK_TABLE_EP_LO

Description: Lower 32 bits of the 64-bit masked endpoint address value to match for this table entry.

Field Name	Bit Range	SW Access	Reset	Description
data	[31:0]	w	0	Match value

Register: MASK_TABLE_EP_HI

Description: Upper 32 bits of the 64-bit masked endpoint address value to match for this table entry.

Field Name	Bit Range	SW Access	Reset	Description
data	[31:0]	w	0	Match value

Register: MASK_TABLE_BAR_LO

Description: Lower 32 bits of the 64-bit Base Address Register (BAR) used for address translation if `translate_addr` is set.

Field Name	Bit Range	SW Access	Reset	Description
data	[31:0]	w	0	Base address

Register: MASK_TABLE_BAR_HI

Description: Upper 32 bits of the 64-bit Base Address Register (BAR) used for address translation if `translate_addr` is set.

Field Name	Bit Range	SW Access	Reset	Description
data	[31:0]	w	0	Base address

Register: ROUTING_TABLE_MATCH

Description: Value to match against {table_offset + ep_id_idx, VC} for dynamic routing override.

Field Name	Bit Range	SW Access	Reset	Description
data	[15:0]	w	0	Match value for dynamic routing table entry lookup.

Register: ROUTING_TABLE_PART_ENTRY

Description: Partial 32-bits of a 64-byte dynamic routing table entry.

Field Name	Bit Range	SW Access	Reset	Description
data	[31:0]	w	0	Part of the dynamic routing table data.

Register: ENDPOINT_TABLE_ENTRY

Description: Contains the destination X/Y coordinates for a translated endpoint.

Field Name	Bit Range	SW Access	Reset	Description
x	[5:0]	wr	0	Destination X coordinate for endpoint
y	[11:6]	wr	0	Destination Y coordinate for endpoint

3.2 Tensix Debug Regs Memory map (per Tensix core)

The Tensix debug registers (per Tensix Core) could be found in the following addresse (As seen by the NoC)

Address Space	Description	Offset Range
tensix_l1	3 MB	0x00000000 - 0x002FFFFFFF
neo_regs[0]	Risc debug regs	0x00800000 - 0x00800253
neo_regs[1]	Risc debug regs	0x00810000 - 0x00810253
neo_regs[2]	Risc debug regs	0x00820000 - 0x00820253
neo_regs[3]	Risc debug regs	0x00830000 - 0x00830253
tensix_global_regs		0x00840000 - 0x00850FFF

Below is a description of all RISC debug registers. Their absolute address is the provided offset address plus the Tensix index debug_reg offset. For example, if the base address is 0x50, for Tensix[2], the absolute address would be:

$$0x00820000 + 0x50 = 0x00820050$$

3.2.1 RISC-V Debug Registers Summary Table

Register Name	Offset (Hex)	Notes
PERF_CNT_INSTRN_THREAD0	0x0	
PERF_CNT_INSTRN_THREAD1	0x4	
PERF_CNT_INSTRN_THREAD2	0x8	
PERF_CNT_TDMA_UNPACK0	0xc	
PERF_CNT_TDMA_UNPACK1	0x10	
PERF_CNT_TDMA_UNPACK2	0x14	
PERF_CNT_FPU0	0x18	
PERF_CNT_FPU1	0x1c	
PERF_CNT_FPU2	0x20	
PERF_CNT_ALL	0x24	
DBG_BUS_CTRL	0x28	
TENSIX_CREG_READ	0x2c	
DBG_RD_DATA	0x30	
DBG_ARRAY_RD_EN	0x34	
DBG_ARRAY_RD_CMD	0x38	
DBG_FEATURE_DISABLE	0x3c	
DBG_ARRAY_RD_DATA	0x40	
CG_CTRL_HYST0	0x44	
CG_CTRL_HYST1	0x48	
TENSIX_CREG_RDDATA	0x4c	
CG_CTRL_HYST2	0x50	
RISC_DBG_CNTL_0	0x54	
RISC_DBG_CNTL_1	0x58	
RISC_DBG_STATUS_0	0x5c	
RISC_DBG_STATUS_1	0x60	
TRISC_PC_BUF_OVERRIDE	0x64	
DBG_INVALID_INSTRN	0x68	
DBG_INSTRN_BUF_CTRL0	0x6c	
DBG_INSTRN_BUF_CTRL1	0x70	
DBG_INSTRN_BUF_STATUS	0x74	
STOCH_RND_MASK0	0x78	
STOCH_RND_MASK1	0x7c	
FPU_STICKY_BITS	0x80	
PERF_CNT_TDMA_PACK0	0x84	
PERF_CNT_TDMA_PACK1	0x88	
PERF_CNT_TDMA_PACK2	0x8c	
PERF_CNT_OUT_L_INSTRN_THREAD	0x90	
PERF_CNT_OUT_H_INSTRN_THREAD	0x94	
PERF_CNT_OUT_L_TDMA_UNPACK	0x98	
PERF_CNT_OUT_H_TDMA_UNPACK	0x9c	
PERF_CNT_OUT_L_TDMA_PACK	0xa0	
PERF_CNT_OUT_H_TDMA_PACK	0xa4	
PERF_CNT_OUT_L_FPU	0xa8	
PERF_CNT_OUT_H_FPU	0xac	
SOFT_RESET_0	0xb0	
SOFT_RESET_1	0xb4	
WATCHDOG_TIMER	0xb8	
WDT_CNTL	0xbc	
WDT_STATUS	0xc0	
WALL_CLOCK_0	0xc4	
WALL_CLOCK_1	0xc8	
WALL_CLOCK_1_AT	0xcc	
TIMESTAMP_DUMP_CMD	0xd0	

continues on next page

Table 3 – continued from previous page

Register Name	Offset (Hex)	Notes
TIMESTAMP_DUMP_CNTL	0xd4	
TIMESTAMP_DUMP_STATUS	0xd8	
TIMESTAMP_DUMP_BUF0_START_ADDR	0xdc	
TIMESTAMP_DUMP_BUF0_END_ADDR	0xe0	
TIMESTAMP_DUMP_BUF1_START_ADDR	0xe4	
TIMESTAMP_DUMP_BUF1_END_ADDR	0xe8	
PERF_CNT_MUX_CTRL	0xec	
LFSR_HIT_MASK	0xf0	
DISABLE_RESET	0xf4	
TRISC0_RESET_PC	0xf8	
TRISC1_RESET_PC	0xfc	
TRISC2_RESET_PC	0x100	
TRISC3_RESET_PC	0x104	
TRISC_RESET_PC_OVERRIDE	0x108	
DEST_CG_CTRL	0x10c	
CG_CTRL_EN	0x110	
CG_KICK	0x114	
DBG_IBUFFER_TIMEOUT	0x118	
DBG_IBUFFER_CNT_EN	0x11c	
TRISC0_BAR	0x120	Array[4]
TRISC0_BAR_SELECT	0x130	
TRISC1_BAR	0x134	Array[4]
TRISC1_BAR_SELECT	0x144	
TRISC2_BAR	0x148	Array[4]
TRISC2_BAR_SELECT	0x158	
TRISC3_BAR	0x15c	Array[4]
TRISC3_BAR_SELECT	0x16c	
TRISC_MAILBOX	0x170	Array[16]
DBG_HALT_CLR	0x1b0	
DBG_MOV2D_ADDR	0x1b4	
DBG_MOV2D	0x1b8	
DBG_TEMP_REGS	0x1bc	Array[32]
DBG_BACKDOOR_INST	0x23c	
DBG_BACKDOOR_INST_ISSUE	0x240	
DBG_TRISC_STACK_LIMIT	0x244	
RISCV_IC_INVALIDATE	0x248	
TRISC_END_PC	0x24c	Array[4]
RISC_PREFETCH_CTRL	0x25c	
RISC_BRANCH_PREDICTION_CTRL	0x260	
CHICKEN_BITS	0x264	
TENSIX_TRISC_SYNC	0x268	Array[4]
TENSIX_CSR_CONFIG	0x278	Array[4]
POWER_RAMP_PROFILE_CTRL	0x288	
POWER_RAMP_MISC_CTRL	0x28c	
ERR_MASK	0x290	
ERR_DATA	0x294	
FPU_SAFETY	0x298	

(The detailed register descriptions follow below...)

3.2.2 PERF_CNT_INSTRN_THREAD0 register

- Base Offset: 0x0
- Size: 0x4

Bits	Identifier	Access	Reset	Name
31:0	perf_cnt_instrn_thread0	rw	0x0	—

perf_cnt_instrn_thread0 field

3.2.3 PERF_CNT_INSTRN_THREAD1 register

- Base Offset: 0x4
- Size: 0x4

Bits	Identifier	Access	Reset	Name
31:0	perf_cnt_instrn_thread1	rw	0x0	—

perf_cnt_instrn_thread1 field

3.2.4 PERF_CNT_INSTRN_THREAD2 register

- Base Offset: 0x8
- Size: 0x4

Bits	Identifier	Access	Reset	Name
0:0	instrn_thread_start	rw	0x0	—
1:1	instrn_thread_stop	rw	0x0	—

instrn_thread_start field

instrn_thread_stop field

3.2.5 PERF_CNT_TDMA_UNPACK0 register

- Base Offset: 0xc
- Size: 0x4

Bits	Identifier	Access	Reset	Name
31:0	perf_cnt_tdma_unpack0	rw	0x0	—

perf_cnt_tdma_unpack0 field

3.2.6 PERF_CNT_TDMA_UNPACK1 register

- Base Offset: 0x10
- Size: 0x4

Bits	Identifier	Access	Reset	Name
31:0	perf_cnt_tdma_unpack1	rw	0x0	—

perf_cnt_tdma_unpack1 field

3.2.7 PERF_CNT_TDMA_UNPACK2 register

- Base Offset: 0x14
- Size: 0x4

Bits	Identifier	Access	Reset	Name
0:0	start	rw	0x0	—
1:1	stop	rw	0x0	—

start field

stop field

3.2.8 PERF_CNT_FPU0 register

- Base Offset: 0x18
- Size: 0x4

Bits	Identifier	Access	Reset	Name
31:0	fpu_ref_period	rw	0x0	—

fpu_ref_period field

3.2.9 PERF_CNT_FPU1 register

- Base Offset: 0x1c
- Size: 0x4

Bits	Identifier	Access	Reset	Name
31:0	fpu_mode	rw	0x0	—

fpu_mode field

3.2.10 PERF_CNT_FPU2 register

- Base Offset: 0x20
- Size: 0x4

Bits	Identifier	Access	Reset	Name
0:0	fpu_start	rw	0x0	—
1:1	fpu_stop	rw	0x0	—

fpu_start field

fpu_stop field

3.2.11 PERF_CNT_ALL register

- Base Offset: 0x24
- Size: 0x4

Bits	Identifier	Access	Reset	Name
0:0	instrn_thread_start_all	rw	0x0	—
1:1	instrn_thread_stop_all	rw	0x0	—

instrn_thread_start_all field

instrn_thread_stop_all field

3.2.12 DBG_BUS_CTRL register

- Base Offset: 0x28
- Size: 0x4

Bits	Identifier	Access	Reset	Name
15:0	risc_dbg_sig_sel	rw	0x0	—
23:16	risc_dbg_daisy_sel	rw	0x0	—
27:24	risc_dbg_rd_sel	rw	0x0	—
28:28	risc_dbg_control_ovr	rw	0x0	—
29:29	risc_dbg_en	rw	0x0	—

risc_dbg_sig_sel field

risc_dbg_daisy_sel field

risc_dbg_rd_sel field

risc_dbg_control_ovr field

risc_dbg_en field

3.2.13 TENSIX_CREG_READ register

- Base Offset: 0x2c
- Size: 0x4

Bits	Identifier	Access	Reset	Name
10:0	tensix_creg_read	rw	0x0	—

tensix_creg_read field

3.2.14 DBG_RD_DATA register

- Base Offset: 0x30
- Size: 0x4

Bits	Identifier	Access	Reset	Name
31:0	dbus_rd_data_d	rw	0x0	—

dbus_rd_data_d field

3.2.15 DBG_ARRAY_RD_EN register

- Base Offset: 0x34
- Size: 0x4

Bits	Identifier	Access	Reset	Name
0:0	dbg_array_rd_en	rw	0x0	—

dbg_array_rd_en field

3.2.16 DBG_ARRAY_RD_CMD register

- Base Offset: 0x38
- Size: 0x4

Bits	Identifier	Access	Reset	Name
15:0	addr	rw	0x0	—
19:16	id	rw	0x0	—

addr field

id field

3.2.17 DBG_FEATURE_DISABLE register

- Base Offset: 0x3c
- Size: 0x4

Bits	Identifier	Access	Reset	Name
15:0	dbg_feature_disable	rw	0x0	—

dbg_feature_disable field

3.2.18 DBG_ARRAY_RD_DATA register

- Base Offset: 0x40
- Size: 0x4

Bits	Identifier	Access	Reset	Name
31:0	rd_data	rw	0x0	—

rd_data field

3.2.19 CG_CTRL_HYST0 register

- Base Offset: 0x44
- Size: 0x4

Bits	Identifier	Access	Reset	Name
6:0	cg_hyst_regblocks	rw	0x0	—
14:8	cg_hyst_fpu	rw	0x0	—
22:16	cg_hyst_move	rw	0x0	—
30:24	cg_hyst_packer	rw	0x0	—

cg_hyst_regblocks field

cg_hyst_fpu field

cg_hyst_move field

cg_hyst_packer field

3.2.20 CG_CTRL_HYST1 register

- Base Offset: 0x48
- Size: 0x4

Bits	Identifier	Access	Reset	Name
6:0	cg_hyst_unpacker	rw	0x0	—
14:8	cg_hyst_xsearch	rw	0x0	—
22:16	cg_hyst_thcon	rw	0x0	—
30:24	cg_hyst_trisc	rw	0x0	—

cg_hyst_unpacker field

cg_hyst_xsearch field

cg_hyst_thcon field

cg_hyst_trisc field

3.2.21 TENSIX_CREG_RDDATA register

- Base Offset: 0x4c
- Size: 0x4

Bits	Identifier	Access	Reset	Name
31:0	tensix_creg_rddata	rw	0x0	—

tensix_creg_rddata field

3.2.22 CG_CTRL_HYST2 register

- Base Offset: 0x50
- Size: 0x4

Bits	Identifier	Access	Reset	Name
6:0	cg_hyst_retmux	rw	0x0	—
14:8	cg_hyst_ithread	rw	0x0	—
22:16	cg_hyst2_l1bank	rw	0x0	—
30:24	cg_hyst2_srcb	rw	0x0	—

cg_hyst_retmux field

cg_hyst_ithread field

cg_hyst2_l1bank field

cg_hyst2_srcb field

3.2.23 RISC_DBG_CNTL_0 register

- Base Offset: 0x54
- Size: 0x4

Bits	Identifier	Access	Reset	Name
31:0	risc_dbg_cntl_0	rw	0x0	—

risc_dbg_cntl_0 field

3.2.24 RISC_DBG_CNTL_1 register

- Base Offset: 0x58
- Size: 0x4

Bits	Identifier	Access	Reset	Name
31:0	risc_dbg_cntl_1	rw	0x0	—

risc_dbg_cntl_1 field

3.2.25 RISC_DBG_STATUS_0 register

- Base Offset: 0x5c
- Size: 0x4

Bits	Identifier	Access	Reset	Name
31:0	status	rw	0x0	—

status field

3.2.26 RISC_DBG_STATUS_1 register

- Base Offset: 0x60
- Size: 0x4

Bits	Identifier	Access	Reset	Name
31:0	status	rw	0x0	—

status field

3.2.27 TRISC_PC_BUF_OVERRIDE register

- Base Offset: 0x64
- Size: 0x4

Bits	Identifier	Access	Reset	Name
31:0	trisc_pc_buf_override	rw	0x0	—

trisc_pc_buf_override field

3.2.28 DBG_INVALID_INSTRN register

- Base Offset: 0x68
- Size: 0x4

Bits	Identifier	Access	Reset	Name
3:0	dbg_invalid_instrn	rw	0x0	—

dbg_invalid_instrn field

3.2.29 DBG_INSTRN_BUF_CTRL0 register

- Base Offset: 0x6c
- Size: 0x4

Bits	Identifier	Access	Reset	Name
3:0	override	rw	0x0	—
7:4	write	rw	0x0	—

override field

write field

3.2.30 DBG_INSTRN_BUF_CTRL1 register

- Base Offset: 0x70
- Size: 0x4

Bits	Identifier	Access	Reset	Name
31:0	dbg_instrn_buf_wrdata	rw	0x0	—

dbg_instrn_buf_wrdata field

3.2.31 DBG_INSTRN_BUF_STATUS register

- Base Offset: 0x74
- Size: 0x4

Bits	Identifier	Access	Reset	Name
31:0	dbg_instrn_buf_status	rw	0x0	—

dbg_instrn_buf_status field

3.2.32 STOCH_RND_MASK0 register

- Base Offset: 0x78
- Size: 0x4

Bits	Identifier	Access	Reset	Name
31:0	stoch_rnd_mask0	rw	0x0	—

stoch_rnd_mask0 field

3.2.33 STOCH_RND_MASK1 register

- Base Offset: 0x7c
- Size: 0x4

Bits	Identifier	Access	Reset	Name
31:0	stoch_rnd_mask1	rw	0x0	—

stoch_rnd_mask1 field

3.2.34 FPU_STICKY_BITS register

- Base Offset: 0x80
- Size: 0x4

Bits	Identifier	Access	Reset	Name
31:0	fpv_sticky_bits	rw	0x0	—

fpv_sticky_bits field

3.2.35 PERF_CNT_TDMA_PACK0 register

- Base Offset: 0x84
- Size: 0x4

Bits	Identifier	Access	Reset	Name
31:0	ref_period	rw	0x0	—

ref_period field

3.2.36 PERF_CNT_TDMA_PACK1 register

- Base Offset: 0x88
- Size: 0x4

Bits	Identifier	Access	Reset	Name
31:0	mode	rw	0x0	—

mode field

3.2.37 PERF_CNT_TDMA_PACK2 register

- Base Offset: 0x8c
- Size: 0x4

Bits	Identifier	Access	Reset	Name
0:0	start	rw	0x0	—
1:1	stop	rw	0x0	—

start field

stop field

3.2.38 PERF_CNT_OUT_L_INSTRN_THREAD register

- Base Offset: 0x90
- Size: 0x4

Bits	Identifier	Access	Reset	Name
31:0	perf_cnt_out_l_instrn_thread	rw	0x0	—

perf_cnt_out_l_instrn_thread field

3.2.39 PERF_CNT_OUT_H_INSTRN_THREAD register

- Base Offset: 0x94
- Size: 0x4

Bits	Identifier	Access	Reset	Name
31:0	perf_cnt_out_h_instrn_thread	rw	0x0	—

perf_cnt_out_h_instrn_thread field

3.2.40 PERF_CNT_OUT_L_TDMA_UNPACK register

- Base Offset: 0x98
- Size: 0x4

Bits	Identifier	Access	Reset	Name
31:0	perf_cnt_out_l_tdma_unpack	rw	0x0	—

perf_cnt_out_l_tdma_unpack field

3.2.41 PERF_CNT_OUT_H_TDMA_UNPACK register

- Base Offset: 0x9c
- Size: 0x4

Bits	Identifier	Access	Reset	Name
31:0	perf_cnt_out_h_tdma_unpack	rw	0x0	—

perf_cnt_out_h_tdma_unpack field

3.2.42 PERF_CNT_OUT_L_TDMA_PACK register

- Base Offset: 0xa0
- Size: 0x4

Bits	Identifier	Access	Reset	Name
31:0	perf_cnt_out_l_tdma_pack	rw	0x0	—

perf_cnt_out_l_tdma_pack field

3.2.43 PERF_CNT_OUT_H_TDMA_PACK register

- Base Offset: 0xa4
- Size: 0x4

Bits	Identifier	Access	Reset	Name
31:0	perf_cnt_out_h_tdma_pack	rw	0x0	—

perf_cnt_out_h_tdma_pack field

3.2.44 PERF_CNT_OUT_L_FPU register

- Base Offset: 0xa8
- Size: 0x4

Bits	Identifier	Access	Reset	Name
31:0	perf_cnt_out_l_fpu	rw	0x0	—

perf_cnt_out_l_fpu field

3.2.45 PERF_CNT_OUT_H_FPU register

- Base Offset: 0xac
- Size: 0x4

Bits	Identifier	Access	Reset	Name
31:0	perf_cnt_out_h_fpu	rw	0x0	—

perf_cnt_out_h_fpu field

3.2.46 SOFT_RESET_0 register

- Base Offset: 0xb0
- Size: 0x4

Bits	Identifier	Access	Reset	Name
2:0	tdma_unpacker_soft_reset	rw	0x0	—
5:3	tdma_packer_soft_reset	rw	0x0	—
6:6	tdma_mover_soft_reset	rw	0x0	—
7:7	tdma_search_soft_reset	rw	0x0	—
8:8	tdma_glue_soft_reset	rw	0x0	—
9:9	tdma_thcon_soft_reset	rw	0x0	—
10:10	fpu_soft_reset	rw	0x0	—
14:11	risc_control_soft_reset	rw	0x0	—
15:15	srca_reg_soft_reset	rw	0x0	—
16:16	srb_reg_soft_reset	rw	0x0	—
17:17	dest_reg_soft_reset	rw	0x0	—
22:19	srca_reg_soft_reset_fpu	rw	0x0	—
23:23	ttsync_soft_reset	rw	0x0	—
24:24	tt_cfg_exu_soft_reset	rw	0x0	—
25:25	tt_sync_exu_soft_reset	rw	0x0	—
26:26	tt_gpr_file_soft_reset	rw	0x0	—
27:27	tt_srcs_registers_soft_reset	rw	0x0	—
28:28	tt_dest_sync_prearb_soft_reset	rw	0x0	—
29:29	tt_power_ramp_fsm_soft_reset	rw	0x0	—
30:30	l1_soft_reset	rw	0x0	—

tdma_unpacker_soft_reset field

tdma_packer_soft_reset field

tdma_mover_soft_reset field

tdma_search_soft_reset field

tdma_glue_soft_reset field

tdma_thcon_soft_reset field

fpu_soft_reset field

risc_control_soft_reset field

srca_reg_soft_reset field

srcb_reg_soft_reset field

dest_reg_soft_reset field

srca_reg_soft_reset_fpu field

ttsync_soft_reset field

tt_cfg_exu_soft_reset field

tt_sync_exu_soft_reset field

tt_gpr_file_soft_reset field

tt_srcs_registers_soft_reset field

tt_dest_sync_prearb_soft_reset field

tt_power_ramp_fsm_soft_reset field

l1_soft_reset field

3.2.47 SOFT_RESET_1 register

- Base Offset: 0xb4
- Size: 0x4

Bits	Identifier	Access	Reset	Name
3:0	tt_replay_unit_soft_reset	rw	0x0	—
7:4	tt_mop_decoder_soft_reset	rw	0x0	—
8:8	pic_soft_reset	rw	0x0	—

tt_replay_unit_soft_reset field

tt_mop_decoder_soft_reset field

pic_soft_reset field

3.2.48 WATCHDOG_TIMER register

- Base Offset: 0xb8
- Size: 0x4

Bits	Identifier	Access	Reset	Name
31:0	wdt	rw	0x0	—

wdt field

3.2.49 WDT_CNTL register

- Base Offset: 0xbc
- Size: 0x4

Bits	Identifier	Access	Reset	Name
0:0	wdt_en	rw	0x0	—
1:1	wdt_expired_intr_en	rw	0x0	—
2:2	wdt_soft_intr	rw	0x0	—

wdt_en field

wdt_expired_intr_en field

wdt_soft_intr field

3.2.50 WDT_STATUS register

- Base Offset: 0xc0
- Size: 0x4

Bits	Identifier	Access	Reset	Name
0:0	wdt_expired	rw	0x0	—

wdt_expired field

3.2.51 WALL_CLOCK_0 register

- Base Offset: 0xc4
- Size: 0x4

Bits	Identifier	Access	Reset	Name
31:0	wall_clock0	rw	0x0	—

wall_clock0 field

3.2.52 WALL_CLOCK_1 register

- Base Offset: 0xc8
- Size: 0x4

Bits	Identifier	Access	Reset	Name
31:0	wall_clock1	rw	0x0	—

wall_clock1 field

3.2.53 WALL_CLOCK_1_AT register

- Base Offset: 0xcc
- Size: 0x4

Bits	Identifier	Access	Reset	Name
31:0	wall_clock_h_latched	rw	0x0	—

wall_clock_h_latched field

3.2.54 TIMESTAMP_DUMP_CMD register

- Base Offset: 0xd0
- Size: 0x4

Bits	Identifier	Access	Reset	Name
31:0	cmd	rw	0x0	—

cmd field

3.2.55 TIMESTAMP_DUMP_CNTL register

- Base Offset: 0xd4
- Size: 0x4

Bits	Identifier	Access	Reset	Name
31:0	cntl	rw	0x0	—

cntl field

3.2.56 TIMESTAMP_DUMP_STATUS register

- Base Offset: 0xd8
- Size: 0x4

Bits	Identifier	Access	Reset	Name
31:0	status	rw	0x0	—

status field

3.2.57 TIMESTAMP_DUMP_BUF0_START_ADDR register

- Base Offset: 0xdc
- Size: 0x4

Bits	Identifier	Access	Reset	Name
31:0	start_addr	rw	0x0	—

start_addr field

3.2.58 TIMESTAMP_DUMP_BUF0_END_ADDR register

- Base Offset: 0xe0
- Size: 0x4

Bits	Identifier	Access	Reset	Name
31:0	end_addr	rw	0x0	—

end_addr field

3.2.59 TIMESTAMP_DUMP_BUF1_START_ADDR register

- Base Offset: 0xe4
- Size: 0x4

Bits	Identifier	Access	Reset	Name
31:0	start_addr	rw	0x0	—

start_addr field

3.2.60 TIMESTAMP_DUMP_BUF1_END_ADDR register

- Base Offset: 0xe8
- Size: 0x4

Bits	Identifier	Access	Reset	Name
31:0	end_addr	rw	0x0	—

end_addr field

3.2.61 PERF_CNT_MUX_CTRL register

- Base Offset: 0xec
- Size: 0x4

Bits	Identifier	Access	Reset	Name
3:0	instrn_thread_perf_cnt_mux_sel	rw	0x0	—
6:4	l1_perf_cnt_mux_sel	rw	0x0	—

instrn_thread_perf_cnt_mux_sel field

l1_perf_cnt_mux_sel field

3.2.62 LFSR_HIT_MASK register

- Base Offset: 0xf0
- Size: 0x4

Bits	Identifier	Access	Reset	Name
15:0	lfsr_hit_mask	rw	0x0	—

lfsr_hit_mask field

3.2.63 DISABLE_RESET register

- Base Offset: 0xf4
- Size: 0x4

Bits	Identifier	Access	Reset	Name
11:0	disable_reset	rw	0x0	—

disable_reset field

3.2.64 TRISC0_RESET_PC register

- Base Offset: 0xf8
- Size: 0x4

Bits	Identifier	Access	Reset	Name
31:0	trisc0_reset_pc	rw	0x0	—

trisc0_reset_pc field

3.2.65 TRISC1_RESET_PC register

- Base Offset: 0xfc
- Size: 0x4

Bits	Identifier	Access	Reset	Name
31:0	trisc1_reset_pc	rw	0x0	—

trisc1_reset_pc field

3.2.66 TRISC2_RESET_PC register

- Base Offset: 0x100
- Size: 0x4

Bits	Identifier	Access	Reset	Name
31:0	trisc2_reset_pc	rw	0x0	—

trisc2_reset_pc field

3.2.67 TRISC3_RESET_PC register

- Base Offset: 0x104
- Size: 0x4

Bits	Identifier	Access	Reset	Name
31:0	trisc3_reset_pc	rw	0x0	—

trisc3_reset_pc field

3.2.68 TRISC_RESET_PC_OVERRIDE register

- Base Offset: 0x108
- Size: 0x4

Bits	Identifier	Access	Reset	Name
3:0	trisc_reset_pc_override	rw	0x0	—

trisc_reset_pc_override field

3.2.69 DEST_CG_CTRL register

- Base Offset: 0x10c
- Size: 0x4

Bits	Identifier	Access	Reset	Name
6:0	dest_cg_hyst	rw	0x0	—
31:8	dest_cg_en	rw	0x0	—

dest_cg_hyst field

dest_cg_en field

3.2.70 CG_CTRL_EN register

- Base Offset: 0x110
- Size: 0x4

Bits	Identifier	Access	Reset	Name
31:0	cg_ctrl_en	rw	0x0	—

cg_ctrl_en field

3.2.71 CG_KICK register

- Base Offset: 0x114
- Size: 0x4

Bits	Identifier	Access	Reset	Name
31:0	cg_kick	rw	0x0	—

cg_kick field

3.2.72 DBG_IBUFFER_TIMEOUT register

- Base Offset: 0x118
- Size: 0x4

Bits	Identifier	Access	Reset	Name
31:0	timeout	rw	0x0	—

timeout field

3.2.73 DBG_IBUFFER_CNT_EN register

- Base Offset: 0x11c
- Size: 0x4

Bits	Identifier	Access	Reset	Name
3:0	enable	rw	0x0	—
7:4	clr_ibuff	rw	0x0	—
8:8	release_ibuff0	rw	0x0	—
9:9	release_ibuff1	rw	0x0	—
10:10	release_ibuff2	rw	0x0	—
11:11	release_ibuff3	rw	0x0	—

enable field

clr_ibuff field

release_ibuff0 field

release_ibuff1 field

release_ibuff2 field

release_ibuff3 field

3.2.74 TRISC0_BAR register

- Base Offset: 0x120
- Size: 0x4
- Array: 4 elements (offsets 0x120, 0x124, 0x128, 0x12c)

Bits	Identifier	Access	Reset	Name
31:0	BAR	rw	0x0	—

BAR field

3.2.75 TRISC0_BAR_SELECT register

- Base Offset: 0x130
- Size: 0x4

Bits	Identifier	Access	Reset	Name
1:0	select	rw	0x0	—

select field

3.2.76 TRISC1_BAR register

- Base Offset: 0x134
- Size: 0x4
- Array: 4 elements (offsets 0x134, 0x138, 0x13c, 0x140)

Bits	Identifier	Access	Reset	Name
31:0	BAR	rw	0x0	—

BAR field

3.2.77 TRISC1_BAR_SELECT register

- Base Offset: 0x144
- Size: 0x4

Bits	Identifier	Access	Reset	Name
1:0	select	rw	0x0	—

select field

3.2.78 TRISC2_BAR register

- Base Offset: 0x148
- Size: 0x4
- Array: 4 elements (offsets 0x148, 0x14c, 0x150, 0x154)

Bits	Identifier	Access	Reset	Name
31:0	BAR	rw	0x0	—

BAR field

3.2.79 TRISC2_BAR_SELECT register

- Base Offset: 0x158
- Size: 0x4

Bits	Identifier	Access	Reset	Name
1:0	select	rw	0x0	—

select field

3.2.80 TRISC3_BAR register

- Base Offset: 0x15c
- Size: 0x4
- Array: 4 elements (offsets 0x15c, 0x160, 0x164, 0x168)

Bits	Identifier	Access	Reset	Name
31:0	BAR	rw	0x0	—

BAR field

3.2.81 TRISC3_BAR_SELECT register

- Base Offset: 0x16c
- Size: 0x4

Bits	Identifier	Access	Reset	Name
1:0	select	rw	0x0	—

select field

3.2.82 TRISC_MAILBOX register

- Base Offset: 0x170
- Size: 0x4
- Array: 16 elements (offsets 0x170 to 0x1ac)

Bits	Identifier	Access	Reset	Name
31:0	data	rw	0x0	—

data field

3.2.83 DBG_HALT_CLR register

- Base Offset: 0x1b0
- Size: 0x4

Bits	Identifier	Access	Reset	Name
0:0	halt_clr	rw	0x0	—

halt_clr field

3.2.84 DBG_MOV2D_ADDR register

- Base Offset: 0x1b4
- Size: 0x4

Bits	Identifier	Access	Reset	Name
11:0	addr	rw	0x0	—

addr field

3.2.85 DBG_MOV2D register

- Base Offset: 0x1b8
- Size: 0x4

Bits	Identifier	Access	Reset	Name
0:0	a2d	rw	0x0	—
1:1	b2d	rw	0x0	—

a2d field

b2d field

3.2.86 DBG_TEMP_REGS register

- Base Offset: 0x1bc
- Size: 0x4
- Array: 32 elements (offsets 0x1bc to 0x238)

Bits	Identifier	Access	Reset	Name
31:0	tmp_reg	rw	0x0	—

tmp_reg field

3.2.87 DBG_BACKDOOR_INST register

- Base Offset: 0x23c
- Size: 0x4

Bits	Identifier	Access	Reset	Name
31:0	inst	rw	0x0	—

inst field

3.2.88 DBG_BACKDOOR_INST_ISSUE register

- Base Offset: 0x240
- Size: 0x4

Bits	Identifier	Access	Reset	Name
0:0	trigger	rw	0x0	—

trigger field

3.2.89 DBG_TRISC_STACK_LIMIT register

- Base Offset: 0x244
- Size: 0x4

Bits	Identifier	Access	Reset	Name
31:0	stack_limit	rw	0x0	—

stack_limit field

3.2.90 RISCV_IC_INVALIDATE register

- Base Offset: 0x248
- Size: 0x4

Bits	Identifier	Access	Reset	Name
3:0	InvalidateAll	rw	0x0	—

InvalidateAll field

3.2.91 TRISC_END_PC register

- Base Offset: 0x24c
- Size: 0x4
- Array: 4 elements (offsets 0x24c, 0x250, 0x254, 0x258)

Bits	Identifier	Access	Reset	Name
31:0	PC	rw	0x0	—

PC field

3.2.92 RISC_PREFETCH_CTRL register

- Base Offset: 0x25c
- Size: 0x4

Bits	Identifier	Access	Reset	Name
3:0	Enable_Trisc	rw	0x0	—
11:4	Max_Req_Count	rw	0x0	—

Enable_Trisc field

Max_Req_Count field

3.2.93 RISC_BRANCH_PREDICTION_CTRL register

- Base Offset: 0x260
- Size: 0x4

Bits	Identifier	Access	Reset	Name
3:0	DISABLE_RISC_BP_Disable_trisc	rw	0x0	—
7:4	DISABLE_RISC_BP_Disable_bmp_clear_trisc	rw	0x0	—

DISABLE_RISC_BP_Disable_trisc field

DISABLE_RISC_BP_Disable_bmp_clear_trisc field

3.2.94 CHICKEN_BITS register

- Base Offset: 0x264
- Size: 0x4

Bits	Identifier	Access	Reset	Name
0:0	sfpu_scbd_disable	rw	0x0	—
3:1	UNUSED	rw	0x0	—
4:4	stallwait_waits_just_for_pack_dest_read	rw	0x0	—
8:5	replay_disable_banked_mode	rw	0x0	—
12:9	replay_disable_safety_stall	rw	0x0	—
13:13	swap_shadow_with_live_instead	rw	0x0	—
17:14	disable_automatic_shadow_reg_committing	rw	0x0	—
18:18	disable_forced_cfg_thcon_parallelization	rw	0x0	—
19:19	disable_power_ramp_per_column_clock_gating	rw	0x0	—
20:20	disable_packer_uses_saved_dest_format	rw	0x0	—
21:21	disable_srca_hw_dvalid_sync	rw	0x0	—
22:22	disable_srcb_hw_dvalid_sync	rw	0x0	—
23:23	disable_srcs_hw_dvalid_sync	rw	0x0	—

sfpu_scbd_disable field

UNUSED field

stallwait_waits_just_for_pack_dest_read field

replay_disable_banked_mode field

replay_disable_safety_stall field

swap_shadow_with_live_instead field

disable_automatic_shadow_reg_committing field

disable_forced_cfg_thcon_parallelization field

disable_power_ramp_per_column_clock_gating field

disable_packer_uses_saved_dest_format field

disable_srca_hw_dvalid_sync field

disable_srcb_hw_dvalid_sync field

disable_srcs_hw_dvalid_sync field

3.2.95 TENSIX_TRISC_SYNC register

- Base Offset: 0x268
- Size: 0x4
- Array: 4 elements (offsets 0x268, 0x26c, 0x270, 0x274)

Bits	Identifier	Access	Reset	Name
0:0	DisableTrackGlobalCfg	rw	0x0	—
2:2	DisableTrackGPR	rw	0x0	—
4:4	DisableTrackTensixInstructions	rw	0x0	—

DisableTrackGlobalCfg field

DisableTrackGPR field

DisableTrackTensixInstructions field

3.2.96 TENSIX_CSR_CONFIG register

- Base Offset: 0x278
- Size: 0x4
- Array: 4 elements (offsets 0x278, 0x27c, 0x280, 0x284)

Bits	Identifier	Access	Reset	Name
0:0	RawBusyStatus	rw	0x0	—

RawBusyStatus field

3.2.97 POWER_RAMP_PROFILE_CTRL register

- Base Offset: 0x288
- Size: 0x4

Bits	Identifier	Access	Reset	Name
7:0	rampup_delay	rw	0x0	—
15:8	rampup_rate	rw	0x0	—
23:16	rampdown_delay	rw	0x0	—
31:24	rampdown_rate	rw	0x0	—

rampup_delay field

rampup_rate field

rampdown_delay field

rampdown_rate field

3.2.98 POWER_RAMP_MISC_CTRL register

- Base Offset: 0x28c
- Size: 0x4

Bits	Identifier	Access	Reset	Name
7:0	max_gap_before_rampdown	rw	0x0	—

max_gap_before_rampdown field

3.2.99 ERR_MASK register

- Base Offset: 0x290
- Size: 0x4

Bits	Identifier	Access	Reset	Name
31:0	value	rw	0x0	—

value field

3.2.100 ERR_DATA register

- Base Offset: 0x294
- Size: 0x4

Bits	Identifier	Access	Reset	Name
31:0	value	rw	0x0	—

value field

3.2.101 FPU_SAFETY register

- Base Offset: 0x298
- Size: 0x4

Bits	Identifier	Access	Reset	Name
0:0	safety_enable	rw	0x0	—

safety_enable field

3.3 Tensix NOC Register Map Reference Guide

This document details the registers within the NOC block, which is included inside both Tensix and Dispatch engines, broken down by sub-module. The NOC block is responsible to issue and receive as well as route incoming and outgoing NOC packets.

Instantiated Modules and Base Addresses:

The base address of the NOC registers within the Tensix memory space is 0x0200_0000

- noc_niu: 0x0200_0000
- noc_config: 0x0200_0100
- noc_status: 0x0200_0200

3.3.1 Address Map: noc_niu

Description: NOC Master Control and Packet Header Programming Registers.

Base: 0x0200_0000

Register Instance Summary

Instance Name	Register Type	Relative Offset	Absolute Offset
TARG_ADDR_LO	TARG_ADDR_LO	0x000	0x000
TARG_ADDR_MID	TARG_ADDR_MID	0x004	0x004
TARG_ADDR_HI	TARG_ADDR_HI	0x008	0x008
RET_ADDR_LO	RET_ADDR_LO	0x00C	0x00C
RET_ADDR_MID	RET_ADDR_MID	0x010	0x010
RET_ADDR_HI	RET_ADDR_HI	0x014	0x014
CMD_LO	CMD_LO	0x018	0x018
CMD_HI	CMD_HI	0x01C	0x01C
BRCST_LO	BRCST_LO	0x020	0x020
BRCST_HI	BRCST_HI	0x024	0x024
AT_LEN	AT_LEN	0x028	0x028

continues on next page

Table 4 – continued from previous page

Instance Name	Register Type	Relative Offset	Absolute Offset
L1_ACC_AT_INSTRN	L1_ACC_AT_INSTRN	0x02C	0x02C
SEC_CTRL	SEC_CTRL	0x030	0x030
AT_DATA	AT_DATA	0x034	0x034
INLINE_DATA_LO	INLINE_DATA_LO	0x038	0x038
INLINE_DATA_HI	INLINE_DATA_HI	0x03C	0x03C
BYTE_ENABLE	BYTE_ENABLE	0x040	0x040
CMD_CTRL	CMD_CTRL	0x060	0x060
NODE_ID	NODE_ID	0x064	0x064
ENDPOINT_ID	ENDPOINT_ID	0x068	0x068
NUM_MEM_PARITY_ERR	NUM_MEM_PARITY_ERR	0x06C	0x06C
NUM_HEADER_1B_ERR	NUM_HEADER_1B_ERR	0x070	0x070
NUM_HEADER_2B_ERR	NUM_HEADER_2B_ERR	0x074	0x074
ECC_CTRL	ECC_CTRL	0x078	0x078
CLEAR_OUTSTANDING_REQ_CNT	CLEAR_OUTSTANDING_REQ_CNT	0x07C	0x07C
CMD_BUF_AVAIL	CMD_BUF_AVAIL	0x080	0x080
CMD_BUF_OVFL	CMD_BUF_OVFL	0x084	0x084
SENT_TARG_ADDR_LO	SENT_TARG_ADDR_LO	0x088	0x088
SENT_TARG_ADDR_MID	SENT_TARG_ADDR_MID	0x08C	0x08C
SENT_TARG_ADDR_HI	SENT_TARG_ADDR_HI	0x090	0x090
SENT_RET_ADDR_LO	SENT_RET_ADDR_LO	0x094	0x094
SENT_RET_ADDR_MID	SENT_RET_ADDR_MID	0x098	0x098
SENT_RET_ADDR_HI	SENT_RET_ADDR_HI	0x09C	0x09C
SENT_CMD_LO	SENT_CMD_LO	0x0A0	0x0A0
SENT_CMD_HI	SENT_CMD_HI	0x0A4	0x0A4
SENT_BRCST_LO	SENT_BRCST_LO	0x0A8	0x0A8
SENT_BRCST_HI	SENT_BRCST_HI	0x0AC	0x0AC
SENT_AT_LEN	SENT_AT_LEN	0x0B0	0x0B0
SENT_L1_ACC_AT_INSTRN	SENT_L1_ACC_AT_INSTRN	0x0B4	0x0B4
SENT_SEC_CTRL	SENT_SEC_CTRL	0x0B8	0x0B8
SENT_AT_DATA	SENT_AT_DATA	0x0BC	0x0BC
SENT_INLINE_DATA_LO	SENT_INLINE_DATA_LO	0x0C0	0x0C0
SENT_INLINE_DATA_HI	SENT_INLINE_DATA_HI	0x0C4	0x0C4
SENT_BYTE_ENABLE	SENT_BYTE_ENABLE	0x0C8	0x0C8

Register Definitions

(Registers related to Header Programming, Master Control, and Sent Header)

Register: TARG_ADDR_LO

Field Name	Bit Range	SW Access	Reset	Description
target_addr_lo	[31:0]	rw	0x0	-

Register: TARG_ADDR_MID

Field Name	Bit Range	SW Access	Reset	Description
target_addr_mid	[31:0]	rw	0x0	-

Register: TARG_ADDR_HI

Field Name	Bit Range	SW Access	Reset	Description
x_coord	[5:0]	rw	0x0	-
y_coord	[11:6]	rw	0x0	-
bc_start_x_coord	[17:12]	rw	0x0	-
bc_start_y_coord	[23:18]	rw	0x0	-

Register: RET_ADDR_LO

Field Name	Bit Range	SW Access	Reset	Description
ret_addr_lo	[31:0]	rw	0x0	-

Register: RET_ADDR_MID

Field Name	Bit Range	SW Access	Reset	Description
ret_addr_mid	[31:0]	rw	0x0	-

Register: RET_ADDR_HI

Field Name	Bit Range	SW Access	Reset	Description
x_coord	[5:0]	rw	0x0	-
y_coord	[11:6]	rw	0x0	-
bc_start_x_coord	[17:12]	rw	0x0	-
bc_start_y_coord	[23:18]	rw	0x0	-

Register: CMD_L0

Field Name	Bit Range	SW Access	Reset	Description
cmd_at_cpy_bit	[0:0]	rw	0x0	-
cmd_rw_bit	[1:1]	rw	0x0	-
cmd_wr_be_bit	[2:2]	rw	0x0	-
cmd_wr_inline_bit	[3:3]	rw	0x0	-
cmd_wr_inline_64_bit	[4:4]	rw	0x0	-
cmd_resp_marked_bit	[5:5]	rw	0x0	-
cmd_brcst_bit	[6:6]	rw	0x0	-
cmd_linked_bit	[7:7]	rw	0x0	-
cmd_path_reserve	[8:8]	rw	0x0	-
cmd_mem_rd_drop_ack	[9:9]	rw	0x0	-
cmd_dyna_routing_en	[10:10]	rw	0x0	-
cmd_ll_acc_at_en	[11:11]	rw	0x0	-
cmd_flush_bit	[12:12]	rw	0x0	-
cmd_snoop_bit	[13:13]	rw	0x0	-
cmd_static_vc	[19:14]	rw	0x0	-
resp_static_vc	[25:20]	rw	0x0	-
cmd_port_req_mask	[30:26]	rw	0x0	-

Register: CMD_HI

Field Name	Bit Range	SW Access	Reset	Description
cmd_pkt_tag_id	[3:0]	rw	0x0	-
cmd_force_dim_routing	[4:4]	rw	0x0	-

Register: BRCST_L0

Field Name	Bit Range	SW Access	Reset	Description
brcst_xy_bit	[0:0]	rw	0x0	-
brcst_src_include	[1:1]	rw	0x0	-
brcst_ctrl_state	[3:2]	rw	0x0	-
brcst_active_node	[4:4]	rw	0x0	-
brcst_end_node	[10:5]	rw	0x0	-
quad_excl_enable	[11:11]	rw	0x0	-
quad_excl_coord_x	[17:12]	rw	0x0	-
quad_excl_coord_y	[23:18]	rw	0x0	-
quad_excl_dir_x	[24:24]	rw	0x0	-
quad_excl_dir_y	[25:25]	rw	0x0	-
strided_keep_x	[27:26]	rw	0x0	-
strided_skip_x	[29:28]	rw	0x0	-
strided_keep_y	[31:30]	rw	0x0	-

Register: BRCST_HI

Field Name	Bit Range	SW Access	Reset	Description
strided_skip_y	[1:0]	rw	0x0	-

Register: AT_LEN

Field Name	Bit Range	SW Access	Reset	Description
at_ind_32	[1:0]	rw	0x0	-
at_wrap	[6:2]	rw	0x0	-
rsvd_7_bits	[9:7]	rw	0x0	-
at_ind_32_src	[10:10]	rw	0x0	-
rsvd_1_bit	[11:11]	rw	0x0	-
at_instrn	[15:12]	rw	0x0	Atomic Instruction (see RDL for encoding)
rsvd_16_bits	[31:16]	rw	0x0	-

Register: L1_ACC_AT_INSTRN

Field Name	Bit Range	SW Access	Reset	Description
data_format	[2:0]	rw	0x0	-
disable_sat	[3:3]	rw	0x0	-
at_instrn	[15:12]	rw	0x0	-

Register: SEC_CTRL

Field Name	Bit Range	SW Access	Reset	Description
group_id	[7:4]	rw	0x0	-
sec_ctrl_level	[3:0]	rw	0x0	-

Register: AT_DATA

Field Name	Bit Range	SW Access	Reset	Description
atomic_data	[31:0]	rw	0x0	-

Register: INLINE_DATA_LO

Field Name	Bit Range	SW Access	Reset	Description
inline_data_lo	[31:0]	rw	0x0	-

Register: INLINE_DATA_HI

Field Name	Bit Range	SW Access	Reset	Description
inline_data_hi	[31:0]	rw	0x0	-

Register: BYTE_ENABLE

Field Name	Bit Range	SW Access	Reset	Description
byte_enable	[31:0]	rw	0x0	-

Register: CMD_CTRL

Field Name	Bit Range	SW Access	Reset	Description
cmd_pending	[0:0]	r	0x0	-

Register: NODE_ID

Field Name	Bit Range	SW Access	Reset	Description
local_nodeid_x	[7:0]	r	-	-
local_nodeid_y	[15:8]	r	-	-
noc_x_size	[23:16]	r	-	-
noc_y_size	[31:24]	r	-	-

Register: ENDPOINT_ID

Field Name	Bit Range	SW Access	Reset	Description
endpoint_id	[31:0]	r	-	-

Register: NUM_MEM_PARITY_ERR

Field Name	Bit Range	SW Access	Reset	Description
num_mem_parity_err	[15:0]	r	0x0	-

Register: NUM_HEADER_1B_ERR

Field Name	Bit Range	SW Access	Reset	Description
num_header_1b_err	[15:0]	r	0x0	-

Register: NUM_HEADER_2B_ERR

Field Name	Bit Range	SW Access	Reset	Description
num_header_2b_err	[15:0]	r	0x0	-

Register: ECC_CTRL

Field Name	Bit Range	SW Access	Reset	Description
mem_parity_err_clear	[0:0]	w	0x0	-
header_1b_err_clear	[1:1]	w	0x0	-
header_2b_err_clear	[2:2]	w	0x0	-
mem_parity_err_force	[3:3]	w	0x0	-
header_1b_err_force	[4:4]	w	0x0	-
header_2b_err_force	[5:5]	w	0x0	-

Register: CLEAR_OUTSTANDING_REQ_CNT

Field Name	Bit Range	SW Access	Reset	Description
clear_counter_0	[0:0]	w	0x0	-
clear_counter_1	[1:1]	w	0x0	-
clear_counter_2	[2:2]	w	0x0	-
clear_counter_3	[3:3]	w	0x0	-
clear_counter_4	[4:4]	w	0x0	-
clear_counter_5	[5:5]	w	0x0	-
clear_counter_6	[6:6]	w	0x0	-
clear_counter_7	[7:7]	w	0x0	-
clear_counter_8	[8:8]	w	0x0	-
clear_counter_9	[9:9]	w	0x0	-
clear_counter_10	[10:10]	w	0x0	-
clear_counter_11	[11:11]	w	0x0	-
clear_counter_12	[12:12]	w	0x0	-
clear_counter_13	[13:13]	w	0x0	-
clear_counter_14	[14:14]	w	0x0	-
clear_counter_15	[15:15]	w	0x0	-

Register: CMD_BUF_AVAIL

Field Name	Bit Range	SW Access	Reset	Description
cmd_buf_avail	[31:0]	r	-	-

Register: CMD_BUF_OVFL

Field Name	Bit Range	SW Access	Reset	Description
cmd_buf_ovfl	[31:0]	r	-	-

Register: SENT_TARG_ADDR_LO

Field Name	Bit Range	SW Access	Reset	Description
target_addr_lo	[31:0]	r	0x0	-

Register: SENT_TARG_ADDR_MID

Field Name	Bit Range	SW Access	Reset	Description
target_addr_mid	[31:0]	r	0x0	-

Register: SENT_TARG_ADDR_HI

Field Name	Bit Range	SW Access	Reset	Description
x_coord	[7:0]	r	0x0	-
y_coord	[15:8]	r	0x0	-
bc_start_x_coord	[23:16]	r	0x0	-
bc_start_y_coord	[31:24]	r	0x0	-

Register: SENT_RET_ADDR_LO

Field Name	Bit Range	SW Access	Reset	Description
ret_addr_lo	[31:0]	r	0x0	-

Register: SENT_RET_ADDR_MID

Field Name	Bit Range	SW Access	Reset	Description
ret_addr_mid	[31:0]	r	0x0	-

Register: SENT_RET_ADDR_HI

Field Name	Bit Range	SW Access	Reset	Description
x_coord	[7:0]	r	0x0	-
y_coord	[15:8]	r	0x0	-

Register: SENT_CMD_LO

Field Name	Bit Range	SW Access	Reset	Description
cmd_at_cpy_bit	[0:0]	r	0x0	-
cmd_rw_bit	[1:1]	r	0x0	-
cmd_wr_be_bit	[2:2]	r	0x0	-
cmd_wr_inline_bit	[3:3]	r	0x0	-
cmd_wr_inline_64_bit	[4:4]	r	0x0	-
cmd_resp_marked_bit	[5:5]	r	0x0	-
cmd_brcst_bit	[6:6]	r	0x0	-
cmd_linked_bit	[7:7]	r	0x0	-
cmd_path_reserve	[8:8]	r	0x0	-
cmd_mem_rd_drop_ack	[9:9]	r	0x0	-
cmd_dyna_routing_en	[10:10]	r	0x0	-
cmd_ll_acc_at_en	[11:11]	r	0x0	-
cmd_flush_bit	[12:12]	r	0x0	-
cmd_snoop_bit	[13:13]	r	0x0	-
cmd_static_vc	[19:14]	r	0x0	-
resp_static_vc	[25:20]	r	0x0	-
cmd_port_req_mask	[30:26]	r	0x0	-

Register: SENT_CMD_HI

Field Name	Bit Range	SW Access	Reset	Description
cmd_pkt_tag_id	[3:0]	r	0x0	-

Register: SENT_BRCST_LO

Field Name	Bit Range	SW Access	Reset	Description
brcst_xy_bit	[0:0]	r	0x0	-
brcst_src_include	[1:1]	r	0x0	-
brcst_ctrl_state	[3:2]	r	0x0	-
brcst_active_node	[4:4]	r	0x0	-
brcst_end_node	[12:5]	r	0x0	-
quad_excl_enable	[13:13]	r	0x0	-
quad_excl_coord_x	[21:14]	r	0x0	-
quad_excl_coord_y	[29:22]	r	0x0	-
quad_excl_dir_x	[30:30]	r	0x0	-
quad_excl_dir_y	[31:31]	r	0x0	-

Register: SENT_BRCST_HI

Field Name	Bit Range	SW Access	Reset	Description
strided_keep_x	[1:0]	r	0x0	-
strided_skip_x	[3:2]	r	0x0	-
strided_keep_y	[5:4]	r	0x0	-
strided_skip_y	[7:6]	r	0x0	-

Register: SENT_AT_LEN

Field Name	Bit Range	SW Access	Reset	Description
at_len	[31:0]	r	0x0	-

Register: SENT_L1_ACC_AT_INSTRN

Field Name	Bit Range	SW Access	Reset	Description
data_format	[2:0]	r	0x0	-
disable_sat	[3:3]	r	0x0	-
at_instrn	[15:12]	r	0x0	-

Register: SENT_SEC_CTRL

Field Name	Bit Range	SW Access	Reset	Description
group_id	[7:4]	r	0x0	-
sec_ctrl_level	[3:0]	r	0x0	-

Register: SENT_AT_DATA

Field Name	Bit Range	SW Access	Reset	Description
atomic_data	[31:0]	r	0x0	-

Register: SENT_INLINE_DATA_LO

Field Name	Bit Range	SW Access	Reset	Description
inline_data_lo	[31:0]	r	0x0	-

Register: SENT_INLINE_DATA_HI

Field Name	Bit Range	SW Access	Reset	Description
inline_data_hi	[31:0]	r	0x0	-

Register: SENT_BYTE_ENABLE

Field Name	Bit Range	SW Access	Reset	Description
byte_enable	[31:0]	r	0x0	-

3.3.2 Address Map: noc_config

Description: NOC NIU and Router Configuration Registers.

Base: 0x0200_0100

Register Instance Summary

Instance Name	Register Type	Relative Offset	Absolute
NIU_CONFIG	NIU_CONFIG	0x000	0x100
ROUTER_CONFIG	ROUTER_CONFIG	0x004	0x104
BROADCAST_ROW_DISABLE	BROADCAST_DISABLE	0x008	0x108
BROADCAST_COL_DISABLE	BROADCAST_DISABLE	0x010	0x110
MEM_SHUTDOWN_CONTROL	MEM_SHUTDOWN_CONTROL	0x018	0x118
DEBUG_COUNTER_RESET	DEBUG_COUNTER_RESET	0x01C	0x11C
MST_IF_ID_TRANSACTION_COUNT_RTZ_CFG	MST_IF_ID_TRANSACTION_COUNT_RTZ_CFG	0x020	0x120
MST_IF_ID_TRANSACTION_COUNT_RTZ_CLR	MST_IF_ID_TRANSACTION_COUNT_RTZ_CLR	0x024	0x124
VC_DIM_ORDER	VC_DIM_ORDER	0x028	0x128
THROTTLER_CYCLES_PER_WINDOW	THROTTLER_CYCLES_PER_WINDOW	0x02C	0x12C
THROTTLER_HANDSHAKES_PER_WINDOW_NIU	THROTTLER_HANDSHAKES_PER_WINDOW	0x030	0x130
THROTTLER_HANDSHAKES_PER_WINDOW_NORTH	THROTTLER_HANDSHAKES_PER_WINDOW	0x034	0x134
THROTTLER_HANDSHAKES_PER_WINDOW_EAST	THROTTLER_HANDSHAKES_PER_WINDOW	0x038	0x138
THROTTLER_HANDSHAKES_PER_WINDOW_SOUTH	THROTTLER_HANDSHAKES_PER_WINDOW	0x03C	0x13C
THROTTLER_HANDSHAKES_PER_WINDOW_WEST	THROTTLER_HANDSHAKES_PER_WINDOW	0x040	0x140
NIU_TIMEOUT_DETECTED	NIU_TIMEOUT_DETECTED	0x044	0x144
NIU_TIMEOUT_VALUE	NIU_TIMEOUT_VALUE	0x048	0x148
INVALID_FENCE_START_ADDR_LO_0	INVALID_FENCE_START_ADDR	0x050	0x150
INVALID_FENCE_START_ADDR_HI_0	INVALID_FENCE_START_ADDR	0x054	0x154
INVALID_FENCE_END_ADDR_LO_0	INVALID_FENCE_END_ADDR	0x058	0x158
INVALID_FENCE_END_ADDR_HI_0	INVALID_FENCE_END_ADDR	0x05C	0x15C
INVALID_FENCE_START_ADDR_LO_1	INVALID_FENCE_START_ADDR	0x060	0x160
INVALID_FENCE_START_ADDR_HI_1	INVALID_FENCE_START_ADDR	0x064	0x164
INVALID_FENCE_END_ADDR_LO_1	INVALID_FENCE_END_ADDR	0x068	0x168
INVALID_FENCE_END_ADDR_HI_1	INVALID_FENCE_END_ADDR	0x06C	0x16C
INVALID_FENCE_START_ADDR_LO_2	INVALID_FENCE_START_ADDR	0x070	0x170
INVALID_FENCE_START_ADDR_HI_2	INVALID_FENCE_START_ADDR	0x074	0x174
INVALID_FENCE_END_ADDR_LO_2	INVALID_FENCE_END_ADDR	0x078	0x178
INVALID_FENCE_END_ADDR_HI_2	INVALID_FENCE_END_ADDR	0x07C	0x17C
INVALID_FENCE_START_ADDR_LO_3	INVALID_FENCE_START_ADDR	0x080	0x180
INVALID_FENCE_START_ADDR_HI_3	INVALID_FENCE_START_ADDR	0x084	0x184
INVALID_FENCE_END_ADDR_LO_3	INVALID_FENCE_END_ADDR	0x088	0x188
INVALID_FENCE_END_ADDR_HI_3	INVALID_FENCE_END_ADDR	0x08C	0x18C
INVALID_FENCE_START_ADDR_LO_4	INVALID_FENCE_START_ADDR	0x090	0x190
INVALID_FENCE_START_ADDR_HI_4	INVALID_FENCE_START_ADDR	0x094	0x194
INVALID_FENCE_END_ADDR_LO_4	INVALID_FENCE_END_ADDR	0x098	0x198
INVALID_FENCE_END_ADDR_HI_4	INVALID_FENCE_END_ADDR	0x09C	0x19C
INVALID_FENCE_START_ADDR_LO_5	INVALID_FENCE_START_ADDR	0x0A0	0x1A0
INVALID_FENCE_START_ADDR_HI_5	INVALID_FENCE_START_ADDR	0x0A4	0x1A4
INVALID_FENCE_END_ADDR_LO_5	INVALID_FENCE_END_ADDR	0x0A8	0x1A8
INVALID_FENCE_END_ADDR_HI_5	INVALID_FENCE_END_ADDR	0x0AC	0x1AC
VC_THROTTLER_HANDSHAKES_PER_WINDOW	VC_THROTTLER_HANDSHAKES_PER_WINDOW	0x0B0	0x1B0
VC_THROTTLER_CYCLES_PER_WINDOW	VC_THROTTLER_CYCLES_PER_WINDOW	0x0C0	0x1C0
FLEX_PORT_ADDR_HASH_EN	FLEX_PORT_ADDR_HASH_EN	0x0C4	0x1C4
FLEX_PORT_ADDR_HASH_CFG_4	FLEX_PORT_ADDR_HASH_CFG_4	0x0C8	0x1C8
FLEX_PORT_ADDR_HASH_CFG_5	FLEX_PORT_ADDR_HASH_CFG_5	0x0CC	0x1CC
FLEX_PORT_ADDR_HASH_CFG_6	FLEX_PORT_ADDR_HASH_CFG_6	0x0D0	0x1D0
FLEX_PORT_ADDR_HASH_CFG_7	FLEX_PORT_ADDR_HASH_CFG_7	0x0D4	0x1D4
FLEX_PORT_ADDR_HASH_CFG_8	FLEX_PORT_ADDR_HASH_CFG_8	0x0D8	0x1D8
FLEX_PORT_ADDR_HASH_CFG_9	FLEX_PORT_ADDR_HASH_CFG_9	0x0DC	0x1DC
FLEX_PORT_ADDR_HASH_CFG_10	FLEX_PORT_ADDR_HASH_CFG_10	0x0E0	0x1E0
FLEX_PORT_ADDR_HASH_CFG_11	FLEX_PORT_ADDR_HASH_CFG_11	0x0E4	0x1E4
FLEX_PORT_RD_PORT_CFG	FLEX_PORT_RD_PORT_CFG	0x0E8	0x1E8

continues on next page

Table 5 – continued from previous page

Instance Name	Register Type	Relative Offset	Absolute
FLEX_PORT_RD_PORT_IN_ORDER_ADDR_HI	FLEX_PORT_RD_PORT_IN_ORDER_ADDR_HI	0x0EC	0x1EC
FLEX_PORT_RD_PORT_IN_ORDER_ADDR_LOW	FLEX_PORT_RD_PORT_IN_ORDER_ADDR_LOW	0x0F0	0x1F0
FLEX_PORT_WR_PORT_CFG	FLEX_PORT_WR_PORT_CFG	0x0F4	0x1F4
FLEX_PORT_WR_PORT_IN_ORDER_ADDR_HI	FLEX_PORT_WR_PORT_IN_ORDER_ADDR_HI	0x0F8	0x1F8
FLEX_PORT_WR_PORT_IN_ORDER_ADDR_LOW	FLEX_PORT_WR_PORT_IN_ORDER_ADDR_LOW	0x0FC	0x1FC

Register Definitions

Register: NIU_CONFIG

Field Name	Bit Range	SW Access	Re-set	Description
clk_gating_en	[0:0]	rw	0x0	Enables clock gating in NIU.
clk_gating_hy	[7:1]	rw	0x2	Hysteresis value for NIU clock gaters.
mem_parity_en	[8:8]	rw	0x0	-
ecc_interrupt	[11:9]	rw	0x0	-
tile_clk_disa	[12:1]	rw	0x0	-
noc2axi_doubl	[13:1]	rw	0x0	Enables the header double store feature in NOC2AXI NIU.
niu_timeout_i	[14:1]	rw	0x0	Enables the NIU timeout interrupt
axi_enable	[15:1]	rw	0x0	Enables AXI ports, disable L1 memory ports. Set to 1 for NOC2AXI NIU and 0 for Tensix NOC NIU. Note that any change is delayed by approx 12 cycles.
cmd_buf_enabl	[16:1]	rw	0x0	Enables usage of command buffer FIFO in NIU. Set to 0 for passthrough.
autoinline_di	[17:1]	rw	0x0	Chicken bit. Disables autoinlining of less than 512b transfers.
always_flush	[18:1]	rw	0x0	Chicken bit. On NIU write, always wait for previous op to finish before sending next one.

Register: ROUTER_CONFIG

Field Name	Bit Range	SW Access	Ac-	Re-set	Description
clk_gating_enable	[0:0]	rw		0x0	-
clk_gating_hyst_val	[7:1]	rw		0x2	-
max_backoff_exp	[11:8]	rw		0x8	-
log2_basic_timeout_delay	[15:12]	rw		0x5	Log base-2 of basic timeout delay.
mem_parity_enable	[16:16]	rw		0x0	-
header_chk_bits_enable	[17:17]	rw		0x0	-
header_secdded_correct_enable	[18:18]	rw		0x0	-

Register: BROADCAST_DISABLE

Field Name	Bit Range	SW Access	Reset	Description
broadcast_disable	[31:0]	rw	0x0	-

Register: MEM_SHUTDOWN_CONTROL

Field Name	Bit Range	SW Access	Reset	Description
tsmc_sd	[0:0]	rw	0x0	Shut down mode.
tsmc_dslp	[1:1]	rw	0x0	Deep sleep mode.
tsmc_dslplv	[2:2]	rw	0x0	Deep sleep retention diode mode.

Register: DEBUG_COUNTER_RESET

Description: Programmable reset to various debug counters.

Field Name	Bit Range	SW Access	Reset	Description
router_outgoing_flit_clear	[0:0]	w	0x0	Write 1 to reset the Router Outgoing Flit Counters. This field is self-cleared.
cmd_buf_ovfl_clear	[4:4]	w	0x0	Write 1 to clear the Command Buffer FIFO Overflow flag. This field is self-cleared.

Register: MST_IF_ID_TRANSACTION_COUNT_RTZ_CFG

Description: Config register to control the MST_IF TAG_ID Transaction Counter Return-to-Zero Interrupt.

Field Name	Bit Range	SW Access	Reset	Description
interrupt_clear_on_read	[15:0]	rw	0x0	If set, when the TAG_ID Transaction Counter goes from non-zero to zero, an interrupt for the corresponding TAG_ID will be asserted. If not set (default), no interrupt will be generated.
clear_on_read	[28:28]	rw	0x0	If set, Clear-On-Read is disabled for MST_IF_INTP_TRANS_COUNT_RTZ_SOURCE.BIT_MAP. If not set (default), reading clears the corresponding bit.

Register: MST_IF_ID_TRANSACTION_COUNT_RTZ_CLR

Description: Config register to clear the detection of MST_IF TAG_ID Transaction Counter Return-to-Zero event. If set, the corresponding bit in MST_IF_INTP_TRANS_COUNT_RTZ_SOURCE.BIT_MAP will get cleared. Write to this register address will trigger a pulse to clear. Reading this register will always return 0.

Field Name	Bit Range	SW Access	Reset	Description
bit_0_clear	[0:0]	w	0x0	-
bit_1_clear	[1:1]	w	0x0	-
bit_2_clear	[2:2]	w	0x0	-
bit_3_clear	[3:3]	w	0x0	-
bit_4_clear	[4:4]	w	0x0	-
bit_5_clear	[5:5]	w	0x0	-
bit_6_clear	[6:6]	w	0x0	-
bit_7_clear	[7:7]	w	0x0	-
bit_8_clear	[8:8]	w	0x0	-
bit_9_clear	[9:9]	w	0x0	-
bit_10_clear	[10:10]	w	0x0	-
bit_11_clear	[11:11]	w	0x0	-
bit_12_clear	[12:12]	w	0x0	-
bit_13_clear	[13:13]	w	0x0	-
bit_14_clear	[14:14]	w	0x0	-
bit_15_clear	[15:15]	w	0x0	-

Register: VC_DIM_ORDER

Description: A bit map that specifies whether a particular VC will route x then y, or y then x. Each bit corresponds to a particular VC.

Field Name	Bit Range	SW Access	Ac-	Reset	Description
data	[31:0]	rw		0xAAAAAAAA	Each bit corresponds to a VC, 0 - route x then y. 1 - route y then x.

Register: UNUSED

Description: A field that is unused and can be replaced by a future register

Field Name	Bit Range	SW Access	Ac-	Reset	Description
data	[31:0]	rw		0x0	A field that is unused and can be replaced by a future register

Register: THROTTLER_CYCLES_PER_WINDOW**Description:** The number of cycles within a throttler window

Field Name	Bit Range	SW Access	Reset	Description
data	[31:0]	rw	0x0	Example BW settings: 50% -> CYCLES=2, HANDSHAKES=1; 25% -> CYCLES=4, HANDSHAKES=1; 75% -> CYCLES=4, HANDSHAKES=3. To disable set CYCLES_PER_WINDOW to 0.

Register: THROTTLER_HANDSHAKES_PER_WINDOW**Description:** The number of handshakes within a throttler window

Field Name	Bit Range	SW Access	Reset	Description
data	[31:0]	rw	0x0	Example BW settings: 50% -> CYCLES=2, HANDSHAKES=1; 25% -> CYCLES=4, HANDSHAKES=1; 75% -> CYCLES=4, HANDSHAKES=3. To disable set CYCLES_PER_WINDOW to 0.

Register: NIU_TIMEOUT_VALUE**Description:** The amount of noc clk cycles that must pass, without getting a response back from a nonposted transaction, to trigger a timeout.

Field Name	Bit Range	SW Access	Reset	Description
data	[31:0]	rw	0x0	If 0 this feature is disabled.

Register: NIU_TIMEOUT_DETECTED**Description:** Whether a timeout has been detected

Field Name	Bit Range	SW Access	Reset	Description
data	[0:0]	rw	0x0	Write 1 to clear timeout.

Register: INVALID_FENCE_START_ADDR

Field Name	Bit Range	SW Access	Reset	Description
start_addr	[31:0]	rw	0xFFFFFFFF	Any address that falls within (i.e. start_addr <= addr <= end_addr) is rejected and logged.

Register: INVALID_FENCE_END_ADDR

Field Name	Bit Range	SW Access	Reset	Description
end_addr	[31:0]	rw	0x0	Any address that falls within (i.e. start_addr <= addr <= end_addr) is rejected and logged.

Register: VC_THROTTLER_HANDSHAKES_PER_WINDOW

Description: The number of handshakes within a throttler window per VC for niu output flits. Example BW settings: 50% -> CYCLES=10, HANDSHAKES=5; 25% -> CYCLES=16, HANDSHAKES=4; 75% -> CYCLES=16, HANDSHAKES=12. To disable set VC_THROTTLER_CYCLES_PER_WINDOW to 0.

Field Name	Bit Range	SW Access	Reset	Description
vc0	[7:0]	rw	0x0	Handshakes for VC0
vc1	[15:8]	rw	0x0	Handshakes for VC1
vc2	[23:16]	rw	0x0	Handshakes for VC2
vc3	[31:24]	rw	0x0	Handshakes for VC3

Register: VC_THROTTLER_CYCLES_PER_WINDOW

Description: The number of cycles within a throttler window for all VCs for niu output flits

Field Name	Bit Range	SW Access	Reset	Description
data	[7:0]	rw	0x0	Example BW settings: 50% -> CYCLES=10, HANDSHAKES=5; 25% -> CYCLES=16, HANDSHAKES=4; 75% -> CYCLES=16, HANDSHAKES=12. To disable set CYCLES_PER_WINDOW to 0.

Register: FLEX_PORT_ADDR_HASH_EN

Description: Enable address hashing on L1 Flex ports used by NoC

Field Name	Bit Range	SW Access	Reset	Description
data	[0:0]	rw	0x0	NoC-wide enable for address hashing

Register: FLEX_PORT_ADDR_HASH_CFG_4

Description: Hash addr cfg 4.

Field Name	Bit Range	SW Access	Reset	Description
data	[17:0]	rw	0x1	Hash addr cfg 4.

Register: FLEX_PORT_ADDR_HASH_CFG_5**Description:** Hash addr cfg 5.

Field Name	Bit Range	SW Access	Reset	Description
data	[17:0]	rw	0x2	Hash addr cfg 5.

Register: FLEX_PORT_ADDR_HASH_CFG_6**Description:** Hash addr cfg 6.

Field Name	Bit Range	SW Access	Reset	Description
data	[17:0]	rw	0x4	Hash addr cfg 6.

Register: FLEX_PORT_ADDR_HASH_CFG_7**Description:** Hash addr cfg 7.

Field Name	Bit Range	SW Access	Reset	Description
data	[17:0]	rw	0x8	Hash addr cfg 7.

Register: FLEX_PORT_ADDR_HASH_CFG_8**Description:** Hash addr cfg 8.

Field Name	Bit Range	SW Access	Reset	Description
data	[17:0]	rw	0x10	Hash addr cfg 8.

Register: FLEX_PORT_ADDR_HASH_CFG_9**Description:** Hash addr cfg 9.

Field Name	Bit Range	SW Access	Reset	Description
data	[17:0]	rw	0x20	Hash addr cfg 9.

Register: FLEX_PORT_ADDR_HASH_CFG_10**Description:** Hash addr cfg 10.

Field Name	Bit Range	SW Access	Reset	Description
data	[17:0]	rw	0x40	Hash addr cfg 10.

Register: FLEX_PORT_ADDR_HASH_CFG_11**Description:** Hash addr cfg 11.

Field Name	Bit Range	SW Access	Reset	Description
data	[17:0]	rw	0x80	Hash addr cfg 11.

Register: FLEX_PORT_RD_PORT_CFG**Description:** Configuration of L1 read Flex ports for NoC

Field Name	Bit Range	SW Access	Reset	Description
write_in_order	[0:0]	rw	0x0	-
rw_barrier_order	[1:1]	rw	0x0	-
all_in_order	[2:2]	rw	0x0	-

Register: FLEX_PORT_RD_PORT_IN_ORDER_ADDR_HI**Description:** Flex port group read in order address high

Field Name	Bit Range	SW Access	Reset	Description
data	[17:0]	rw	0x0	address high 21:4

Register: FLEX_PORT_RD_PORT_IN_ORDER_ADDR_LOW**Description:** Flex port group read in order address low

Field Name	Bit Range	SW Access	Reset	Description
data	[17:0]	rw	0x0	address low 21:4

Register: FLEX_PORT_WR_PORT_CFG**Description:** Configuration of L1 write Flex ports for NoC

Field Name	Bit Range	SW Access	Reset	Description
write_in_order	[0:0]	rw	0x0	-
rw_barrier_order	[1:1]	rw	0x0	-
all_in_order	[2:2]	rw	0x0	-

Register: FLEX_PORT_WR_PORT_IN_ORDER_ADDR_HI**Description:** Flex port group write in order address high

Field Name	Bit Range	SW Access	Reset	Description
data	[17:0]	rw	0x0	address high 21:4

Register: FLEX_PORT_WR_PORT_IN_ORDER_ADDR_LOW**Description:** Flex port group write in order address low

Field Name	Bit Range	SW Access	Reset	Description
data	[17:0]	rw	0x0	address low 21:4

3.3.3 Address Map: noc_status**Description:** NOC NIU and Router Status Registers.**Base Offset:** 0x0200_0200**Register Instance Summary**

Instance Name	Register Type	Relative Offset
NIU_MST_ATOMIC_RESP_RECEIVED	NIU_MST_ATOMIC_RESP_RECEIVED	0x000
NIU_MST_WR_ACK_RECEIVED	NIU_MST_WR_ACK_RECEIVED	0x004
NIU_MST_RD_RESP_RECEIVED	NIU_MST_RD_RESP_RECEIVED	0x008
NIU_MST_RD_DATA_WORD_RECEIVED	NIU_MST_RD_DATA_WORD_RECEIVED	0x00C
NIU_MST_CMD_ACCEPTED	NIU_MST_CMD_ACCEPTED	0x010
NIU_MST_RD_REQ_SENT	NIU_MST_RD_REQ_SENT	0x014
NIU_MST_NON_POSTED_ATOMIC_SENT	NIU_MST_NON_POSTED_ATOMIC_SENT	0x018
NIU_MST_POSTED_ATOMIC_SENT	NIU_MST_POSTED_ATOMIC_SENT	0x01C
NIU_MST_NON_POSTED_WR_DATA_WORD_SENT	NIU_MST_NON_POSTED_WR_DATA_WORD_SENT	0x020
NIU_MST_POSTED_WR_DATA_WORD_SENT	NIU_MST_POSTED_WR_DATA_WORD_SENT	0x024
NIU_MST_NON_POSTED_WR_REQ_SENT	NIU_MST_NON_POSTED_WR_REQ_SENT	0x028
NIU_MST_POSTED_WR_REQ_SENT	NIU_MST_POSTED_WR_REQ_SENT	0x02C
NIU_MST_NON_POSTED_WR_REQ_STARTED	NIU_MST_NON_POSTED_WR_REQ_STARTED	0x030
NIU_MST_POSTED_WR_REQ_STARTED	NIU_MST_POSTED_WR_REQ_STARTED	0x034
NIU_MST_RD_REQ_STARTED	NIU_MST_RD_REQ_STARTED	0x038
NIU_MST_NON_POSTED_ATOMIC_STARTED	NIU_MST_NON_POSTED_ATOMIC_STARTED	0x03C
NIU_MST_ID_TRANSACTION_COUNT	NIU_MST_ID_TRANSACTION_COUNT	0x040
NIU_MST_ID_OUTGOING_WRITES_COUNT	NIU_MST_ID_OUTGOING_WRITES_COUNT	0x080
NIU_SLV_ATOMIC_RESP_SENT	NIU_SLV_ATOMIC_RESP_SENT	0x0C0
NIU_SLV_WR_ACK_SENT	NIU_SLV_WR_ACK_SENT	0x0C4
NIU_SLV_RD_RESP_SENT	NIU_SLV_RD_RESP_SENT	0x0C8
NIU_SLV_RD_DATA_WORD_SENT	NIU_SLV_RD_DATA_WORD_SENT	0x0CC
NIU_SLV_REQ_ACCEPTED	NIU_SLV_REQ_ACCEPTED	0x0D0
NIU_SLV_RD_REQ_RECEIVED	NIU_SLV_RD_REQ_RECEIVED	0x0D4
NIU_SLV_NON_POSTED_ATOMIC_RECEIVED	NIU_SLV_NON_POSTED_ATOMIC_RECEIVED	0x0D8

continues

Table 6 – continued from previous page

Instance Name	Register Type	Relative Offset
NIU_SLV_POSTED_ATOMIC_RECEIVED	NIU_SLV_POSTED_ATOMIC_RECEIVED	0x0DC
NIU_SLV_NON_POSTED_WR_DATA_WORD_RECEIVED	NIU_SLV_NON_POSTED_WR_DATA_WORD_RECEIVED	0x0E0
NIU_SLV_POSTED_WR_DATA_WORD_RECEIVED	NIU_SLV_POSTED_WR_DATA_WORD_RECEIVED	0x0E4
NIU_SLV_NON_POSTED_WR_REQ_RECEIVED	NIU_SLV_NON_POSTED_WR_REQ_RECEIVED	0x0E8
NIU_SLV_POSTED_WR_REQ_RECEIVED	NIU_SLV_POSTED_WR_REQ_RECEIVED	0x0EC
NIU_SLV_NON_POSTED_WR_REQ_STARTED	NIU_SLV_NON_POSTED_WR_REQ_STARTED	0x0F0
NIU_SLV_POSTED_WR_REQ_STARTED	NIU_SLV_POSTED_WR_REQ_STARTED	0x0F4
MST_IF_ID_TRANSACTION_COUNT_RTZ_NUM	MST_IF_ID_TRANSACTION_COUNT_RTZ_NUM	0x0F8
MST_IF_ID_TRANSACTION_COUNT_RTZ_DETECTED	MST_IF_ID_TRANSACTION_COUNT_RTZ_DETECTED	0x0FC
ROUTER_NIU_OUT_PORT_NIU_STATUS_LO	ROUTER_OUT_PORT_NIU_STATUS_LO	0x100
ROUTER_NIU_OUT_PORT_NIU_STATUS_HI	ROUTER_OUT_PORT_NIU_STATUS_HI	0x104
ROUTER_OUT_PORT_REMOTE_1_STATUS	ROUTER_OUT_PORT_REMOTE_STATUS	0x108
ROUTER_OUT_PORT_REMOTE_2_STATUS	ROUTER_OUT_PORT_REMOTE_STATUS	0x110
ROUTER_IN_PORT_NIU_STATUS	ROUTER_IN_PORT_STATUS	0x118
ROUTER_IN_PORT_REMOTE_1_STATUS	ROUTER_IN_PORT_STATUS	0x138
ROUTER_IN_PORT_REMOTE_2_STATUS	ROUTER_IN_PORT_STATUS	0x158
NIU_OUT_PORT_STATUS	NIU_OUT_PORT_STATUS	0x180
NIU_MEM_IF_OUT_STATUS	NIU_MEM_IF_OUT_STATUS	0x184
NIU_MST_IF_REQ_VC_STATUS	NIU_MST_IF_REQ_VC_STATUS	0x18C
NIU_MST_IF_RESP_VC_STATUS	NIU_MST_IF_RESP_VC_STATUS	0x1A4
NIU_SLV_IF_REQ_VC_STATUS	NIU_SLV_IF_REQ_VC_STATUS	0x1B4
NIU_SLV_IF_RESP_VC_STATUS	NIU_SLV_IF_RESP_VC_STATUS	0x1E4

(Note: RSVD entries @ 0x10C, 0x114, 0x128-0x134, 0x188, 0x1EC-0x1FC are omitted)

Register Definitions

(Registers related to NIU Master/Slave Interface Status, Router Status, Interrupts, and Debug)

Register: NIU_MST_ATOMIC_RESP_RECEIVED

Field Name	Bit Range	SW Access	Reset	Description
atomic_resp_received	[31:0]	r	-	-

Register: NIU_MST_WR_ACK_RECEIVED

Field Name	Bit Range	SW Access	Reset	Description
wr_ack_received	[31:0]	r	-	-

Register: NIU_MST_RD_RESP_RECEIVED

Field Name	Bit Range	SW Access	Reset	Description
rd_resp_received	[31:0]	r	-	-

Register: NIU_MST_RD_DATA_WORD_RECEIVED

Field Name	Bit Range	SW Access	Reset	Description
rd_data_word_received	[31:0]	r	-	-

Register: NIU_MST_CMD_ACCEPTED

Field Name	Bit Range	SW Access	Reset	Description
cmd_accepted	[31:0]	r	-	-

Register: NIU_MST_RD_REQ_SENT

Field Name	Bit Range	SW Access	Reset	Description
rd_req_sent	[31:0]	r	-	-

Register: NIU_MST_NON_POSTED_ATOMIC_SENT

Field Name	Bit Range	SW Access	Reset	Description
non_posted_atomic_sent	[31:0]	r	-	-

Register: NIU_MST_POSTED_ATOMIC_SENT

Field Name	Bit Range	SW Access	Reset	Description
posted_atomic_sent	[31:0]	r	-	-

Register: NIU_MST_NON_POSTED_WR_DATA_WORD_SENT

Field Name	Bit Range	SW Access	Reset	Description
non_posted_wr_data_word_sent	[31:0]	r	-	-

Register: NIU_MST_POSTED_WR_DATA_WORD_SENT

Field Name	Bit Range	SW Access	Reset	Description
posted_wr_data_word_sent	[31:0]	r	-	-

Register: NIU_MST_NON_POSTED_WR_REQ_SENT

Field Name	Bit Range	SW Access	Reset	Description
non_posted_wr_req_sent	[31:0]	r	-	-

Register: NIU_MST_POSTED_WR_REQ_SENT

Field Name	Bit Range	SW Access	Reset	Description
posted_wr_req_sent	[31:0]	r	-	-

Register: NIU_MST_NON_POSTED_WR_REQ_STARTED

Field Name	Bit Range	SW Access	Reset	Description
non_posted_wr_req_started	[31:0]	r	-	-

Register: NIU_MST_POSTED_WR_REQ_STARTED

Field Name	Bit Range	SW Access	Reset	Description
posted_wr_req_started	[31:0]	r	-	-

Register: NIU_MST_RD_REQ_STARTED

Field Name	Bit Range	SW Access	Reset	Description
rd_req_started	[31:0]	r	-	-

Register: NIU_MST_NON_POSTED_ATOMIC_STARTED

Field Name	Bit Range	SW Access	Reset	Description
non_posted_atomic_started	[31:0]	r	-	-

Register: NIU_MST_ID_TRANSACTION_COUNT

Field Name	Bit Range	SW Access	Reset	Description
id_transaction	[7:0]	r	-	-

Register: NIU_MST_ID_OUTGOING_WRITES_COUNT

Field Name	Bit Range	SW Access	Reset	Description
id_outgoing_writes	[7:0]	r	-	-

Register: NIU_SLV_ATOMIC_RESP_SENT

Field Name	Bit Range	SW Access	Reset	Description
atomic_resp_sent	[31:0]	r	-	-

Register: NIU_SLV_WR_ACK_SENT

Field Name	Bit Range	SW Access	Reset	Description
wr_ack_sent	[31:0]	r	-	-

Register: NIU_SLV_RD_RESP_SENT

Field Name	Bit Range	SW Access	Reset	Description
rd_resp_sent	[31:0]	r	-	-

Register: NIU_SLV_RD_DATA_WORD_SENT

Field Name	Bit Range	SW Access	Reset	Description
rd_data_word_sent	[31:0]	r	-	-

Register: NIU_SLV_REQ_ACCEPTED

Field Name	Bit Range	SW Access	Reset	Description
req_accepted	[31:0]	r	-	-

Register: NIU_SLV_RD_REQ_RECEIVED

Field Name	Bit Range	SW Access	Reset	Description
rd_req_received	[31:0]	r	-	-

Register: NIU_SLV_NON_POSTED_ATOMIC_RECEIVED

Field Name	Bit Range	SW Access	Reset	Description
non_posted_atomic_received	[31:0]	r	-	-

Register: NIU_SLV_POSTED_ATOMIC_RECEIVED

Field Name	Bit Range	SW Access	Reset	Description
posted_atomic_received	[31:0]	r	-	-

Register: NIU_SLV_NON_POSTED_WR_DATA_WORD_RECEIVED

Field Name	Bit Range	SW Access	Reset	Description
non_posted_wr_data_word_received	[31:0]	r	-	-

Register: NIU_SLV_POSTED_WR_DATA_WORD_RECEIVED

Field Name	Bit Range	SW Access	Reset	Description
posted_wr_data_word_received	[31:0]	r	-	-

Register: NIU_SLV_NON_POSTED_WR_REQ_RECEIVED

Field Name	Bit Range	SW Access	Reset	Description
non_posted_wr_req_received	[31:0]	r	-	-

Register: NIU_SLV_POSTED_WR_REQ_RECEIVED

Field Name	Bit Range	SW Access	Reset	Description
posted_wr_req_received	[31:0]	r	-	-

Register: NIU_SLV_NON_POSTED_WR_REQ_STARTED

Field Name	Bit Range	SW Access	Reset	Description
non_posted_wr_req_started	[31:0]	r	-	-

Register: NIU_SLV_POSTED_WR_REQ_STARTED

Field Name	Bit Range	SW Access	Reset	Description
posted_wr_req_started	[31:0]	r	-	-

Register: ROUTER_OUT_PORT_NIU_STATUS_LO

Field Name	Bit Range	SW Access	Reset	Description
in_port_0_vcs_rdy	[15:0]	r	-	-
in_port_1_vcs_rdy	[31:16]	r	-	-

Register: ROUTER_OUT_PORT_NIU_STATUS_HI

Field Name	Bit Range	SW Access	Reset	Description
in_port_2_vcs_rdy	[15:0]	r	-	-

Register: ROUTER_OUT_PORT_REMOTE_STATUS

Field Name	Bit Range	SW Access	Reset	Description
out_vcs_stall	[15:0]	r	-	-
out_vcs_full_credit	[31:16]	r	-	-

Register: ROUTER_IN_PORT_STATUS

Field Name	Bit Range	SW Access	Reset	Description
vc_0_state	[3:0]	r	-	-
vc_0_ports_flit_forwarded	[6:4]	r	-	-
vc_1_state	[19:16]	r	-	-
vc_1_ports_flit_forwarded	[22:20]	r	-	-

Register: MST_IF_ID_TRANSACTION_COUNT_RTZ_NUM

Field Name	Bit Range	SW Access	Reset	Description
id_transaction_count_rtz_num	[3:0]	r	-	-

Register: MST_IF_ID_TRANSACTION_COUNT_RTZ_DETECTED

Field Name	Bit Range	SW Access	Reset	Description
id_transaction_count_rtz_detected	[15:0]	r	-	-

Register: NIU_OUT_PORT_STATUS

Field Name	Bit Range	SW Access	Reset	Description
data_flit_output_last	[0:0]	r	-	-
data_flit_output_vc	[3:1]	r	-	-
out_vc_data_active	[11:4]	r	-	-

Register: NIU_MEM_IF_OUT_STATUS

Field Name	Bit Range	SW Access	Reset	Description
resp_valid	[0:0]	r	-	-
mem_rden	[4:1]	r	-	-
mem_wren	[8:5]	r	-	-
mem_port_busy	[12:9]	r	-	-

Register: NIU_MST_IF_REQ_VC_STATUS

Field Name	Bit Range	SW Access	Reset	Description
state	[2:0]	r	-	-
req_head_flit_metadata	[15:3]	r	-	-

Register: NIU_MST_IF_RESP_VC_STATUS

Field Name	Bit Range	SW Access	Reset	Description
state	[1:0]	r	-	-
mem_addr	[31:2]	r	-	-

Register: NIU_SLV_IF_REQ_VC_STATUS

Field Name	Bit Range	SW Access	Reset	Description
state	[2:0]	r	-	-
ongoing_req_cmd	[18:3]	r	-	-

Register: NIU_SLV_IF_RESP_VC_STATUS

Field Name	Bit Range	SW Access	Reset	Description
state	[3:0]	r	-	-
ongoing_write_resp	[4:4]	r	-	-
ongoing_atomic_resp	[5:5]	r	-	-
ongoing_read_req	[6:6]	r	-	-
curr_req_vc	[10:7]	r	-	-

3.4 Tensix Overlay Register Map Reference Guide

This document details the registers within the Overlay block, which is included inside both Tensix and Dispatch engines, broken down by sub-module. The Overlay block has several components, that include 8 RISC-V cores, DMA's , L1 and L2 Cache modules, ports to Tensix-L1 , etc.. The list of registers below is limited to general module control registers , RISC-V Debug extension registers, watchdog and cache control registers. **Instantiated Modules and Base Addresses:**

The base address of the instantiated modules within the Tensix memory space is **0x0300_0000**.

- **tt_cluster_ctrl:** **0x0300_0000** (Cluster Control and Scratch Registers)
- **tt_debug_module_apb:** **0x0300_A000** (RISC-V Debug Module Registers)
- **tt_cluster_coreX_wdt:** **0x0400_8000+** (Watchdog Timer, instantiated per core, X=0..7)
- **tt_cache_controller:** **0x0401_0000** (L2 Cache Controller Registers)

(Note: Base address for **tt_cluster_coreX_wdt** starts at **0x0400_8000** for Core 0 and increments by **0x1000** for each subsequent core, up to **0x0400_F000** for Core 7)

3.4.1 Address Map: tt_cluster_ctrl

Description: Cluster Control and Scratch Registers

Base Offset: 0x03000000

Register Instance Summary

Instance Name	Relative Offset
reset_vector_0	0x0
reset_vector_1	0x8
reset_vector_2	0x10
reset_vector_3	0x18
reset_vector_4	0x20
reset_vector_5	0x28
reset_vector_6	0x30
reset_vector_7	0x38
scratch_0	0x40
scratch_1	0x44
... (up to scratch_31)	...
scratch_31	0xBC
rocc_mem_chicken	0xC0
scatter_list_magic_num_lo	0xC4
scatter_list_magic_num_hi	0xC8
clock_gating	0xCC
clock_gating_hyst	0xD0
wb_pc_reg_c0	0xD4
wb_pc_reg_c1	0xDC
wb_pc_reg_c2	0xE4
wb_pc_reg_c3	0xEC
wb_pc_reg_c4	0xF4
wb_pc_reg_c5	0xFC
wb_pc_reg_c6	0x104
wb_pc_reg_c7	0x10C
wb_pc_ctrl	0x114
ecc_parity_control	0x118
ecc_parity_status	0x11C
noc_snoop_tl_master_cfg	0x120
asserts	0x124
prefetcher_control	0x128
bus_error_unit_data_c0	0x12C
bus_error_unit_data_c1	0x134
bus_error_unit_data_c2	0x13C
bus_error_unit_data_c3	0x144
bus_error_unit_data_c4	0x14C
bus_error_unit_data_c5	0x154
bus_error_unit_data_c6	0x15C
bus_error_unit_data_c7	0x164
l2_dir_errors_0	0x16C
l2_dir_errors_1	0x170
l2_dir_errors_2	0x174
l2_dir_errors_3	0x178
l2_banks_errors_0	0x17C
l2_banks_errors_1	0x180
... (up to l2_banks_errors_15)	...

continues on next page

Table 7 – continued from previous page

Instance Name	Relative Offset
l2_banks_errors_15	0x1B8
debug_dmactive	0x1BC
debug_dmactiveAck	0x1C0
t6_l1_csr	0x1C4
debug_snoop	0x1C8
overlay_info	0x1CC
global_counter_ctrl	0x1D0
sw_ras	0x1D4

Register Definitions

Register: reset_vector

Description: Reset vector for each core. (64-bit register accessed via two 32-bit reads/writes at offsets +0x0 and +0x4 relative to instance offset). Default value set to start of memory port.

Field Name	Bit Range	SW Access	Reset	Description
data	[51:0]	rw	0x0	Reset vector data.

Register: scratch

Description: Scratch register to be used as general purpose register (GPR).

Field Name	Bit Range	SW Access	Reset	Description
data	[31:0]	rw	0x0	GPR data.

Register: rocc_mem_chicken

Field Name	Bit Range	SW Access	Reset	Description
data	[31:0]	rw	0x2E	ROCC mem chicken bits

Register: scatter_list_magic_num

Description: 64-bit register accessed via two 32-bit reads/writes (_lo at +0xC4, _hi at +0xC8). When the scatter list value is greater than this number the scatter list FSM will be finished.

Field Name	Bit Range	SW Access	Reset	Description
data	[63:0]	rw	0xFFFFFFFF (lo)	Magic number data.
			0xFFFFFFFF (hi)	

Register: clock_gating

Field Name	Bit Range	SW Access	Access	Reset	Description
rocc	[0:0]	rw		0x0	ROCC clock gating enable bit.
idma	[1:1]	rw		0x0	iDMA clock gating enable bit.
cluster_ctrl	[2:2]	rw		0x0	Cluster control clock gating enable bit.
context_switch	[3:3]	rw		0x0	Context switch clock gating enable bit.
llk_intf	[4:4]	rw		0x0	LLK interface clock gating enable bit.
snoop	[5:5]	rw		0x0	Snoop clock gating enable bit.
global_cmdbuf	[6:6]	rw		0x0	Global command buffer clock gating enable bit.
l1_flex_client_idma	[7:7]	rw		0x0	iDMA L1 flex client port clock gating enable bit.
l1_flex_client_overl	[8:8]	rw		0x0	Overlay L1 flex client port clock gating enable bit.
fds	[9:9]	rw		0x0	Fast Dispatch Synchronization clock gating enable bit.

Register: clock_gating_hyst

Field Name	Bit Range	SW Access	Reset	Description
hyst	[6:0]	rw	0x8	Clock gating hysteresis.

Register: wb_pc_reg

Description: Rocket Core PC status register (64-bit accessed via two 32-bit reads at +0x0 and +0x4 relative to instance offset). Enough bits to cover all of TL1.

Field Name	Bit Range	SW Access	Reset	Description
pc	[57:0]	r	0x0	Core PC.

Register: wb_pc_ctrl

Field Name	Bit Range	SW Access	Ac-	Re-set	Description
capture_en	[0:0]	rw		0x1	Enable capturing of PC into registers. Enabled by default

Register: ecc_parity_control

Field Name	Bit Range	SW Access	Reset	Description
en	[0:0]	rw	0x0	ECC/Parity enable.

Register: ecc_parity_status

Field Name	Bit Range	SW Access	Ac-	Re-set	Description
context_switch_parity_error	[0:0]	rw		0x0	Context switch parity error status.
context_switch_parity_error_ind	[5:1]	r		0x0	Context switch parity error address.

Register: noc_snoop_tl_master_cfg

Field Name	Bit Range	SW Access	Re-set	Description
disable_	[0:0]	rw	0x0	Disables snooping logic from executing one-way cache coherency through the use of more performant inline writes to the L2 cache controller. If set, all snoops will cause invalidation, forcing cores to fetch from T6 L1 memory.

Register: asserts

Field Name	Bit Range	SW Access	Reset	Description
data	[31:0]	r	0x0	Hardware assertions.

Register: prefetcher_control

Field Name	Bit Range	SW Access	Access	Reset	Description
dcache_prefetcher_ctl	[7:0]	rw		0xFF	Enable or disable the data cache prefetcher (per core)
icache_prefetcher_ctl	[15:8]	rw		0xFF	Enable or disable the instruction cache prefetcher (per core)

Register: bus_error_unit_data

Description: Bus Error Unit Status per core (64-bit accessed via two 32-bit reads at +0x0 and +0x4 relative to instance offset).

Field Name	Bit Range	SW Access	Access	Reset	Description
icache_tl_bus_error	[0:0]	r		0x0	Instruction Cache TileLink Bus Error
dcache_tl_bus_error	[1:1]	r		0x0	Data Cache TileLink Bus Error
icache_ecc_correctable	[2:2]	r		0x0	Instruction Cache ECC Correctable Error
dcache_ecc_correctable	[3:3]	r		0x0	Data Cache ECC Correctable Error
dcache_ecc_uncorrectable	[4:4]	r		0x0	Data Cache ECC Uncorrectable Error
dcache_error_addr	[56:5]	r		0x0	Data Cache Error Address
beu_cause_reg	[59:57]	r		0x0	Bus Error Unit Cause Register

Register: l2_dir_errors

Field Name	Bit Range	SW Access	Reset	Description
error	[1:0]	rw	0x0	Correctable/uncorrectable error
error_index	[7:2]	r	0x0	Error address

Register: 12_banks_errors

Field Name	Bit Range	SW Access	Reset	Description
error	[1:0]	rw	0x0	Correctable/uncorrectable error
error_index	[9:2]	r	0x0	Error address

Register: debug_dmactive

Field Name	Bit Range	SW Access	Reset	Description
dmactive	[0:0]	r	0x0	

Register: debug_dmactiveAck

Field Name	Bit Range	SW Access	Reset	Description
dmactiveAck	[0:0]	rw	0x0	

Register: t6_l1_csr

Description: *(Definition not provided in source)*

Field Name	Bit Range	SW Access	Reset	Description
N/A				

Register: debug_snoop

Field Name	Bit Range	SW Access	Reset	Description
debug_tl_req_fifo_empty	[0:0]	r	0x0	
debug_tl_req_fifo_full	[1:1]	r	0x0	
debug_tl_req_in_progress	[2:2]	r	0x0	

Register: overlay_info

Field Name	Bit Range	SW Access	Reset	Description
dispatch_inst	[0:0]	r	0x0	
is_customer	[1:1]	r	0x0	
has_pll	[2:2]	r	0x0	
has_smn	[3:3]	r	0x0	
tensix_version	[9:4]	r	0x0	
noc_version	[15:10]	r	0x0	
overlay_version	[21:16]	r	0x0	
rsvd	[31:22]	rw	0x0	

Register: global_counter_ctrl

Field Name	Bit Range	SW Access	Reset	Description
global_counter_en	[0:0]	rw	0x1	Global counter enable.
global_counter_clear	[1:1]	rw	0x0	Global counter clear

Register: sw_ras

Field Name	Bit Range	SW Access	Access	Reset	Description
data	[0:0]	rw		0x0	Can trigger RAS event through software by writing to this register

3.4.2 Address Map: tt_debug_module_apb

Description: RISC-V Debug Module Registers

Base Offset: 0x0300a000

Register Instance Summary

Instance Name	Relative Offset
DATA_0	0x10
DATA_1	0x14
DATA_2	0x18
DATA_3	0x1C
DMCONTROL	0x40
STATUS	0x44
DMI_HARTINFO	0x48
HALTSUMMARY1	0x4C
HAWINDOW	0x54
ABSTRACTS	0x58
COMMAND	0x5C
ABSTRACTAUTO	0x60
PROGBUF_0	0x80
PROGBUF_1	0x84
... (up to PROGBUF_15)	...
PROGBUF_15	0xBC
STATUS2	0xC8
SBCS	0xE0
SBADDR0	0xE4
SBADDR1	0xE8
SBDATA0	0xF0
SBDATA1	0xF4
HALTSUMMARY0	0x100

Register Definitions

Register: DATA

Description: Abstract command data registers (128-bit register accessed via four 32-bit reads/writes: DATA_0 at +0x10, DATA_1 at +0x14, DATA_2 at +0x18, DATA_3 at +0x1C).

Field Name	Bit Range	SW Access	Reset	Description
dmi_data_0	[7:0]	rw	0x0	
dmi_data_1	[15:8]	rw	0x0	
dmi_data_2	[23:16]	rw	0x0	
dmi_data_3	[31:24]	rw	0x0	
dmi_data_4	[39:32]	rw	0x0	
dmi_data_5	[47:40]	rw	0x0	
dmi_data_6	[55:48]	rw	0x0	
dmi_data_7	[63:56]	rw	0x0	
dmi_data_8	[71:64]	rw	0x0	
dmi_data_9	[79:72]	rw	0x0	
dmi_data_10	[87:80]	rw	0x0	
dmi_data_11	[95:88]	rw	0x0	
dmi_data_12	[103:96]	rw	0x0	
dmi_data_13	[111:104]	rw	0x0	
dmi_data_14	[119:112]	rw	0x0	
dmi_data_15	[127:120]	rw	0x0	

Register: DMCONTROL

Field Name	Bit Range	SW Access	Reset	Description
dmactive	[0:0]	rw	0x0	Debug module active
ndmreset	[1:1]	rw	0x0	Debug module reset output
clrresethaltreq	[2:2]	w	0x0	Clear reset halt request
setresethaltreq	[3:3]	w	0x0	Set reset halt request
rsvd_0	[15:4]	r	0x0	Reserved
hartsello	[18:16]	rw	0x0	Hart select low
rsvd_1	[25:19]	r	0x0	Reserved
hasel	[26:26]	rw	0x0	Hart array select
rsvd_2	[27:27]	r	0x0	Reserved
ackhavereset	[28:28]	w	0x0	Acknowledge reset
rsvd_3	[29:29]	r	0x0	Reserved
resumereq	[30:30]	w	0x0	Resume request
haltreq	[31:31]	rw	0x0	Halt request

Register: STATUS

Field Name	Bit Range	SW Access	Reset	Description
dmi_dmstatus_version	[3:0]	r	2	Version
dmi_dmstatus_confstrp	[4:4]	r	0	Configuration string pointer valid
dmi_dmstatus_hasreset	[5:5]	r	1	Indicates whether the debug system supports a reset and halt request
dmi_dmstatus_authbusy	[6:6]	r	0	Indicates if authorization is busy
dmi_dmstatus_authentic	[7:7]	r	1	Indicates if the system is authenticated
dmi_dmstatus_anyhalted	[8:8]	r	0	Indicates if any components are halted
dmi_dmstatus_allhalted	[9:9]	r	0	Indicates if all components are halted
dmi_dmstatus_anyrunning	[10:10]	r	1	Indicates if any components are running
dmi_dmstatus_allrunning	[11:11]	r	1	Indicates if all components are running
dmi_dmstatus_anyunavailable	[12:12]	r	0	Indicates if any components are unavailable
dmi_dmstatus_allunavailable	[13:13]	r	0	Indicates if all components are unavailable
dmi_dmstatus_anynonexistent	[14:14]	r	0	Indicates if any components do not exist
dmi_dmstatus_allnonexistent	[15:15]	r	0	Indicates if all components do not exist
dmi_dmstatus_anyresume	[16:16]	r	1	Indicates if any resume acknowledge is present
dmi_dmstatus_allresume	[17:17]	r	1	Indicates if all resume acknowledges are present
dmi_dmstatus_anyhavebeenreset	[18:18]	r	0	Indicates if any components have been reset
dmi_dmstatus_allhavebeenreset	[19:19]	r	0	Indicates if all components have been reset
dmi_dmstatus_reserved	[21:20]	r	0	Reserved bits
dmi_dmstatus_impebreal	[22:22]	r	1	Indicates if an implicit ebreak has occurred

Register: DMI_HARTINFO

Field Name	Bit Range	SW Access	Reset	Description
dataaddr	[11:0]	r	0x0	Data address
datasize	[15:12]	r	0x0	Number of DATA registers
dataaccess	[16:16]	r	0x0	Data access type
rsvd_0	[19:17]	r	0x0	Reserved
nscratch	[23:20]	r	0x0	Number of scratch registers

Register: HALTSUMMARY1

Field Name	Bit Range	SW Access	Reset	Description
dmi_haltsum1	[31:0]	r	-	Halt summary 1

Register: HAWINDOW

Field Name	Bit Range	SW Access	Reset	Description
hawindow	[7:0]	rw	–	Hart array window

Register: ABSTRACTS

Description: Abstract command control/status

Field Name	Bit Range	SW Access	Access	Reset	Description
dmi_abstracts_datacount	[3:0]	r		4	Number of DATA registers
dmi_abstracts_reserved1	[7:4]	r		0	Reserved
dmi_abstracts_cmderr	[10:8]	r		0	Command error
dmi_abstracts_reserved2	[11:11]	r		0	Reserved
dmi_abstracts_busy	[12:12]	r		0	Indicates if the abstract command is busy
dmi_abstracts_reserved3	[23:13]	r		0	Reserved
dmi_abstracts_progbuFSIZE	[28:24]	r		16	Number of PROGBUF registers

Register: COMMAND

Field Name	Bit Range	SW Access	Reset	Description
dmi_command	[31:0]	rw	0	Abstract command register

Register: ABSTRACTAUTO

Description: Abstract command autoexec

Field Name	Bit Range	SW Access	Access	Reset	Description
dmi_abstractauto_execdata	[3:0]	rw		0	Abstract command data autoexec
dmi_abstractauto_reserved1	[15:4]	rw		0	Reserved
dmi_abstractauto_execprogbuf	[31:16]	rw		0	Abstract command progbuf autoexec

Register: PROGBUF

Description: Abstract command progbuf registers (512-bit register accessed via sixteen 32-bit reads/writes from PROGBUF_0 at +0x80 to PROGBUF_15 at +0xBC).

Field Name	Bit Range	SW Access	Reset	Description
dmi_progbuf_0	[7:0]	rw	0	
dmi_progbuf_1	[15:8]	rw	0	
...	...	...	...	...
dmi_progbuf_63	[511:504]	rw	0	

Register: STATUS2

Description: Debug module control/status register 2

Field Name	Bit Range	SW Access	Reset	Description
dmi_dmcs2_hgselect	[0:0]	rw	0	Select halt groups or external triggers
dmi_dmcs2_hgwrite	[1:1]	w	0	Write 1 to change halt groups
dmi_dmcs2_haltgroup	[6:2]	rw	0	Halt group
dmi_dmcs2_reserved	[10:7]	r	0	Reserved

Register: SBCS

Description: System bus access control and status

Field Name	Bit Range	SW Access	Reset	Description
sbcs_sbaccess8	[0:0]	rw	1	8-bit accesses supported
sbcs_sbaccess16	[1:1]	rw	1	16-bit accesses supported
sbcs_sbaccess32	[2:2]	rw	1	32-bit accesses supported
sbcs_sbaccess64	[3:3]	rw	1	64-bit accesses supported
sbcs_sbaccess128	[4:4]	rw	0	128-bit accesses supported
sbcs_sbaccess	[11:5]	rw	52	Bits in address
sbcs_sberror	[13:12]	rw	0	System bus error
sbcs_sbreadondata	[15:15]	rw	0	System bus read on data
sbcs_sbautoincrement	[16:16]	rw	0	System bus auto-increment address
sbcs_sbaccess	[19:17]	rw	2	System bus access size
sbcs_sbreadonaddr	[20:20]	rw	0	System bus read on data
sbcs_sbbusy	[21:21]	r	0	System bus access is busy
sbcs_sbbusyerror	[22:22]	rw	0	System bus busy error
sbcs_reserved	[28:23]	r	0	Reserved
sbcs_sbversion	[31:29]	r	1	System bus access version

Register: SBADDR0

Description: SBA address register 0

Field Name	Bit Range	SW Access	Reset	Description
dmi_sbaddr0	[31:0]	rw	0	

Register: SBADDR1

Description: SBA address register 1

Field Name	Bit Range	SW Access	Reset	Description
dmi_sbaddr1	[31:0]	rw	0	

Register: SBDATA0

Description: SBA data register 0

Field Name	Bit Range	SW Access	Reset	Description
dmi_sbdata0	[31:0]	rw	0	

Register: SBDATA1

Description: SBA data register 1

Field Name	Bit Range	SW Access	Reset	Description
dmi_sbdata1	[31:0]	rw	0	

Register: HALTSUMMARY0

Field Name	Bit Range	SW Access	Reset	Description
dmi_haltsum0	[31:0]	r	-	Halt summary 0

3.4.3 Address Map: wdt (Watchdog Timer)

Description: Watchdog Timer Registers (Instantiated per core)

Base Offset: 0x04008000 (Core 0), 0x04009000 (Core 1), ..., 0x0400F000 (Core 7)

Register Instance Summary (Offsets relative to instance base)

Instance Name	Relative Offset
CTRL	0x0
COUNT	0x8
SCALED_COUNT	0x10
FEED	0x18
KEY	0x1C
CMP	0x20

Register Definitions

Register: CTRL

Field Name	Bit Range	SW Access	Reset	Description
wdogscale	[3:0]	rw	0x0	WDT counter scale value. Scales the current cycle count (time) by count / 2 ^ wdogscale value
rsvd0	[7:4]	r	0x0	Reserved
wdogrsten	[8:8]	rw	0x0	Enables a sticky reset signal to outputted to top level, can be used for a second stage wdt
wdogzerocm	[9:9]	rw	0x0	Enable cycle counter to be reset to 0 when compare value has been reached
rsvd1	[11:10]	r	0x0	Reserved
wdogenalwa	[12:12]	rw	0x0	Enable WDT to always count. This has priority over wdogcore-awake
wdogcoreaw	[13:13]	rw	0x0	Enable WDT to only count when the core is not in reset
rsvd2	[18:14]	r	0x0	Reserved
wdogip0	[19:19]	rw	0x0	Pending interrupt

Register: COUNT

Field Name	Bit Range	SW Access	Reset	Description
wdogcount	[30:0]	rw	0x0	31 bit WDT counter registers

Register: SCALED_COUNT

Field Name	Bit Range	SW Access	Reset	Description
wdogs	[15:0]	rw	0x0	Scaled value of WDT counter. (i.e. count >> wdogscale)

Register: FEED

Field Name	Bit Range	SW Access	Reset	Description
wdogfeed	[31:0]	rw	0x0	Feed register. Writing 0xD09F00D will reset the counter value back to 0.

Register: KEY

Field Name	Bit Range	SW Access	Reset	Description
wdogkey	[31:0]	rw	0x0	Key register. Magic key (0x51F15E) must be written before write to any other config register. Must be written every time

Register: CMP

Field Name	Bit Range	SW Access	Reset	Description
wdogcmp0	[15:0]	rw	0x1000	Compare register. Value that gets compared to wdogs

3.4.4 Address Map: tt_cache_controller

Description: L2 Cache Controller Registers

Base Offset: 0x04010000

Register Instance Summary

Instance Name	Relative Offset
Configuration	0x0
Flush64	0x200
Flush32	0x240
Invalidate64	0x280
Invalidate32	0x2C0
FullInvalidate	0x300

Register Definitions

Register: Configuration

Field Name	Bit Range	SW Access	Reset	Description
banks	[7:0]	r	0x8	Number of banks in the cache
ways	[15:8]	r	0x8	Number of ways per bank
lgSets	[23:16]	r	0x5	Base-2 logarithm of the sets per bank
lgBlockBytes	[31:24]	r	0x6	Base-2 logarithm of the bytes per cache block

Register: Flush64

Description: Flush address register (64-bit accessed via two 32-bit writes at +0x200 (lo) and +0x204 (hi)).

Field Name	Bit Range	SW Access	Ac-	Re-set	Description
flush_addr	[63:0]	rw		-	Flush the physical address equal to the 64-bit written data from the cache

Register: Flush32

Field Name	Bit Range	SW Access	Ac-	Re-set	Description
flush_addr	[31:0]	rw		-	Flush the physical address equal to the 32-bit written data << 4 from the cache

Register: Invalidate64

Description: Invalidate address register (64-bit accessed via two 32-bit writes at +0x280 (lo) and +0x284 (hi)).

Field Name	Bit Range	SW Access	Ac-	Re-set	Description
invalidate_addr	[63:0]	rw		-	Invalidate the physical address equal to the 64-bit written data from the cache

Register: Invalidate32

Field Name	Bit Range	SW Access	Ac-	Re-set	Description
invalidate_addr	[31:0]	rw		-	Invalidate the physical address equal to the 32-bit written data << 4 from the cache

Register: FullInvalidate

Field Name	Bit Range	SW Access	Ac-	Re-set	Description
invalidate_cache	[31:0]	rw		-	Invalidate the entire L2 cache (does not invalidate L1 caches)